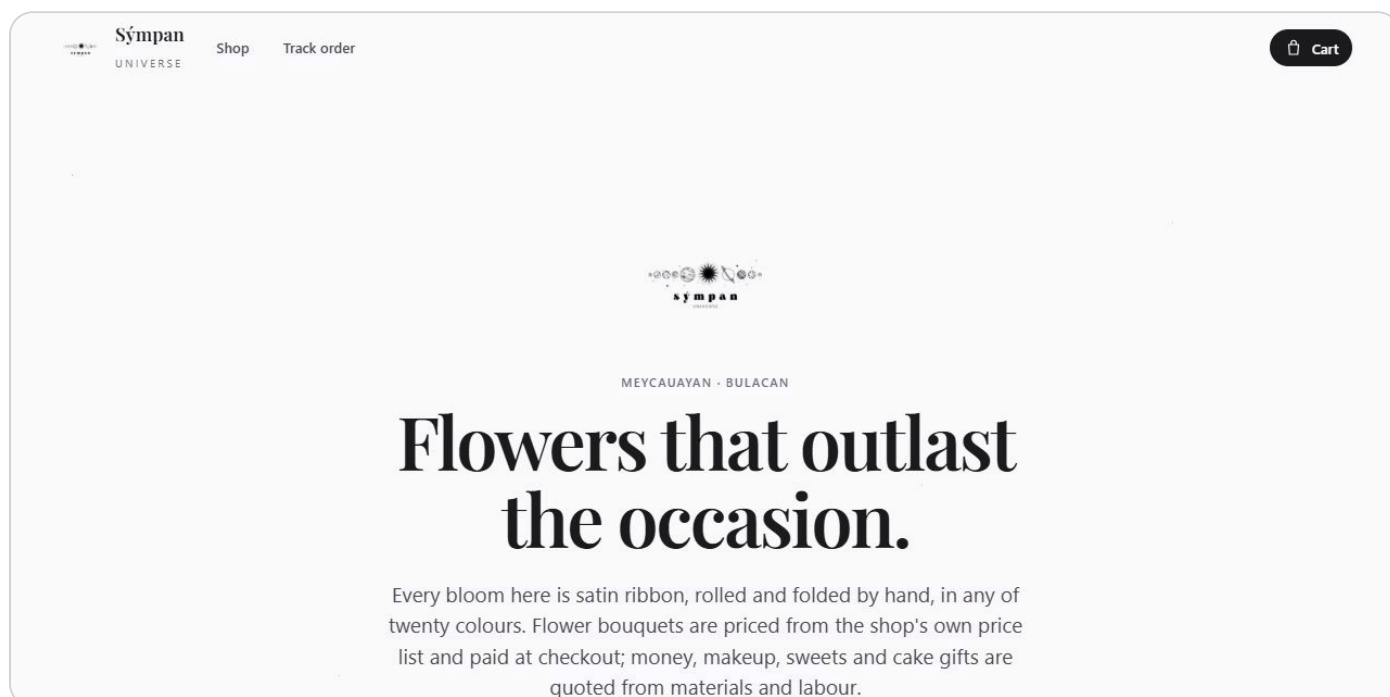


Sýmpan Universe Defense Reviewer

Our module-by-module manual: the problem each part solves, its input, process and output, the data structures and algorithms inside, their time and space complexity, why we chose them, how to demonstrate them live — and a plain-language walk through the code.



Part 1 · The shop window

1 Catalogue & Best Sellers · 2 Category Navigation · 3 Search & Sort

Part 2 · Choosing a gift

4 Flower Customiser · 5 Gift Customiser · 6 Cart

Part 3 · Checkout and payment

7 Checkout & Delivery · 8 Quotation Desk · 9 Payment, Receipts & Email

Part 4 · Behind the counter

10 Production & Tracking · 11 Order Records · 12 Order Desk & Accounts · 13 Security & Audit Logs

Sýmpan Universe — handcrafted ribbon bouquets and gift cakes, Lawa, Meycauayan, Bulacan · Built with HTML, CSS, Bootstrap 5 and JavaScript, with every data structure and algorithm written by hand · October 2026

How to use this reviewer

Every module has the same layout, in the same order the panel asks the questions:

1 • Business process

What task or problem of the client (Sýmpan Universe) the module solves.

2 • Input

The data the module needs — what the user types or picks, and the stored records it reads.

3 • Process

The data structure and algorithm used, step by step, with an Input → Process → Output picture.

4 • Output

What the module produces: records, screens, messages.

5 • Complexity analysis

Time and space complexity of each operation, and what n means.

6 • Justification

Why that data structure or algorithm, and why it fits the problem better than the alternatives.

7 • Demonstration

How to show it live, real screenshots, and a trace computed from the real code and data.

Code walkthrough

The actual functions from our files (with line numbers), a basic explanation of each step, and their time and space complexity.

Each of the four parts also has a **one-page summary** before its modules: a short pitch, the structures to be able to explain, and likely panel questions with answers. All numbers in the traces were computed by running our real JavaScript on the demo data — they match the screenshots.

Running the demo

1. Open `index.html` by double-clicking it (Edge or Chrome). No server is needed; the internet is used only for Bootstrap's stylesheet and for sending email.
2. The order desk is at `index.html#admin` — nothing on the shop links to it. Sign in with:

Role	E-mail	Password	Then
Owner	kurlchester31feliciano@gmail.com	admin123	the six-digit code sent to that address (shown on screen, clearly labelled, while EmailJS is not set up)

3. Sample order for *Track order*: SU-201 with mobile 0917 555 0142. For a payment, any 13-digit GCash reference not used before, e.g. 1234567890123.
4. **Reloading the page resets all data** to the demo history (the arrays live in memory). Demo dates are relative to today, so there are always orders due this week, quotations waiting, and so on.

5. "Undo last" in the order desk reverses the last "Mark ready".

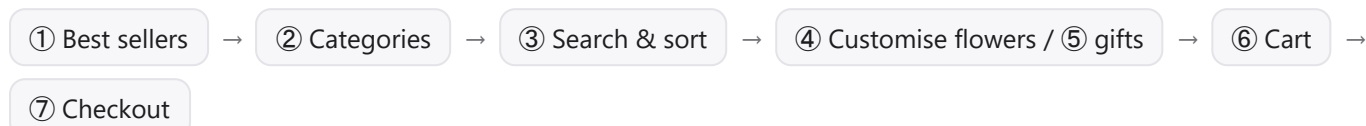
Contents

1. System overview and the project rules
2. Data structures and algorithms used — a primer
3. Part 1 · The shop window — Module 1 Catalogue & Best Sellers · Module 2 Category Navigation · Module 3 Search & Sort
4. Part 2 · Choosing a gift — Module 4 Flower Customiser · Module 5 Gift Customiser · Module 6 Cart
5. Part 3 · Checkout and payment — Module 7 Checkout & Delivery · Module 8 Quotation Desk · Module 9 Payment, Receipts & Email
6. Part 4 · Behind the counter — Module 10 Production & Tracking · Module 11 Order Records · Module 12 Order Desk & Accounts · Module 13 Security & Audit Logs
7. Appendix — complexity summary, general panel questions, glossary, file map

System overview

Sýmpan Universe makes handcrafted ribbon bouquets and gift cakes in Lawa, Meycauayan, Bulacan. Without a system, every order is a chat thread: the owner prices each request by hand, keeps payments and pickup dates in notes, and answers "is my order ready?" one message at a time. The website handles the customer's whole order — browsing, customising, checkout, payment, tracking — and the owner's work behind it.

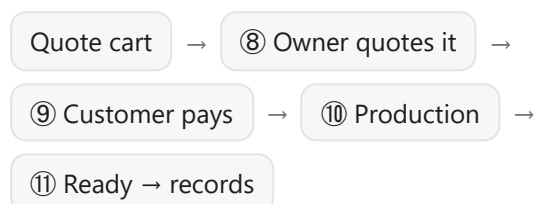
The customer and owner flow



A cart of flowers or picture bouquets (priced from the price list)



A cart with a money, makeup, sweets, diaper or beer gift (quoted by the owner)



⑫ The owner and staff each sign in with their own account (password + emailed code); the owner watches the Overview by day, month or year, edits orders and products, and adds or disables staff accounts. ⑬ Every sign-in attempt and every change is in the security and audit logs; a forgotten password is reset with an emailed code. ⑭ Courier deliveries get the courier's tracking number or link.

How the program is built

One page, many screens

Arrays live in the browser's memory, and memory is wiped whenever a new page opens — data cannot transfer between separate HTML files. So the whole site is one `index.html`: the shop and the order desk (`#admin`) are sections shown and hidden. The arrays survive every screen change.

Layers

`js/core.js`, `structures.js`, `algorithms.js` — Our own array, text and date helpers; the linked list, circular queue, min-heap, stack, n-ary tree and hash table; the sorts, searches, recursion and greedy algorithm.

`js/data.js`, `store.js`, `orders.js`, `notify.js`, `editing.js`, `reports.js` — The "database" (arrays) and the business rules: the catalogue, cart, checkout, quotations, payments, production, records, emails and reports.

`js/ui.js`, `motion.js`, `mail.js`, `receipt.js` — Screens, sheets, animation, the email sender (EmailJS) and receipts.

`js/shop-*.js`, `admin-*.js`, `app.js` — One script per screen: the shop, and the order desk at `index.html#admin`.

The data

Array / structure	Key	How it stays in order	Used by modules
orders	order number (201, 202 ...)	numbers only grow → append, $O(1)$; the array stays sorted for binary search	1, 7–12
products	product id (101 ...)	the fixed catalogue, edited in place	1–5, 12
receipts, outbox	receipt / email number	append, $O(1)$	9, 12
basket	position	singly linked list, in the order added	6
quoteQueue, standardLane, mailQueue	order id / outbox position	circular queues, first in first out	8, 9, 10
rushLane	(due date, arrival)	min-heap	10
completedStack	{ id, position }	stack, last in first out	11
categoryTree	slug	n-ary tree stored in one array	2
referenceIndex, areaIndex	"flower arrangement colour"; city name	hash tables with chaining	4, 7
auditLog	entry number (time order)	append-only array: entries are only added at the end, $O(1)$	13 (written by every module)
failedSignIns	the email typed	hash table with chaining	13
staffAccounts	account id (1, 2 ...)	the credentials array: ids only grow → append, $O(1)$; sorted for binary search	12
staffEmailIndex	the account's email	hash table email → account id	12

The project rules and how we met them

Rule from our instructor	How the code follows it
HTML, CSS, Bootstrap and JavaScript only; JS is the "backend"	No server, no framework, no database program. Bootstrap 5.3 gives the grid; everything else is our own code.
Data cannot transfer between pages (arrays)	One page, index.html: the shop and the order desk (#admin) are shown and hidden, so the arrays survive every screen change. No localStorage, no cookies, no server. A reload starts again from the demo data.
Procedural, not OOP	Only functions and plain records: no class, this, prototype or new (except new Date once, to read the clock). A queue is the record { items, head, tail, count, capacity } plus functions like cqEnqueue(queue, value).
No built-in helpers: push, pop, shift, unshift, splice, slice, concat, sort, reverse, indexOf, includes, find, filter, map, forEach, reduce, some, every, join; split, replace, trim, toLowerCase, toUpperCase, padStart, substring; no regular expressions	All replaced by our own functions in js/core.js, js/structures.js and js/algorithms.js — listAdd, copyRange, keepWhere, firstWhere, cutText, strip, toLower, insertionSort, binarySearch ... tests/rubric.js scans every script and fails on any of them.

Rule from our instructor	How the code follows it
State the time and space complexity, using only $O(1)$, $O(\log n)$, $O(n)$ and $O(n^2)$	Written above every function in js/. The rubric fails on a missing note or on any other notation — which is why there is no merge sort: it is $O(n \log n)$.
Use Date only to read the clock	readClock() holds the only new Date. Adding days, weekdays, month lengths and leap years are our own day-number arithmetic (dayNumber, dateOfDayNumber in js/core.js).

Built-ins that are used are only for input and output: the page itself (the DOM), Date (to read the clock), Math, Number, String, charAt and charCodeAt, the length of an array or text, timers, crypto.getRandomValues (the six-digit codes and the password salts), and fetch with JSON.stringify to hand an email to EmailJS.

Data structures & algorithms primer

n = the size of whatever an operation goes through (records, items in a queue, or the characters of a text). Only the four notations from class are used: $O(1)$, $O(\log n)$, $O(n)$, $O(n^2)$. All of these live in `js/structures.js` and `js/algorithms.js`.

Array (our "table")

201

202

203

204

[0]

[1]

[2]

[3] new

listAdd writes at index length: $O(1)$. Order numbers only grow, so appending keeps the orders array sorted.

Reading `a[i]`: $O(1)$.

Linear search — $O(n)$

Rose

Plumeria

Dahlia

Sunflower

1st

2nd

match

Money

Check the items one by one until one matches. Works on any array, sorted or not. Best case $O(1)$, worst $O(n)$. The search box and many look-ups use it.

Binary search — $O(\log n)$

203

204

205

206

207

208

middle

Needs a sorted array. Look at the middle; if the target is bigger keep the right half, else the left. Each step halves the range: 22 orders need at most 5 steps. Finds an order by number.

Insertion sort — best $O(n)$, worst $O(n^2)$, stable

Beer

Dahlia

Rose

Money

held

Take the next item and slide the larger items before it one place right until its place is found. Very fast when the list is already almost in order — ours usually are — so it is our general sort.

Selection sort — always $O(n^2)$

Rose 7

Dahlia 3

Money 5

Diaper 2

largest

For each position, scan the rest for the item that belongs there and move it in. Always $n(n-1)/2$ comparisons. Ranks the best sellers; ours shifts instead of swapping, so it is stable.

Naive string matching — $O(n^2)$

m

a

t

c

h

a

.

r

start?

Try the search word at every starting position of the text and compare letter by letter. "matcha" finds the four flower bouquets through their colour names.

Singly linked list — $O(1)$ append

Rose

→

Picture

→

Money

head

tail

Nodes in one array, each with the index of the next.

Adding at the tail is $O(1)$; removing re-points one link. The cart.

Stack — LIFO, push / pop $O(1)$

SU-214

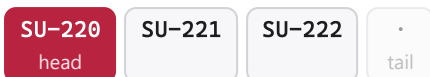
SU-215

SU-216

top

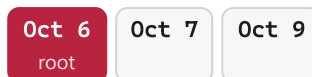
Record { items, top }. "Mark ready" pushes { order, its place }; Undo pops the newest. Also the depth-first walks of the category tree.

Circular queue — FIFO, $O(1)$



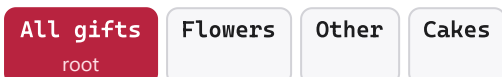
Record { items, head, tail, count, capacity }; the tail moves to $(\text{tail} + 1) \% \text{capacity}$, so nothing is shifted. Quote requests, standard production and outgoing email.

Min-heap — $O(\log n)$



An array where every parent comes before its children; the earliest due date is always at index 0. Insert sifts up, extract sifts down: $O(\log n)$. The rush lane.

N-ary tree — $O(n)$ walk



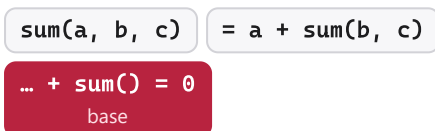
Each node keeps its parent and its children's indices. The shop's three columns and their ten lines; "everything under a branch" is its leaves.

Hash table — $O(1)$ average



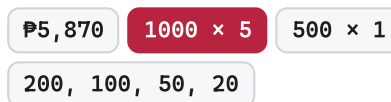
FNV-1a turns a key into a bucket number; keys that share a bucket (a collision) form a short chain. The 160 photographs and the delivery areas.

Recursion — $O(n)$



A function that calls itself on the rest of the list, ending at the empty list. Adds up the add-ons and the quote lines.

Greedy — $O(n)$



Take the largest note that fits, then the next. For the peso this always gives the fewest bills. The money bouquet.

Why procedural?

Every structure is a plain record plus functions that receive it — `stackPush(stack, value)`, `cqEnqueue(queue, value)`, `hashPut(table, key, value)` — instead of objects with methods. This follows the project rule and makes each step of the algorithm visible.

PART 1

The shop window

Module 1 Catalogue & Best Sellers · Module 2 Category Navigation · Module 3 Search & Sort

60-second pitch

These modules are what a customer sees first. Catalogue & Best Sellers ranks the products by the pieces really sold — one pass over the paid orders with a hash table, then a selection sort. Category Navigation stores the shop's three columns as an n-ary tree, so "everything under Flower Bouquets" is a depth-first walk with a stack. Search & Sort finds words anywhere in a product with a linear search and naive string matching, and orders the results with a stable insertion sort.

Structures and algorithms to be able to explain

Where	Structure / algorithm	Time	Space
Count pieces sold	Hash table (product id → tally row) over the order lines	$O(n^2)$ — every line of every order	$O(n)$
Rank the best sellers	Selection sort (stable version)	$O(n^2)$ always	$O(n)$ copy
Branch → its lines	N-ary tree, depth-first search with a stack	$O(n)$	$O(n)$
Search box	Linear search + naive string matching	$O(n^2)$	$O(n)$
Sort menu	Insertion sort (stable)	$O(n)$ best, $O(n^2)$ worst	$O(n)$ copy

Likely panel questions

Why selection sort for the best sellers?

For ten products any of the sorts from class is instant. Selection sort always makes $n(n-1)/2 = 45$ comparisons — a fixed cost that is easy to show — and each pass answers the business question "which product is the biggest seller left?". Ours shifts instead of swapping, so it is stable: equal sellers keep their catalogue order.

Why a hash table to count sales?

Without it, the count went through every order once for each product. With a row per product found by `hashGet` in $O(1)$ on average, one pass over the orders and their lines does it.

Why a tree for the categories instead of a list?

The categories are a hierarchy. "All lines under this branch" becomes "all leaves under this node", with no list of which line belongs where written into the page. Adding a line is adding one leaf.

How does the tree search work without recursion?

`treeFind` and `treeLeaves` use our own stack: push the root, pop a node, push its children — depth-first, each node visited once, $O(n)$.

Why not binary search for the search box?

Binary search needs an exact key in a sorted array. The search box asks "does this word appear anywhere in the name, materials or colours?", which only a scan of each record can answer.

Why is the text search $O(n^2)$?

textHas tries the word at every starting position of the text and compares letter by letter — a loop inside a loop. Run over a whole table, that is the $O(n^2)$ naive string matching from class.

Catalogue & Best Sellers

Screens	Home page — the "Best sellers" strip and every product card; the Overview's best sellers (order desk)
Files	js/store.js (salesTally, salesRank, topSellers) · js/algorithms.js (selectionSortDesc) · js/shop-catalog.js
Data	orders — array sorted by order number; products — 10 records; a hash table product id → tally row
DSA	Hash table · Selection sort

1 Business process

What task or problem from the client are you solving?

The shop needs a front window: every bouquet and cake the customer can order, with its photo, its price label and how many have sold. Customers trust what other customers buy, so the products that sell the most are lifted into a "Best sellers" strip at the top.

The owner used to decide by hand what to call a best seller. Now the ranking is worked out from the real paid orders: buy ten roses and the rose moves up at once. Cancelled (voided) orders and requests that were never paid do not count.

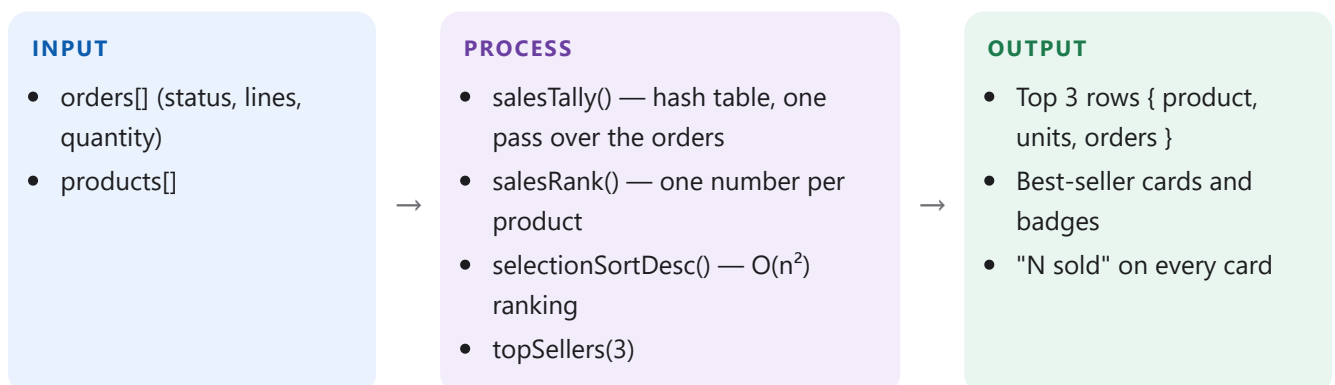
2 Input

What data is needed by the module?

Input	What it is
Stored: orders	Every order record — its status (only paid and completed orders count) and its lines, each with a product id and a quantity.
Stored: products	The catalogue — name, price list, photos, and whether the product is active.
Customer picks	Nothing is typed: the strip is built when the page opens and after every sale.

3 Process

What data structure and/or algorithm did you use? How does it work step by step?



Step by step (Hash table · Selection sort):

- 1 Tally: make one row per product and put each row in a hash table keyed by product id. Then walk every paid or completed order once; each of its lines finds its product's row with one hash look-up and adds its pieces (units). An order is counted once per product.
- 2 Give each product a rank key: $\text{units} \times 100000 + \text{orders}$, so pieces sold decide first and the number of orders only breaks a tie.
- 3 Selection sort, descending: for position 0, scan the rest of the list for the largest key and move it to the front; repeat for position 1, 2, ...
- 4 Keep the first three that are active and have sold at least one piece; draw them as cards with a "Best seller" badge.

4 Output

What result does the module produce?

- The "Best sellers" strip on the home page (three cards).
- A "Best seller" badge on those products in the catalogue and on the product sheet.
- The "N sold" count on every card, and the "Best selling" order of the catalogue sort.
- The Best sellers panel in the order desk Overview (top five, for the chosen day, month or year).

5 Complexity analysis

What is the time complexity? What is the space complexity?

Operation	Time	Space	Note
<code>salesTally()</code>	$O(n^2)$ — each line of each order	$O(n)$	Counts, for every product, how many pieces were sold and in how many orders.
<code>salesRank()</code>	$O(1)$	$O(1)$	Turns a row into one number so the sort can compare rows.
<code>selectionSortDesc()</code>	$O(n^2)$	$O(n)$	The selection sort itself — largest key first, and stable.
<code>topSellers()</code>	$O(n^2)$	$O(n)$	Ranks a tally and keeps the first k that are active and have sold.
<code>renderBestSellers()</code>	$O(n^2)$	$O(n)$	Draws the strip.

What n means here: For the sort, n = number of products (10). For the tally, n = the order lines it reads — every line of every paid order.

Each Time and Space is read from the comment above the function in the source, where every function in `js/` states its own cost.

6 Justification

Why did you choose that data structure or algorithm? Why is it appropriate?

Selection sort: the list being ranked is tiny — ten products — so its $O(n^2)$ costs at most 45 comparisons. It is the clearest sort to explain and trace: each pass picks "the biggest one left". It is written to be stable (it shifts instead of swapping), so two products with the same sales keep their catalogue order instead of trading places on every refresh.

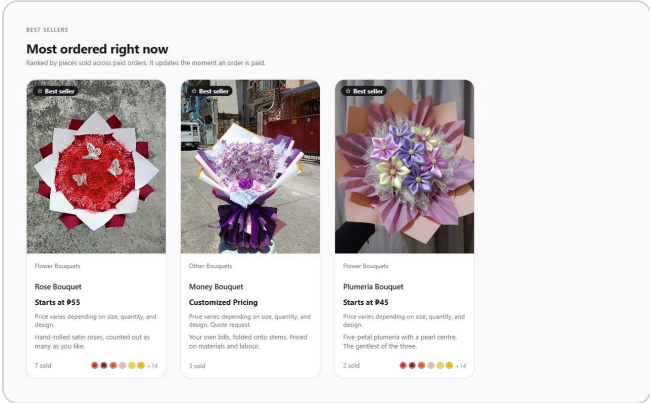
Hash table for the tally: counting product by product meant reading every order once per product. A row per product, found by its id in $O(1)$ on average, lets one pass over the orders do the whole count.

A hand-kept "best seller" flag would go stale. Counting from the paid orders means it is always right, and a voided order drops out by itself.

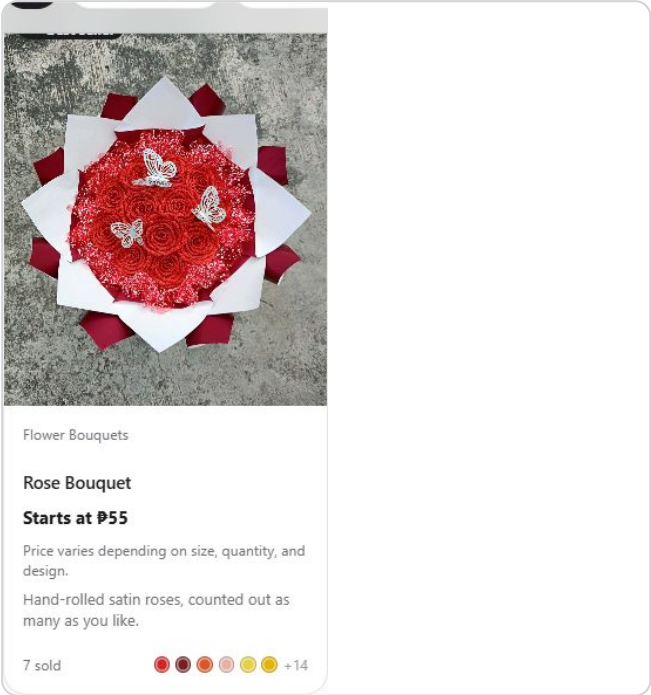
7 **Demonstration**

Show that the module works correctly — the input, the process and the output.

- 1 Open index.html. Point at the "Best sellers" strip and the "N sold" labels.
- 2 Add 10 Plumeria bouquets to the cart and check out (pay in full, any 13-digit reference such as 1234567890123).
- 3 Scroll back up: Plumeria has jumped into first place, because 10 pieces beat the rose's 7.
- 4 In the order desk, void that order: Plumeria drops back. Cancelled orders do not count.



The best-seller strip, ranked from paid orders



A product card with its "sold" count

Trace — the best-seller ranking (computed by our code)

Input — salesTally() over the 22 demo orders (catalogue order)

One pass over the paid and completed orders and their lines; each line finds its product's row with one hash-table look-up (hashGet by product id), so the orders are read once, not once per product.

#	Product	Orders	Units	Rank key
0	Rose Bouquet	6	7	700006
1	Plumeria Bouquet	2	2	200002
2	Dahlia Bouquet	1	1	100001
3	Sunflower Bouquet	0	0	0
4	Money Bouquet	3	3	300003
5	Makeup Bouquet	1	1	100001
6	Picture Bouquet	1	1	100001
7	Sweets & Snacks Bouquet	0	0	0
8	Diaper Cake	1	1	100001
9	Beer-in-Can Cake	1	1	100001

Process — selectionSortDesc(tally, salesRank)

The rank function was called 90 times: two reads per comparison, so 45 comparisons for $n = 10$ — exactly $n(n-1)/2 = 45$, the $O(n^2)$ of selection sort.

Rank	Product	Units	Orders
1	Rose Bouquet	7	6
2	Money Bouquet	3	3
3	Plumeria Bouquet	2	2
4	Dahlia Bouquet	1	1
5	Makeup Bouquet	1	1
6	Picture Bouquet	1	1
7	Diaper Cake	1	1
8	Beer-in-Can Cake	1	1
9	Sunflower Bouquet	0	0
10	Sweets & Snacks Bouquet	0	0

Output — bestSellers(3)

1. Rose Bouquet (7 sold) 2. Money Bouquet (3 sold) 3. Plumeria Bouquet (2 sold)

Then — placeOrder() for 10 Plumeria bouquets, paid in full

bestSellers(3) is now: 1. Plumeria Bouquet (12 sold) 2. Rose Bouquet (7 sold) 3. Money Bouquet (3 sold). Ten pieces moved Plumeria to first place at once.

Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

js/store.js · lines 593–617 · salesTally() — Counts, for every product, how many pieces were sold and in how many orders.

```
593  /* Orders and units per product, optionally only for orders a test
594     accepts (a date range). One row per product, in catalogue order; a
595     hash table keyed by product id finds a line's row in O(1) average, so
596     every order is read once instead of once per product.
597                                     Time O(n²) – each line of each order · Space O(n) */
598  function salesTally(include) {
599  ❶    var out = [], rowFor = hashCreate(31);
600      for (var p = 0; p < products.length; p++) {
601          var row = { product: products[p], orders: 0, units: 0, lastOrder: -1 };
602          listAdd(out, row);
603  ❷    hashPut(rowFor, String(products[p].id), row);
604      }
605      for (var o = 0; o < orders.length; o++) {
606  ❸    if (!countsAsSale(orders[o]) || (include && !include(orders[o]))) continue;
607          var items = orders[o].items;
608          for (var i = 0; i < items.length; i++) {
609  ❹    var hit = hashGet(rowFor, String(items[i].productId));
610          if (hit === null) continue;
611  ❺    hit.units += items[i].quantity;
612          // Count the order once, however many of its lines are this product.
613  ❻    if (hit.lastOrder !== o) { hit.orders++; hit.lastOrder = o; }
614      }
615      }
616      return out;
617  }
```

- ❶ Line 599 A list of rows, and a hash table to find a row by product id.
- ❷ Line 603 Make one row per product and index it by the product id.
- ❸ Line 606 Skip orders that are not paid or completed — voided and unpaid ones never count.
- ❹ Line 609 Each line finds its product's row with one hash look-up.
- ❺ Line 611 Add the line's pieces to that product.
- ❻ Line 613 Count the order once per product, however many lines it has.

Time O(n²) – each line of each order · Space O(n)

js/store.js · lines 631–636 · salesRank() — Turns a row into one number so the sort can compare rows.

```
631  /* The rank key: total units sold first, then the number of orders to
632     break a tie. Buying more of a product counts every piece, so a large
633     order moves it up the ranking at once. Time O(1) · Space O(1) */
634  function salesRank(row) {
635  ❶    return row.units * 100000 + row.orders;
636  }
```

- ❶ Line 635 Units dominate; the order count only matters when units tie.

Time O(1) · Space O(1)

js/algorithms.js · lines 62–78 · selectionSortDesc() — The selection sort itself — largest key first, and stable.

```
62  function selectionSortDesc(records, countOf) {
63  ❶    var work = copyArray(records);
64  ❷    for (var i = 0; i < work.length - 1; i++) {
65        var best = i;
66        for (var j = i + 1; j < work.length; j++) {
67            // Strictly greater, so ties keep their catalogue order.
68  ❸    if (countOf(work[j]) > countOf(work[best])) best = j;
69        }
70        if (best !== i) {
71            var held = work[best];
72            // Shift rather than swap, so the sort stays stable.
73  ❹    for (var k = best; k > i; k--) work[k] = work[k - 1];
74  ❺    work[i] = held;
75        }
76    }
77    return work;
78 }
```

- ❶ line 63 Sort a copy, so the original tally is untouched.
- ❷ line 64 Pass i fills position i.
- ❸ line 68 Scan everything after i for the largest key. Strictly greater keeps ties in their old order.
- ❹ line 73 Shift the items between i and best one place right...
- ❺ line 74 ...and drop the largest into position i. Shifting (not swapping) is what makes it stable.

Time $O(n^2)$ · Space $O(n)$

js/store.js · lines 645–654 · topSellers() — Ranks a tally and keeps the first k that are active and have sold.

```
645  /* Rank any tally – the whole history, or one day, month or year.
646                                     Time  $O(n^2)$  · Space  $O(n)$  */
647  function topSellers(tally, k) {
648  ❶    var ranked = selectionSortDesc(tally, salesRank);
649        var out = [];
650        for (var i = 0; i < ranked.length && out.length < k; i++) {
651  ❷    if (ranked[i].units > 0 && ranked[i].product.active) listAdd(out, ranked[i]);
652        }
653        return out;
654 }
```

- ❶ line 648 Sort the rows by the rank key.
- ❷ line 651 Leave out disabled products and products nobody has bought.

Time $O(n^2)$ · Space $O(n)$

js/shop-catalog.js · lines 72–84 · renderBestSellers() — Draws the strip.

```
72  /*                                     Time  $O(n^2)$  · Space  $O(n)$  */
73  function renderBestSellers() {
74  ❶    var top = bestSellers(3), ids = [];
75        for (var i = 0; i < top.length; i++) listAdd(ids, top[i].product.id);
76        var grid = $('#bestSellers');
77        if (top.length === 0) {
78            grid.innerHTML = '<p class="t-callout dim">No paid orders yet – the first ones will appear here.
79        </p>';
80            return;
81        }
82        var rows = [];
83        for (var k = 0; k < top.length; k++) listAdd(rows, { product: top[k].product, orders: top[k].orders,
84            units: top[k].units });
85  ❷    grid.innerHTML = renderEach(rows, function (row) { return productCard(row, ids); });
```

- ❶ line 74 Ask for the top three.
- ❷ line 83 Render each one as a product card with the badge.

Time $O(n^2)$ · Space $O(n)$

Category Navigation

Screens	Catalogue — the branch chips (All, Flower Bouquets, Other Bouquets, Gift Cakes) and their line chips
Files	js/structures.js (treeAdd, treeFromOutline, treeFind, treeLeaves, stack) · js/data.js (CATEGORY_OUTLINE) · js/shop-catalog.js
Data	categoryTree — 14 nodes in one array { label, slug, parent, children, depth }; CATEGORY_OUTLINE
DSA	N-ary tree · Stack

1 Business process

What task or problem from the client are you solving?

The shop's own order sheet groups everything in three columns: Flower Bouquets, Other Bouquets and Gift Cakes, each with its own lines (Rose, Plumeria, Money, Diaper Cake...). Customers browse the same way.

Picking a branch such as "Flower Bouquets" must show every line under it, and picking a line such as "Rose" must show only that line — without the page hard-coding which lines belong where.

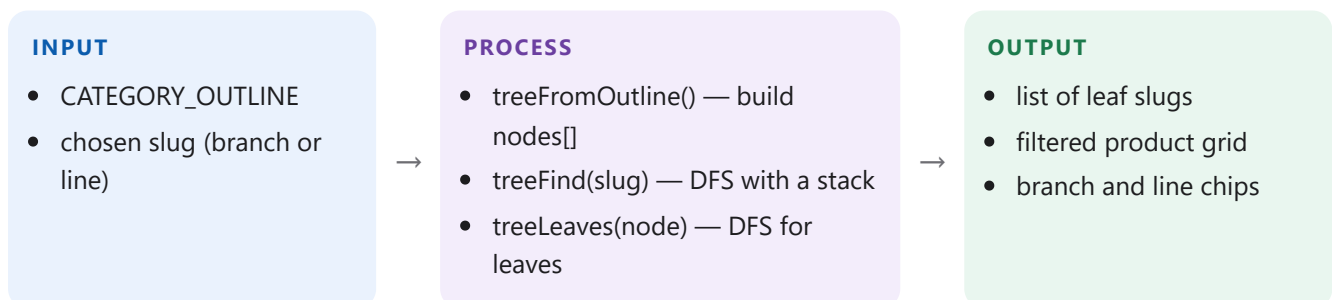
2 Input

What data is needed by the module?

Input	What it is
Stored: CATEGORY_OUTLINE	The three branches and their lines, from the order sheet (js/data.js).
Customer picks	A branch chip (e.g. "Gift Cakes"), then optionally a line chip (e.g. "Diaper Cake").
Stored: products	Each product's line (its leaf in the tree).

3 Process

What data structure and/or algorithm did you use? How does it work step by step?



Step by step (N-ary tree · Stack):

- 1 Build the tree once: a root, three branch nodes, ten leaf nodes. Each node stores its parent's index and its children's indices in one array.
- 2 When a chip is pressed, find that node by its slug with a depth-first search that uses our stack.
- 3 Collect every leaf under it, again depth-first with a stack.
- 4 Show only the products whose line is one of those leaves.

4 Output

What result does the module produce?

- The branch chips and, after a branch is picked, its line chips.
- The catalogue filtered to the chosen branch or line, with the "N of 10 items match" count.
- The branch name shown on every card ("Flower Bouquets").

5 Complexity analysis

What is the time complexity? What is the space complexity?

Operation	Time	Space	Note
<code>treeAdd()</code>	$O(1)$	$O(1)$	Adds one node under a parent.
<code>treeFromOutline()</code>	$O(n)$	$O(n)$	Builds the whole tree from the order sheet outline.
<code>treeFind()</code>	$O(n)$	$O(n)$	Finds a node by slug with our stack (depth-first).
<code>treeLeaves()</code>	$O(n)$	$O(n)$	Collects every leaf under a node.
<code>catalogRows()</code>	$O(n^2)$	$O(n)$	Applies the tree filter (then search and sort — module 3).

What n means here: n = number of nodes in the tree (14: the root, 3 branches, 10 lines). Each node is visited at most once.

Each Time and Space is read from the comment above the function in the source, where every function in `js/` states its own cost.

6 Justification

Why did you choose that data structure or algorithm? Why is it appropriate?

The categories are a hierarchy, and a tree is the structure of a hierarchy. "All lines under Flower Bouquets" becomes "all leaves under this node" — the filter needs no list of which lines belong to which branch.

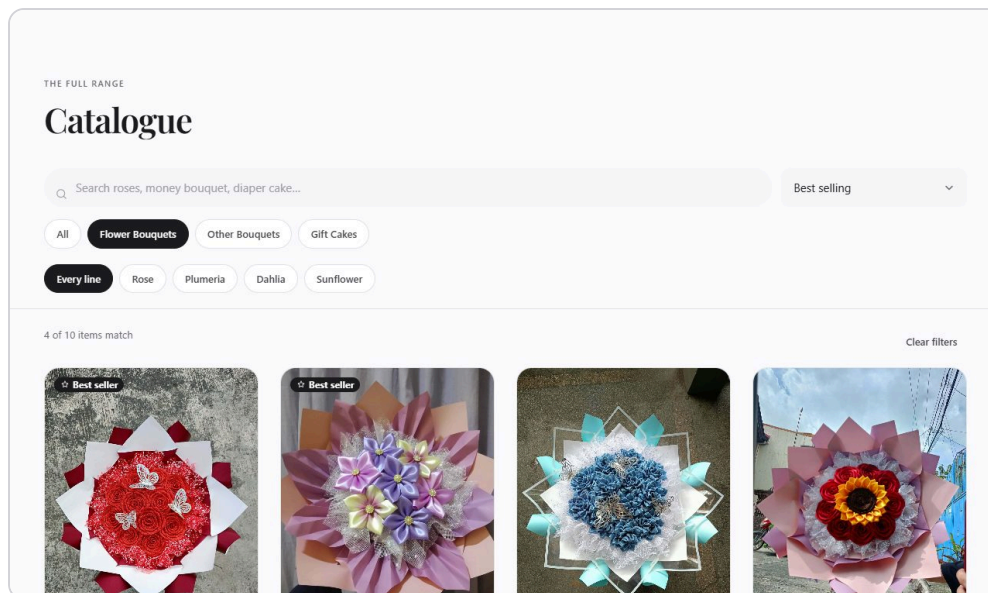
Adding a line (say, "Tulip") is one new leaf; nothing else changes. A flat list of strings would need the branch written on every product and special cases for "show everything under".

The walks use our own stack instead of recursion, which shows the depth-first order step by step. Storing children as index arrays keeps it a plain array of records, matching the procedural, array-only rule.

7 Demonstration

Show that the module works correctly — the input, the process and the output.

- 1 On the catalogue, press "Gift Cakes": two items, and the line chips "Diaper Cake" and "Beer-in-Can Cake" appear.
- 2 Press "Flower Bouquets": four items — every leaf under that branch.
- 3 Press "Rose": one item. Press "All" to clear.



Branch and line chips after choosing Flower Bouquets

Trace — building and searching the tree (computed by our code)

The tree as built by `treeFromOutline()`: one array of nodes

Index	Label	Slug	Parent	Children	Depth
0	All gifts	root	NIL	[1, 6, 11]	0
1	Flower Bouquets	flower-bouquets	0	[2, 3, 4, 5]	1
2	Rose	rose	1	[]	2
3	Plumeria	plumeria	1	[]	2
4	Dahlia	dahlia	1	[]	2
5	Sunflower	sunflower	1	[]	2
6	Other Bouquets	other-bouquets	0	[7, 8, 9, 10]	1
7	Paper Bills	paper-bills	6	[]	2
8	Makeup	makeup	6	[]	2
9	Pictures	pictures	6	[]	2
10	Sweets / Snacks	sweets	6	[]	2
11	Gift Cakes	gift-cakes	0	[12, 13]	1
12	Diaper Cake	diaper-cake	11	[]	2
13	Beer-in-Can Cake	beer-cake	11	[]	2

`treeFind(tree, "dahlia")` — the nodes popped off the stack, in order

All gifts → Flower Bouquets → Rose → Plumeria → Dahlia ⇒ index 4

treeLeaves(tree, "Flower Bouquets")

Leaves: rose, plumeria, dahlia, sunflower. The catalogue then shows the 4 products whose line is one of these.

Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

js/structures.js · lines 389–401 · `treeAdd()` — Adds one node under a parent.

```
389  /* Attach a node under a parent.           Time O(1) · Space O(1) */
390  function treeAdd(tree, label, slug, parentIndex) {
391  ❶    var index = tree.nodes.length;
392      tree.nodes[index] = {
393          label: label,
394          slug: slug,
395          parent: parentIndex,
396          children: [],
397  ❷    depth: parentIndex === NIL ? 0 : tree.nodes[parentIndex].depth + 1
398      };
399  ❸    if (parentIndex !== NIL) listAdd(tree.nodes[parentIndex].children, index);
400      return index;
401  }
```

- ❶ line 391 The new node's index is the next free slot.
- ❷ line 397 Depth is the parent's depth + 1 (the root is 0).
- ❸ line 399 Link it: the parent remembers its new child's index.

Time O(1) · Space O(1)

js/structures.js · lines 456–466 · `treeFromOutline()` — Builds the whole tree from the order sheet outline.

```
456  /* Build from [{ label, slug, children: [{ label, slug }] }].
457                        Time O(n) · Space O(n) */
458  function treeFromOutline(rootLabel, branches) {
459  ❶    var tree = treeCreate(rootLabel);
460      for (var i = 0; i < branches.length; i++) {
461  ❷    var parent = treeAdd(tree, branches[i].label, branches[i].slug, tree.root);
462          var kids = branches[i].children || [];
463  ❸    for (var j = 0; j < kids.length; j++) treeAdd(tree, kids[j].label, kids[j].slug, parent);
464      }
465      return tree;
466  }
```

- ❶ line 459 Start with a root.
- ❷ line 461 Add each branch under the root...
- ❸ line 463 ...and each line under its branch.

Time O(n) · Space O(n)

js/structures.js · lines 403–414 · `treeFind()` — Finds a node by slug with our stack (depth-first).

```
403  /* Depth-first search for a slug, with an explicit stack.
404                        Time O(n) · Space O(n) */
405  function treeFind(tree, slug) {
406      var stack = stackCreate();
407  ❶    stackPush(stack, tree.root);
408      while (stack.top !== NIL) {
```

```

409 ②    var at = stackPop(stack), node = tree.nodes[at];
410 ③    if (node.slug === slug) return at;
411 ④    for (var i = node.children.length - 1; i >= 0; i--) stackPush(stack, node.children[i]);
412    }
413    return NIL;
414    }

```

- ① Line 407 Start at the root.
- ② Line 409 Take the most recently added node (last in, first out).
- ③ Line 410 Found it — return its index.
- ④ Line 411 Otherwise push its children, last first, so the first child is visited next.

Time $O(n)$ · Space $O(n)$

js/structures.js · lines 431–444 · treeLeaves() — Collects every leaf under a node.

```

431  /* Every leaf slug at or below a node - what the catalogue filters on.
432                                     Time  $O(n)$  · Space  $O(n)$  */
433  function treeLeaves(tree, index) {
434      var out = [];
435      if (index === NIL) return out;
436      var stack = stackCreate();
437      stackPush(stack, index);
438      while (stack.top !== NIL) {
439          var at = stackPop(stack), node = tree.nodes[at];
440  ①    if (node.children.length === 0) listAdd(out, node.slug);
441  ②    else for (var i = node.children.length - 1; i >= 0; i--) stackPush(stack, node.children[i]);
442      }
443      return out;
444  }

```

- ① Line 440 A node with no children is a leaf: keep its slug.
- ② Line 441 Otherwise descend into its children.

Time $O(n)$ · Space $O(n)$

js/shop-catalog.js · lines 130–144 · catalogRows() — Applies the tree filter (then search and sort — module 3).

```

130  /* The rows to show, in order.           Time  $O(n^2)$  · Space  $O(n)$  */
131  function catalogRows() {
132      var slug = catalogState.line || catalogState.branch;
133  ①    var leaves = slug ? treeLeaves(categoryTree, treeFind(categoryTree, slug)) : null;
134  ②    var inBranch = keepWhere(activeProducts(), function (p) { return leaves === null || isIn(leaves,
135      p.line); });
136
137      var found = linearSearch(inBranch, catalogState.query, searchText);
138
139      var tally = salesTally(), rows = [];
140      for (var i = 0; i < found.length; i++) {
141          var t = firstWhere(tally, function (r) { return r.product.id === found[i].id; });
142          listAdd(rows, { product: found[i], orders: t ? t.orders : 0, units: t ? t.units : 0 });
143      }
144      var sort = firstWhere(SORTS, function (s) { return s.id === catalogState.sort; }) || SORTS[0];
145      return insertionSort(rows, sort.compare);
146  }

```

- ① Line 133 The chosen chip's leaves, or no filter.
- ② Line 134 Keep active products whose line is one of those leaves.

Time $O(n^2)$ · Space $O(n)$

Search & Sort

Screens	Catalogue — the search box and the sort menu (Best selling, Name A–Z, Starting price, Newest)
Files	js/core.js (normaliseKey, textHas) · js/algorithms.js (linearSearch, insertionSort) · js/shop-catalog.js (catalogRows, SORTS)
Data	products — 10 records; SORTS — the four comparisons; each product's searchable text
DSA	Linear search · Insertion sort

1 Business process

What task or problem from the client are you solving?

A customer who knows what they want types it: "matcha", "money", "diaper". The search has to find it in names, materials, branch names and even colour names, however it is typed (" MATCHA ", "Matcha rose").

They also want to order the results: best selling, by name, cheapest first, or newest — and the order must be steady, not shuffling equal items around.

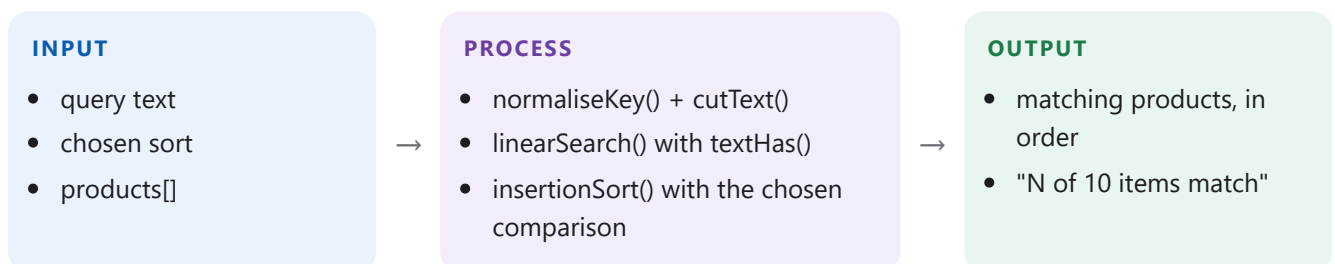
2 Input

What data is needed by the module?

Input	What it is
Customer types	Search words in the search box.
Customer picks	A sort: Best selling, Name A–Z, Starting price, Newest.
Stored: products	Name, line, branch, short description, materials, colours, starting price, date added.

3 Process

What data structure and/or algorithm did you use? How does it work step by step?



Step by step (Linear search · Insertion sort):

- 1 Normalise the query: lower-case, trim, collapse spaces, then cut it into words.
- 2 Linear search: for each product, build its searchable text and keep it only if every word appears in it — naive string matching, trying the word at every position of the text.
- 3 Insertion sort the matches with the chosen comparison: take each product in turn and slide the products before it that sort later one place right, until its place is found. Equal products never pass each other, so the order is stable.

4 Output

What result does the module produce?

- The catalogue grid, filtered and ordered.
- The result count, and a "Clear filters" button when filtered.

5 Complexity analysis

What is the time complexity? What is the space complexity?

Operation	Time	Space	Note
<code>normaliseKey()</code>	$O(n)$	$O(n)$	Puts text into one comparable form.
<code>textHas()</code>	$O(n^2)$ – every start position × the needle	$O(1)$	Does the text contain the word? Naive string matching.
<code>linearSearch()</code>	$O(n^2)$ – naive string matching on every record	$O(n)$	Keeps every record whose text has every word.
<code>insertionSort()</code>	$O(n^2)$ worst, $O(n)$ best	$O(n)$ copy	The sort used everywhere in the system.
<code>catalogRows()</code>	$O(n^2)$	$O(n)$	Search, then sort.

What n means here: n = number of products searched or sorted. Inside `textHas`, n = the length of the text being scanned.

Each Time and Space is read from the comment above the function in the source, where every function in `js/` states its own cost.

6 Justification

Why did you choose that data structure or algorithm? Why is it appropriate?

Linear search is the right search here: the match is "contains these words anywhere in several fields". Binary search can only find an exact key in a sorted array, so it cannot answer a "contains" question; at n = 10 a full scan is instant.

Naive string matching tries the word at every starting position and compares letter by letter — $O(n^2)$, the cost the course gives for searching text across a table. It needs no extra index and handles any word.

Insertion sort is stable and simple: $O(n)$ when the list is already in order, $O(n^2)$ at worst — 45 comparisons for 10 products. It is one of the sorts from class; merge sort would be $O(n \log n)$, a notation the course does not use, and quick sort is not stable.

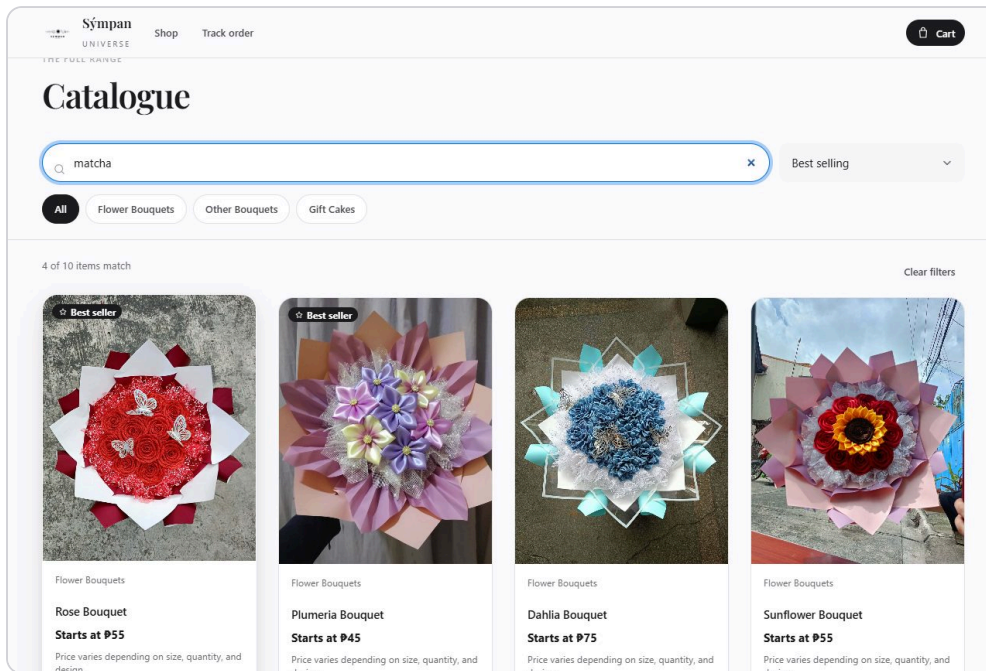
The same `insertionSort` serves every list in the system that needs ordering: completed orders, voids, the outbox, past quotations.

7 Demonstration

Show that the module works correctly — the input, the process and the output.

- 1 Type "matcha" in the search box: the four flower bouquets appear, because Matcha is one of their ribbon colours.
- 2 Type "money": only the Money Bouquet.

3 Change the sort to "Name, A-Z": Beer-in-Can Cake comes first.



Searching "matcha" finds the flowers by colour name

Trace — the search and the insertion sort (computed by our code)

linearSearch(products, " Matcha ROSE ") — normalised to the words "matcha", "rose"

14 textHas() checks were made across 10 products (a product stops at its first missing word).

Product text begins	Word	Found?
rose bouquet rose flow...	matcha	yes
rose bouquet rose flow...	rose	yes
plumeria bouquet plume...	matcha	yes
plumeria bouquet plume...	rose	no
dahlia bouquet dahlia ...	matcha	yes
dahlia bouquet dahlia ...	rose	no
sunflower bouquet sunf...	matcha	yes
sunflower bouquet sunf...	rose	yes
money bouquet paper bi...	matcha	no
makeup bouquet makeup ...	matcha	no
picture bouquet pictur...	matcha	no
sweets & snacks bouque...	matcha	no
diaper cake diaper cak...	matcha	no
beer-in-can cake beer-...	matcha	no

Result: Rose Bouquet, Sunflower Bouquet

insertionSort(10 products, by name) — each pass takes the next product and slides it left past the larger names

Pass	Product taken	Comparisons	Result
1	Plumeria Bouquet	1	slides 1 left
2	Dahlia Bouquet	2	slides 2 left
3	Sunflower Bouquet	1	stays
4	Money Bouquet	4	slides 3 left
5	Makeup Bouquet	5	slides 4 left
6	Picture Bouquet	4	slides 3 left
7	Sweets & Snacks Bouquet	1	stays
8	Diaper Cake	8	slides 7 left
9	Beer-in-Can Cake	9	slides 9 left

35 comparisons in all. Worst case for 10 items is $n(n-1)/2 = 45$ (a list in reverse order); best case is $n - 1 = 9$ (already in order).

Result: Beer-in-Can Cake, Dahlia Bouquet, Diaper Cake, Makeup Bouquet, Money Bouquet, Picture Bouquet, Plumeria Bouquet, Rose Bouquet, Sunflower Bouquet, Sweets & Snacks Bouquet

Best and worst case, counted on the real function

Input (10 numbers)	Comparisons	Case
1, 2, 3, ... 10 (already sorted)	9	best — $O(n)$
10, 9, 8, ... 1 (reversed)	45	worst — $O(n^2)$

Orders, emails and receipts are appended in time order, so the "newest first" and "oldest first" lists the shop sorts are close to one of these two cases.

Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

js/core.js · lines 281–296 · normaliseKey() — Puts text into one comparable form.

```
281  /* Lower-case, trimmed, inner runs of space collapsed - the form keys are
282     compared in, so " Quezon  City " finds "quezon city". Time O(n) · Space O(n) */
283  function normaliseKey(text) {
284  ❶    var s = strip(toLower(text)), out = '', lastSpace = false;
285      for (var i = 0; i < s.length; i++) {
286          var c = s.charAt(i);
287  ❷    if (isSpace(c) || c === '.' || c === ',') {
288  ❸        if (!lastSpace) out += ' ';
289          lastSpace = true;
290      } else {
291          out += c;
292          lastSpace = false;
293      }
294  }
295  return strip(out);
296  }
```

- ❶ line 284 Lower-case and trim.
- ❷ line 287 Spaces, dots and commas all become one space...
- ❸ line 288 ...and runs of them collapse to a single space.

Time $O(n)$ · Space $O(n)$

js/core.js · lines 210–220 · `textHas()` — Does the text contain the word? Naive string matching.

```

210  /* Does the text contain the needle? Naive scan. Time  $O(n^2)$  - every start position × the needle · Space
    0(1) */
211  function textHas(haystack, needle) {
212      var text = String(haystack), find = String(needle);
213      if (find.length === 0) return true;
214  ❶   for (var start = 0; start + find.length <= text.length; start++) {
215          var k = 0;
216  ❷   while (k < find.length && text.charAt(start + k) === find.charAt(k)) k++;
217  ❸   if (k === find.length) return true;
218      }
219      return false;
220  }
```

- ❶ line 214 Try every starting position...
- ❷ line 216 ...and match the characters one by one (a loop inside a loop).
- ❸ line 217 All matched: found.

Time $O(n^2)$ - every start position × the needle · Space $O(1)$

js/algorithms.js · lines 100–118 · `linearSearch()` — Keeps every record whose text has every word.

```

100  /* Linear search: every record whose text contains every word typed.
101     textOf builds the searchable text for one record.
102                                     Time  $O(n^2)$  - naive string matching on every record · Space  $O(n)$ 
    */
103  function linearSearch(records, query, textOf) {
104  ❶   var words = cutText(normaliseKey(query), ' ');
105      var wanted = keepWhere(words, function (w) { return w.length > 0; });
106  ❷   if (wanted.length === 0) return copyArray(records);
107
108      var out = [];
109      for (var i = 0; i < records.length; i++) {
110  ❸   var haystack = normaliseKey(textOf(records[i]));
111          var all = true;
112          for (var w = 0; w < wanted.length && all; w++) {
113  ❹   if (!textHas(haystack, wanted[w])) all = false;
114          }
115  ❺   if (all) listAdd(out, records[i]);
116      }
117      return out;
118  }
```

- ❶ line 104 Normalise the query and cut it into words.
- ❷ line 106 No words: everything matches.
- ❸ line 110 Build this record's searchable text.
- ❹ line 113 A missing word rules the record out.
- ❺ line 115 Keep it.

Time $O(n^2)$ - naive string matching on every record · Space $O(n)$

js/algorithms.js · lines 36–52 · insertionSort() — The sort used everywhere in the system.

```
36  /* Returns a new sorted array; the input is left untouched. Each item is
37     taken in turn and the larger items before it slide one slot right until
38     its place is found. "> 0" (not ">= 0") keeps equal items in their
39     original order, so the sort is stable.
40                                     Time O(n²) worst, O(n) best · Space O(n) copy */
41  function insertionSort(list, compare) {
42  ❶    var work = copyArray(list);
43  ❷    for (var i = 1; i < work.length; i++) {
44        var held = work[i], j = i - 1;
45  ❸    while (j >= 0 && compare(work[j], held) > 0) {
46  ❹        work[j + 1] = work[j];
47        j--;
48    }
49  ❺    work[j + 1] = held;
50    }
51    return work;
52  }
```

❶ line 42 Sort a copy; the input is left untouched.

❷ line 43 Take each item in turn, from the second on.

❸ line 45 While the item before it sorts later — "> 0", not ">= 0", so equal items never pass each other (stable)...

❹ line 46 ...slide that item one place right.

❺ line 49 Drop the held item into the gap.

Time O(n²) worst, O(n) best · Space O(n) copy

js/shop-catalog.js · lines 130–144 · catalogRows() — Search, then sort.

```
130  /* The rows to show, in order.          Time O(n²) · Space O(n) */
131  function catalogRows() {
132      var slug = catalogState.line || catalogState.branch;
133      var leaves = slug ? treeLeaves(categoryTree, treeFind(categoryTree, slug)) : null;
134      var inBranch = keepWhere(activeProducts(), function (p) { return leaves === null || isIn(leaves,
135  ❶    p.line); });
136      var found = linearSearch(inBranch, catalogState.query, searchText);
137      var tally = salesTally(), rows = [];
138      for (var i = 0; i < found.length; i++) {
139          var t = firstWhere(tally, function (r) { return r.product.id === found[i].id; });
140          listAdd(rows, { product: found[i], orders: t ? t.orders : 0, units: t ? t.units : 0 });
141      }
142      var sort = firstWhere(SORTS, function (s) { return s.id === catalogState.sort; }) || SORTS[0];
143  ❷    return insertionSort(rows, sort.compare);
144  }
```

❶ line 135 Linear search over the products in the chosen branch.

❷ line 143 Order them with the chosen comparison.

Time O(n²) · Space O(n)

PART 2

Choosing a gift

Module 4 Flower Customiser · Module 5 Gift Customiser · Module 6 Cart

60-second pitch

These modules turn a product into something in the cart. The Flower Customiser finds the right photograph among 160 combinations with a hash table keyed "flower|arrangement|colour", and adds up the add-ons with a recursive sum. The Gift Customiser suggests the fewest bills for a money bouquet with a greedy algorithm. The Cart is a singly linked list: adding is $O(1)$ at the tail, and removing re-points one link.

Structures and algorithms to be able to explain

Where	Structure / algorithm	Time	Space
Find the photograph	Hash table with chaining (FNV-1a)	$O(1)$ average	$O(n)$
Add-on total	Recursion	$O(n)$	$O(n)$ call stack
Fewest bills	Greedy change-making	$O(n)$	$O(n)$
Add to cart	Linked list append (tail pointer)	$O(1)$	$O(1)$
Remove a line	Linked list remove (walk to the node before it)	$O(n)$	$O(1)$
Show the cart	Linked list walk from the head	$O(n)$	$O(n)$

Likely panel questions

What happens when two keys land in the same bucket?

That is a collision. Each bucket holds a chain (a short list); the second key is added to the chain and a look-up compares keys along it. With 160 keys in 199 buckets the chains are about one long.

Why 199 buckets?

A prime a little above the 160 keys: the load factor stays near 0.8 and the remainder spreads the FNV-1a hashes evenly over the buckets.

Is the greedy answer always the fewest bills?

For the peso, yes. Its notes (1000, 500, 200, 100, 50, 20) form a canonical system, where taking the largest note that fits is optimal. For odd currencies greedy can fail and dynamic programming would be needed.

Why a linked list for the cart instead of an array?

The cart adds at the end and removes from anywhere. Removing a node re-points one link; an array would shift every later item. The tail pointer makes adding $O(1)$.

Where is the recursion, and what stops it?

sumRecursive adds the first item to the sum of the rest. The base case is the empty rest, which adds 0. The add-on list is at most four long, so the call stack stays tiny.

Flower Customiser

Screens	The product sheet of a flower bouquet (Rose, Plumeria, Dahlia, Sunflower)
Files	js/core.js (fnv1a) · js/structures.js (hashPut, hashGet) · js/store.js (buildReferenceIndex, makeFlowerItem) · js/algorithms.js (sumRecursive) · js/shop-product.js
Data	referenceIndex — hash table, 199 buckets, 160 keys; COLORS (20), ARRANGEMENTS, REAL_REFERENCES (14); the product's price list
DSA	Hash table · Recursion

1 Business process

What task or problem from the client are you solving?

A flower bouquet comes in two arrangements (Round, Layered), up to nine flower counts and twenty ribbon colours from the shop's colour chart. Customers need to see what they are ordering: the right photo for exactly that arrangement and colour, and the exact price.

There are 160 combinations of flower × arrangement × colour. 14 have real photos of the shop's work; the other 146 have previews made from the shop's own photos, recoloured to the chart colour. Add-ons (fairy lights, topper, card, glitter) add to the price.

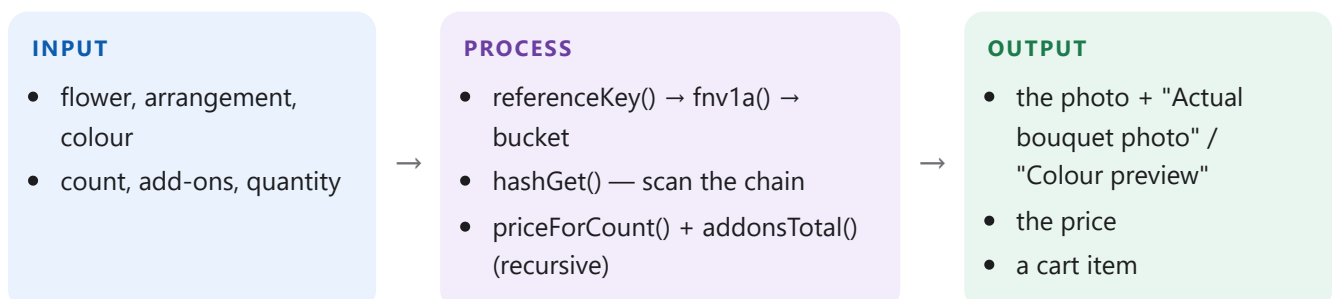
2 Input

What data is needed by the module?

Input	What it is
Customer picks	Arrangement, flower count, one of 20 colours, add-ons, card flower, quantity, notes, an optional reference image.
Stored: REAL_REFERENCES, COLORS, ARRANGEMENTS	Which photos are real; the colour chart; the counts each arrangement offers.
Stored: the product's price list	Price per flower count, set by the owner.

3 Process

What data structure and/or algorithm did you use? How does it work step by step?



Step by step (Hash table · Recursion):

- At start-up, build a hash table of all 160 photos keyed "flower|arrangement|colour": first every generated preview, then the 14 real photos written over their entries.

- 2 When the customer picks, build the key, hash it with FNV-1a, go to that bucket and scan its short chain for the key: the photo, and whether it is real.
- 3 Price = the price-list price for that flower count + the add-ons, added up recursively (each call adds one add-on to the sum of the rest).
- 4 Check the choice and build the cart item.

4 Output

What result does the module produce?

- The large photo and its caption on the product sheet.
- The price in the footer ("P555.00").
- A checked cart item.

5 Complexity analysis

What is the time complexity? What is the space complexity?

Operation	Time	Space	Note
<code>fnv1a()</code>	$O(n)$	$O(1)$	Turns text into a 32-bit number (the hash).
<code>hashPut()</code>	$O(1)$ average, amortised	$O(1)$ amortised	Inserts or overwrites a key.
<code>hashGet()</code>	$O(1)$ average	$O(1)$	Looks a key up.
<code>buildReferenceIndex()</code>	$O(n)$ – one <code>hashPut</code> per flower × arrangement × colour ($n = 160$) and per photo	$O(n)$	Fills the table with all 160 photos.
<code>sumRecursive()</code>	$O(n)$	$O(n)$ call stack	Adds up a list recursively.
<code>makeFlowerItem()</code>	$O(n^2)$	$O(n)$	Checks the choice and prices it.

What n means here: For the table's space, n = keys stored (160). Hashing a key is $O(n)$ in its length. For the recursive sum, n = the add-ons chosen (at most four).

Each Time and Space is read from the comment above the function in the source, where every function in `js/` states its own cost.

6 Justification

Why did you choose that data structure or algorithm? Why is it appropriate?

Every click must find one photo among 160. A hash table answers "what is stored under this key?" in $O(1)$ on average, and with 199 buckets for 160 keys the chains stay about one long. A linear scan of 160 records would grow with every colour or flower added; binary search would need the keys kept sorted.

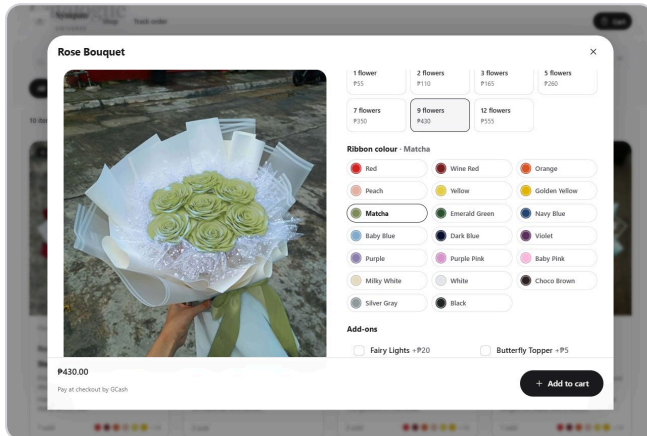
Chaining handles collisions simply: two keys landing in the same bucket just sit in the same short list.

Recursion fits a sum naturally: the total of a list is its first item plus the total of the rest, ending at the empty list. The add-on list is at most four long, so the call stack is tiny.

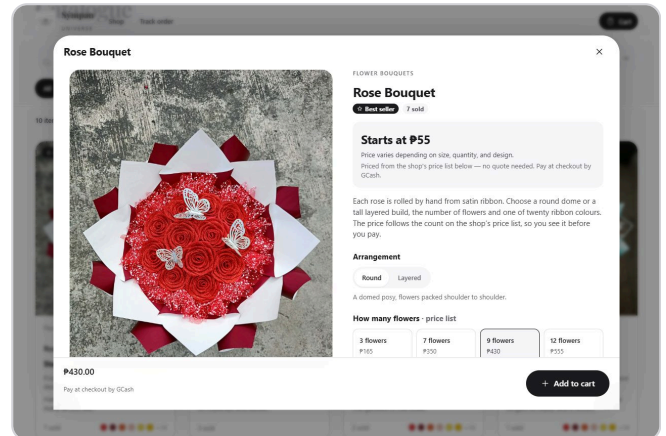
7 Demonstration

Show that the module works correctly — the input, the process and the output.

- 1 Open the Rose Bouquet. It starts Round, Red — caption "Actual bouquet photo".
- 2 Pick Emerald Green: the photo changes and the caption says "Colour preview".
- 3 Switch to Layered: a different photo — round and layered never share one.
- 4 Tick Fairy Lights and the card: the price in the footer goes up by ₱40.



Rose, Layered, Matcha — a colour preview



Rose, Round, Red — a real photograph

Trace — the photo look-up and the add-on sum (computed by our code)

`referenceImage("rose", "layered", "matcha")`

Step	Value
1. <code>referenceKey()</code>	<code>rose layered matcha</code>
2. <code>fnv1a(key)</code>	281994334
3. <code>hashIndex = fnv1a mod 199</code>	190
4. chain length in bucket 190	2
5. compared keys until found	1
Result	<code>assets/colors/rose-layered-matcha.jpg</code> (colour preview)

`[190]` `rose|layered|matcha` → `plumeria|round|emerald-green`

The whole table after `buildReferenceIndex()`

160 keys in 199 buckets: load factor 0.8, longest chain 5. Real photographs overwrote 14 previews (e.g. `rose|round|red` → `assets/products/roses-round-1.jpg`).

Chain length	0	1	2	3	4+
Buckets	93	67	30	4	5

addonsTotal(["lights", "card", "glitter"], 12 flowers) — the recursive calls, innermost first

Call	This item	Returns
sumRecursive(..., 3)	(empty — base case)	₱0
sumRecursive(..., 2)	glitter = ₱15	₱15
sumRecursive(..., 1)	card = ₱20	₱35
sumRecursive(..., 0)	lights = ₱20	₱55

Total add-ons: ₱55 (glitter for 12 flowers is ₱15).

Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

js/core.js · lines 536–546 · fnv1a() — Turns text into a 32-bit number (the hash).

```
536  /* 32-bit FNV-1a. Used for the hash table's bucket index and to compare the
537      order-desk passcode without keeping it in the page as plain text.
538      Time O(n) · Space O(1) */
539  function fnv1a(text) {
540  ❶    var s = String(text), hash = 2166136261;
541      for (var i = 0; i < s.length; i++) {
542  ❷        hash = hash ^ s.charCodeAt(i);
543  ❸        hash = Math.imul(hash, 16777619) >>> 0;
544      }
545      return hash >>> 0;
546  }
```

- ❶ Line 540 Start from the FNV offset basis.
- ❷ Line 542 Mix in each character with XOR...
- ❸ Line 543 ...then multiply by the FNV prime, kept to 32 bits.

Time O(n) · Space O(1)

js/structures.js · lines 493–502 · hashPut() — Inserts or overwrites a key.

```
493  /* Insert or overwrite. Time O(1) average, amortised · Space O(1) amortised */
494  function hashPut(table, key, value) {
495  ❶    var chain = table.buckets[hashIndex(table, key)];
496      for (var i = 0; i < chain.length; i++) {
497  ❷        if (chain[i].key === key) { chain[i].value = value; return; }
498      }
499  ❸    chain[chain.length] = { key: key, value: value };
500      table.size++;
501      if (table.size > 2 * table.bucketCount) hashGrow(table);
502  }
```

- ❶ Line 495 Hash the key to its bucket.
- ❷ Line 497 Key already there: overwrite it (how real photos replace previews).
- ❸ Line 499 Otherwise add it to the end of the chain.

Time O(1) average, amortised · Space O(1) amortised

js/structures.js · lines 520–527 · hashGet() — Looks a key up.

```
520  /* The value for a key, or null. Time O(1) average · Space O(1) */
```

```

521 function hashGet(table, key) {
522 ❶ var chain = table.buckets[hashIndex(table, key)];
523   for (var i = 0; i < chain.length; i++) {
524 ❷   if (chain[i].key === key) return chain[i].value;
525   }
526   return null;
527 }

```

❶ Line 522 Go straight to the bucket.

❷ Line 524 Scan the short chain for the exact key.

Time $O(1)$ average · Space $O(1)$

js/store.js · lines 220–243 · buildReferenceIndex() — Fills the table with all 160 photos.

```

220 /* Index every combination: generated previews first, then the real
221    photographs written over the entries they exist for.
222                                     Time  $O(n)$  – one hashPut per flower × arrangement × colour (n =
160) and per photo · Space  $O(n)$  */
223 function buildReferenceIndex() {
224   // 160 keys over 199 buckets: a load factor near 0.8 keeps chains short.
225 ❶ var table = hashCreate(199);
226   for (var f = 0; f < FLOWERS.length; f++) {
227     for (var a = 0; a < ARRANGEMENTS.length; a++) {
228       for (var c = 0; c < COLORS.length; c++) {
229         var flower = FLOWERS[f], arrangement = ARRANGEMENTS[a].id, color = COLORS[c].id;
230         hashPut(table, referenceKey(flower, arrangement, color), {
231 ❷   src: IMG_COLORS + flower + '-' + arrangement + '-' + color + '.jpg',
232     real: false
233       });
234     }
235   }
236 }
237 for (var r = 0; r < REAL_REFERENCES.length; r++) {
238   var ref = REAL_REFERENCES[r];
239   hashPut(table, referenceKey(ref.flower, ref.arrangement, ref.color),
240 ❸   { src: IMG_PRODUCTS + ref.file, real: true });
241 }
242 return table;
243 }

```

❶ Line 225 199 buckets: a prime a little above 160 keeps chains short.

❷ Line 231 Every combination first gets its generated preview...

❸ Line 240 ...then the 14 real photos overwrite their entries.

Time $O(n)$ – one hashPut per flower × arrangement × colour (n = 160) and per photo · Space $O(n)$

js/algorithms.js · lines 124–131 · sumRecursive() — Adds up a list recursively.

```

124 /* Sum a list by peeling off one element at a time. The base case is the
125    empty tail. Used for the add-on total and the quote's line totals.
126                                     Time  $O(n)$  · Space  $O(n)$  call stack */
127 function sumRecursive(items, valueOf, index) {
128   var at = index === undefined ? 0 : index;
129 ❶ if (at >= items.length) return 0;
130 ❷ return (Number(valueOf(items[at])) || 0) + sumRecursive(items, valueOf, at + 1);
131 }

```

❶ Line 129 Base case: the empty rest adds nothing.

❷ Line 130 This item plus the sum of the rest.

Time $O(n)$ · Space $O(n)$ call stack

```

285  /* Validate a flower choice and build its item.  Time O(n²) · Space O(n)
286     choice: { arrangement, count, color, quantity, addons, cardFlower, notes, reference } */
287  function makeFlowerItem(product, choice) {
288      if (!product || !product.active || product.kind !== 'flower') return { ok: false, error: 'This bouquet
is not available.' };
289      if (!isIn(product.arrangements, choice.arrangement)) return { ok: false, error: 'Choose an
arrangement.' };
290  ❶ if (!isIn(countsFor(product, choice.arrangement), choice.count)) return { ok: false, error: 'Choose how
many flowers.' };
291      if (!isIn(product.colors, choice.color)) return { ok: false, error: 'Choose a ribbon colour.' };
292      if (!isWholeIn(choice.quantity, 1, MAX_QUANTITY)) return { ok: false, error: 'Quantity must be 1 to ' +
MAX_QUANTITY + '.' };
293      if (String(choice.notes || '').length > MAX_NOTE) return { ok: false, error: 'Notes are limited to ' +
MAX_NOTE + ' characters.' };
294
295      var addons = keepWhere(choice.addons || [], function (id) { return addonById(id) !== null; });
296  ❷ var unit = priceForCount(product, choice.count) + addonsTotal(addons, choice.count);
297      return { ok: true, item: {
298          productId: product.id, productName: product.name, kind: 'flower', quoteType: 'flower',
299          arrangement: choice.arrangement, count: choice.count, color: choice.color,
300          size: '', detail: '', provided: false,
301          quantity: choice.quantity, addons: addons,
302          cardFlower: isIn(addons, 'card') ? (choice.cardFlower || CARD_FLOWERS[0]) : '',
303          notes: strip(choice.notes), reference: choice.reference || null,
304  ❸ unitEstimate: unit, estimate: unit * choice.quantity,
305          materials: 0, labor: 0, itemExpense: 0
306      } };
307  }

```

- ❶ Line 290 The count must be one this arrangement offers.
- ❷ Line 296 Price list + add-ons for one bouquet.
- ❸ Line 304 Times the quantity.

Time $O(n^2)$ · Space $O(n)$

Gift Customiser

Screens	The product sheet of the five quote gifts (money, makeup, sweets & snacks, diaper cake, beer-in-can cake) and the picture bouquet
Files	js/algorithms.js (breakIntoBills, describeBills) · js/store.js (makeQuoteItem) · js/shop-product.js (billHint)
Data	DENOMINATIONS — 1000, 500, 200, 100, 50, 20; QUOTE_TYPES; the product's sizes
DSA	Greedy algorithm

1 Business process

What task or problem from the client are you solving?

Five gifts have no price list and are quoted by the owner: the money bouquet, makeup bouquet, sweets & snacks bouquet, diaper cake and beer-in-can cake. The customer describes what they want — size, how many bills or cans, whether they will bring the items themselves — and the owner prices it. The picture bouquet has one set price (₱650).

For a money bouquet the customer usually knows the total they want to give (say ₱5,870) but not how many bills that is. The number of bills decides the size of the bouquet and the labour, so the site suggests the fewest bills for that amount.

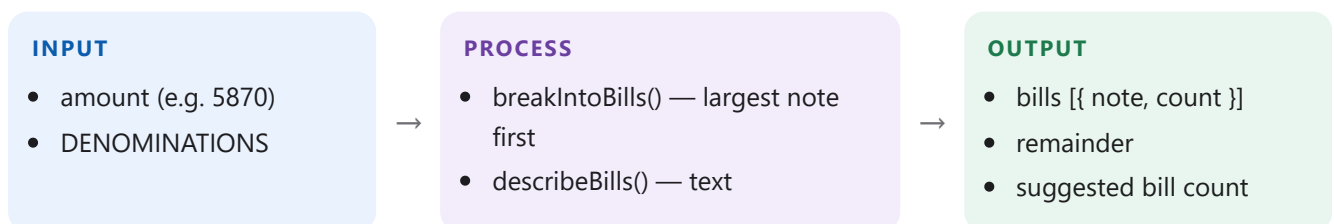
2 Input

What data is needed by the module?

Input	What it is
Customer types / picks	Size, number of bills / items / cans, whether they provide the items, brand or details, add-ons, quantity, notes, reference image.
Customer types (money)	The total amount to put inside, e.g. 5870.
Stored: DENOMINATIONS	Philippine notes, largest first: 1000, 500, 200, 100, 50, 20.

3 Process

What data structure and/or algorithm did you use? How does it work step by step?



Step by step (Greedy algorithm):

- Greedy change-making: start with the amount; for each note from the largest down, take as many as fit (amount ÷ note, rounded down) and subtract them.
- Whatever is left at the end (under ₱20) cannot be made in bills and is reported.

- 3 Show "Fewest bills: ₱1,000 × 5 · ₱500 × 1 · ... = 10 bills" with a button to use that count.
- 4 Check the gift and add it to the cart as a quote item (priced later by the owner).

4 Output

What result does the module produce?

- The bill suggestion under the amount box, and the "Use N bills" button.
- A quote item in the cart, labelled "Quote".
- For the sweets bouquet and the beer-in-can cake, which the shop had not photographed, openly licensed sample photos from the web — labelled "Sample photo" and credited (author, licence, source).

5 Complexity analysis

What is the time complexity? What is the space complexity?

Operation	Time	Space	Note
<code>breakIntoBills()</code>	$O(n)$	$O(n)$	Greedy change-making.
<code>billHint()</code>	$O(n)$	$O(n)$	Turns the result into the hint text.
<code>makeQuoteItem()</code>	$O(n)$	$O(n)$	Checks a gift and builds its cart item.

What n means here: n = number of note denominations (6).

Each Time and Space is read from the comment above the function in the source, where every function in js/ states its own cost.

6 Justification

Why did you choose that data structure or algorithm? Why is it appropriate?

For a canonical currency like the peso, greedy is provably optimal: taking the largest note that fits always gives the fewest notes. It runs in $O(n)$ — six steps — with no search.

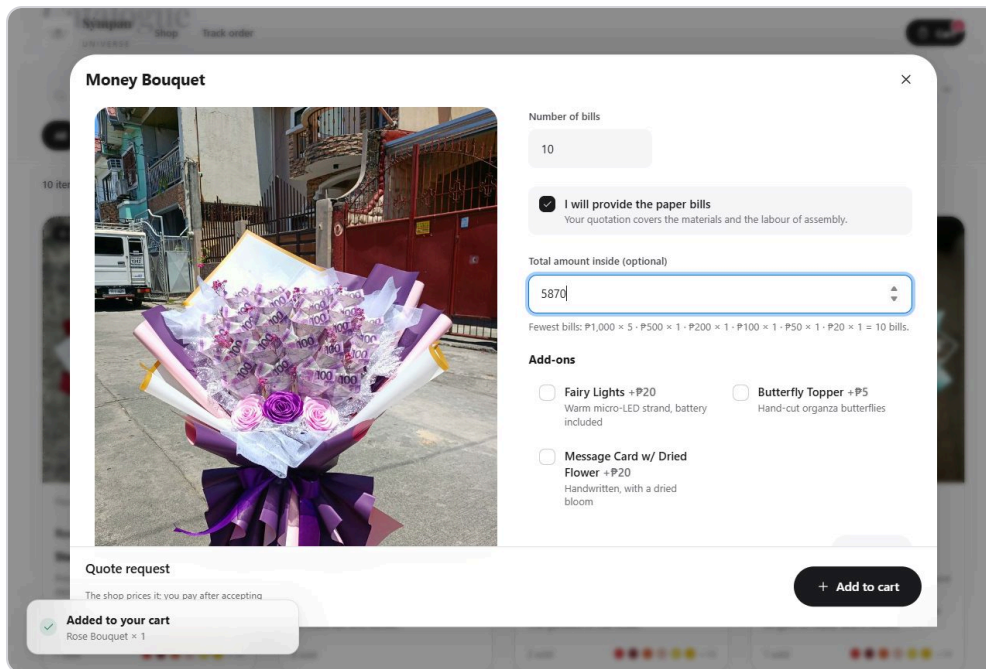
The alternative, dynamic programming over every amount up to ₱5,870, is needed only for odd currencies where greedy can fail; here it would be thousands of steps to reach the same answer.

Fewer bills means a smaller, cheaper bouquet — which is what a customer asking "how many bills?" usually wants.

7 Demonstration

Show that the module works correctly — the input, the process and the output.

- 1 Open the Money Bouquet. Type 5870 in "Total amount inside".
- 2 The hint shows ₱1,000 × 5 · ₱500 × 1 · ₱200 × 1 · ₱100 × 1 · ₱50 × 1 · ₱20 × 1 = 10 bills.
- 3 Press "Use 10 bills": the bill count updates.



The money bouquet with the greedy bill suggestion

Trace — the fewest bills (computed by our code)

breakIntoBills(5870, [1000, 500, 200, 100, 50, 20])

Note	Remaining before	Taken	Remaining after
₱1000	₱5870	5	₱870
₱500	₱870	1	₱370
₱200	₱370	1	₱170
₱100	₱170	1	₱70
₱50	₱70	1	₱20
₱20	₱20	1	₱0

Result: $\text{₱1,000} \times 5 + \text{₱500} \times 1 + \text{₱200} \times 1 + \text{₱100} \times 1 + \text{₱50} \times 1 + \text{₱20} \times 1 = 10$ bills.

breakIntoBills(3460, [1000, 500, 200, 100, 50, 20])

Note	Remaining before	Taken	Remaining after
₱1000	₱3460	3	₱460
₱500	₱460	0	₱460
₱200	₱460	2	₱60
₱100	₱60	0	₱60
₱50	₱60	1	₱10
₱20	₱10	0	₱10

Result: $\text{₱1,000} \times 3 + \text{₱200} \times 2 + \text{₱50} \times 1 = 6$ bills; ₱10 cannot be made in bills.

breakIntoBills(15, [1000, 500, 200, 100, 50, 20])

Note	Remaining before	Taken	Remaining after
₱1000	₱15	0	₱15
₱500	₱15	0	₱15
₱200	₱15	0	₱15
₱100	₱15	0	₱15
₱50	₱15	0	₱15
₱20	₱15	0	₱15

Result: no bills = 0 bills; ₱15 cannot be made in bills.

Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

js/algorithms.js · lines 141–154 · breakIntoBills() — Greedy change-making.

```
141 function breakIntoBills(amount, denominations) {
142 ① var remaining = Math.floor(Number(amount) || 0);
143   if (remaining < 0) remaining = 0;
144   var bills = [];
145 ② for (var i = 0; i < denominations.length; i++) {
146     var note = denominations[i];
147 ③ var count = Math.floor(remaining / note);
148     if (count > 0) {
149         listAdd(bills, { note: note, count: count });
150 ④ remaining -= note * count;
151     }
152 }
153 ⑤ return { bills: bills, remainder: remaining, total: sumRecursive(bills, function (b) { return b.count;
    }) };
154 }
```

- ① line 142 Start with the whole amount.
- ② line 145 Go through the notes, largest first.
- ③ line 147 Take as many of this note as fit.
- ④ line 150 Subtract them; move on to the next smaller note.
- ⑤ line 153 Return the bills, what could not be made, and the total count.

Time $O(n)$ · Space $O(n)$

js/shop-product.js · lines 179–188 · billHint() — Turns the result into the hint text.

```
179 /* The fewest bills for the amount typed – greedy change-making.
180      Time  $O(n)$  · Space  $O(n)$  */
181 function billHint() {
182     var amount = toNumber(productState.detail);
183     if (!(amount > 0)) return 'Optional. Tell us the total and we will suggest the fewest bills.';
184 ① var b = breakIntoBills(amount, DENOMINATIONS);
185     if (b.total === 0) return 'Bills start at ₱20.';
186 ② return 'Fewest bills: ' + describeBills(b) + ' = ' + b.total + ' ' + plural(b.total, 'bill') +
187         (b.remainder > 0 ? ' (₱' + b.remainder + ' cannot be made in bills)' : '') + '.';
188 }
```

- 1 line 184 Run the greedy break-down.
- 2 line 186 Show the notes and the total.

Time $O(n)$ · Space $O(n)$

js/store.js · lines 309–341 · makeQuoteItem() — Checks a gift and builds its cart item.

```
309  /* Validate a gift choice and build its item. Quote products (money,
310     makeup, sweets, diaper, beer) are priced later by the owner; a
311     fixed-price product (the picture bouquet) is priced here.
312                                     Time  $O(n)$  · Space  $O(n)$ 
313     choice: { size, count, detail, provided, quantity, addons, cardFlower, notes, reference } */
314  function makeQuoteItem(product, choice) {
315      if (!product || !product.active || (product.kind !== 'quote' && product.kind !== 'fixed')) {
316          return { ok: false, error: 'This item is not available.' };
317      }
318      var type = quoteTypeById(product.quoteType);
319      1 if (!isIn(product.sizes, choice.size)) return { ok: false, error: 'Choose a size.' };
320      2 if (!isWholeIn(choice.count, 1, 500)) return { ok: false, error: type.countLabel + ' must be a whole
    number from 1 to 500.' };
321      if (!isWholeIn(choice.quantity, 1, MAX_QUANTITY)) return { ok: false, error: 'Quantity must be 1 to ' +
    MAX_QUANTITY + '.' };
322      if (String(choice.detail || '').length > 160) return { ok: false, error: 'Keep the details under 160
    characters.' };
323      if (String(choice.notes || '').length > MAX_NOTE) return { ok: false, error: 'Notes are limited to ' +
    MAX_NOTE + ' characters.' };
324
325      var addons = keepWhere(choice.addons || [], function (id) {
326          var addon = addonById(id);
327          return addon !== null && !addon.flowersOnly;
328      });
329      3 var unit = (product.kind === 'fixed' ? product.price : 0) + addonsTotal(addons, 0);
330      return { ok: true, item: {
331          productId: product.id, productName: product.name, kind: product.kind, quoteType: product.quoteType,
332          arrangement: '', count: choice.count, color: '',
333          size: choice.size, detail: strip(choice.detail),
334          provided: type.provided ? choice.provided === true : false,
335          quantity: choice.quantity, addons: addons,
336          cardFlower: isIn(addons, 'card') ? (choice.cardFlower || CARD_FLOWERS[0]) : '',
337          notes: strip(choice.notes), reference: choice.reference || null,
338          unitEstimate: unit, estimate: unit * choice.quantity,
339          materials: 0, labor: 0, itemExpense: 0
340      } };
341  }
```

- 1 Line 319 The size must be one the product offers.
- 2 Line 320 The count must be a whole number from 1 to 500.
- 3 Line 329 Quote items carry only their add-ons; the picture bouquet carries its set price.

Time $O(n)$ · Space $O(n)$

Cart

Screens	The cart sheet (the bag button) — lines, + / – and Remove, the subtotal
Files	js/structures.js (llAppend, llRemove, llUpdate, llEntries) · js/store.js (basketAdd, basketRemove, basketSetQuantity) · js/shop-basket.js
Data	basket — linked list { nodes, head, tail, size }, each node { value, next, live }
DSA	Singly linked list

1 Business process

What task or problem from the client are you solving?

Customers collect several items before checking out — a rose bouquet, a picture bouquet — change quantities, and remove things they no longer want, in any order.

The cart must keep the items in the order they were added and remove any one of them cleanly.

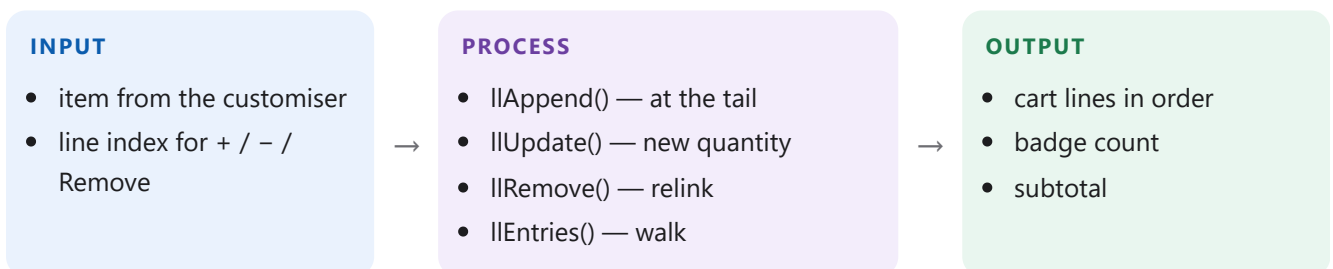
2 Input

What data is needed by the module?

Input	What it is
Customer picks	"Add to cart" on a product; + / – on a line; "Remove".
Built by modules 4 and 5	The checked item: product, choices, quantity, price.

3 Process

What data structure and/or algorithm did you use? How does it work step by step?



Step by step (Singly linked list):

- 1 Append: the new item becomes a node at the tail; the old tail's "next" points to it ($O(1)$, thanks to the tail pointer).
- 2 Change quantity: replace that node's value with a copy carrying the new quantity and price.
- 3 Remove: walk from the head to find the node before it, re-point that node's "next" past it, and mark the removed node dead — nothing else moves.
- 4 Show: walk from the head along the "next" pointers.

4 Output

What result does the module produce?

- The cart sheet with its lines, the Cart badge count, and the subtotal.

5 Complexity analysis

What is the time complexity? What is the space complexity?

Operation	Time	Space	Note
<code>llAppend()</code>	$O(1)$	$O(1)$	Adds a node at the tail.
<code>llRemove()</code>	$O(n)$	$O(1)$	Unlinks one node.
<code>llEntries()</code>	$O(n)$	$O(n)$	Walks the list in order.
<code>basketSetQuantity()</code>	$O(1)$	$O(1)$	Changes a line's quantity.

What n means here: n = number of lines in the cart.

Each Time and Space is read from the comment above the function in the source, where every function in `js/` states its own cost.

6 Justification

Why did you choose that data structure or algorithm? Why is it appropriate?

A linked list is the textbook structure for "add at the end, remove from anywhere, keep the order". Removing a node re-points one link instead of shifting every later item down as an array would.

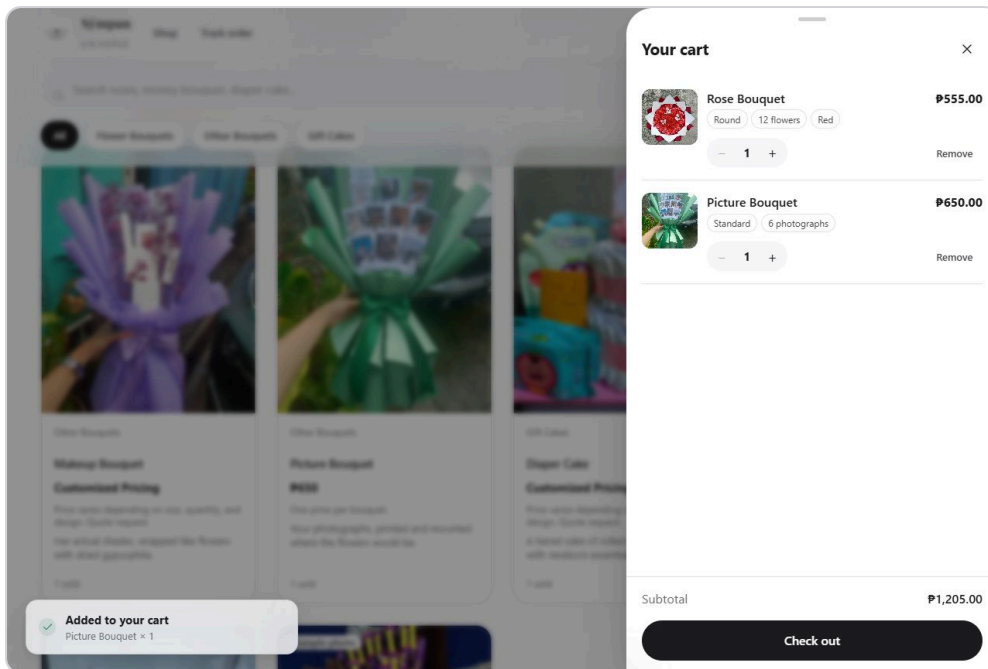
The tail pointer makes adding $O(1)$. Keeping the nodes in one array with whole-number "next" links shows the pointer mechanics plainly while still obeying the array-only rule.

The cart is short, so the $O(n)$ walk to find the node before the one removed costs nothing in practice.

7 Demonstration

Show that the module works correctly — the input, the process and the output.

- 1 Add a Rose Bouquet and a Picture Bouquet; open the cart: two lines, in that order.
- 2 Press + on the first line: its quantity and price update.
- 3 Remove the first line: the second stays exactly as it was.



The cart with two lines

Trace — the cart's nodes and links (computed by our code)

After `basketAdd()` three times



head = 0 · tail = 2 · size = 3

basketRemove(1) — the middle node: node 0 now points past it; nothing moved



head = 0 · tail = 2 · size = 2

basketAdd(Money Bouquet) at the tail, and basketSetQuantity(0, 2)



head = 0 · tail = 3 · size = 3

basketRemove(0) — the head: head moves to node 2



head = 2 · tail = 3 · size = 2

Walking from the head (llEntries): Dahlia Bouquet × 1 → Money Bouquet × 1

Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

js/structures.js · lines 29–39 · `llAppend()` — Adds a node at the tail.

```
29  /* Append at the tail. Keeping a tail pointer is what makes this O(1)
30     rather than a walk to the end.           Time O(1) · Space O(1) */
31  function llAppend(list, value) {
32      var index = list.nodes.length;
33  ❶    list.nodes[index] = { value: value, next: NIL, live: true };
34  ❷    if (list.head === NIL) list.head = index;
35  ❸    else list.nodes[list.tail].next = index;
36  ❹    list.tail = index;
37      list.size++;
38      return index;
39  }
```

❶ line 33 Make a node in the next free slot; it points nowhere yet.

❷ line 34 If the list was empty it becomes the head...

❸ line 35 ...otherwise the old tail points to it.

❹ line 36 It is the new tail.

Time O(1) · Space O(1)

js/structures.js · lines 41–62 · `llRemove()` — Unlinks one node.

```
41  /* Unlink the node at an index. A singly linked list has no back pointer,
42     so the predecessor is found by walking from the head.
43                                     Time O(n) · Space O(1) */
44  function llRemove(list, index) {
45      var node = list.nodes[index];
46      if (!node || !node.live) return false;
47
48  ❶    if (list.head === index) {
49          list.head = node.next;
50          if (list.tail === index) list.tail = NIL;
51      } else {
52          var prev = list.head;
53  ❷    while (prev !== NIL && list.nodes[prev].next !== index) prev = list.nodes[prev].next;
54          if (prev === NIL) return false;
55  ❸    list.nodes[prev].next = node.next;
56          if (list.tail === index) list.tail = prev;
57      }
58  ❹    node.live = false;
59      node.next = NIL;
60      list.size--;
61      return true;
62  }
```

❶ line 48 Removing the head: the head moves to the next node.

❷ line 53 Otherwise walk to the node before it.

❸ line 55 Skip over it.

❹ line 58 Mark it dead; nothing else moves.

Time O(n) · Space O(1)

js/structures.js · lines 78–88 · `llEntries()` — Walks the list in order.

```
78  /* Walk head to tail: [{ index, value }]. Time O(n) · Space O(n) */
79  function llEntries(list) {
80  ❶    var out = [], cursor = list.head, guard = 0;
81      while (cursor !== NIL && guard <= list.nodes.length) {
82          var node = list.nodes[cursor];
83          out[out.length] = { index: cursor, value: node.value };
84  ❷    cursor = node.next;
85          guard++;
86      }
87      return out;
88  }
```

❶ line 80 Start at the head.

❷ line 84 Follow each "next" pointer until NIL.

Time $O(n)$ · Space $O(n)$

js/store.js · lines 396–401 · `basketSetQuantity()` — Changes a line's quantity.

```
396  /* Change one line's quantity; its estimate follows. Time O(1) · Space O(1) */
397  function basketSetQuantity(index, quantity) {
398      var item = llGet(basket, index);
399      if (!item || !isWholeIn(quantity, 1, MAX_QUANTITY)) return false;
400  ❶    return llUpdate(basket, index, copyRecord(item, { quantity: quantity, estimate: item.unitEstimate *
      quantity }));
401  }
```

❶ line 400 Replace the node's value with a copy carrying the new quantity and price.

Time $O(1)$ · Space $O(1)$

PART 3

Checkout and payment

Module 7 Checkout & Delivery · Module 8 Quotation Desk · Module 9 Payment, Receipts & Email

60-second pitch

These modules take the customer from cart to paid order. Checkout & Delivery finds the delivery fee for whatever city is typed with a hash table of 45 names and aliases, and checks every field by scanning its characters — no regular expressions. The Quotation Desk answers quote requests first come, first served with a circular queue, and pre-fills each quotation from the owner's past quotes, put newest first with insertion sort. Payment, Receipts & Email takes GCash payments, snapshots each one into a receipt, and sends every email in order through a second circular queue.

Structures and algorithms to be able to explain

Where	Structure / algorithm	Time	Space
City → delivery area	Hash table (normalised names and aliases)	$O(1)$ average	$O(n)$
Check the checkout form	Character scans; each cart line's product looked up	$O(n^2)$	$O(n)$
Quote requests in order	Circular queue (FIFO)	$O(1)$ enqueue / dequeue	$O(n)$
Pre-fill a quotation	Insertion sort of past quotes + linear search	$O(n^2)$	$O(n)$
Reference used before?	Linear search of the receipts	$O(n)$	$O(1)$
Send emails in order	Circular queue (FIFO)	$O(1)$ per email	$O(n)$

Likely panel questions

Why does a circular queue never shift its items?

The head and the tail move forward and wrap around with "% capacity". Dequeueing only moves the head; its slot is reused later. A plain array that removes from the front would move every remaining item: $O(n)$.

What happens when the queue is full?

cqGrow doubles the buffer and copies the items in queue order, so the head lands at slot 0. That copy is $O(n)$ but rare, so enqueue stays $O(1)$ amortised.

How is " QUEZON city " understood?

normaliseKey lower-cases, trims and collapses runs of spaces, dots and commas, character by character. Every city and alias went into the table the same way, so any spelling hashes to the same bucket.

How is a GCash reference stopped from paying twice?

gcashProblem checks for 13 digits, then receiptWithRef searches the receipts for the same reference — a linear search, $O(n)$.

Why is a receipt a copy of the order?

A receipt records what was paid at that moment. If the order is edited later, the receipt must not change.

Why send emails through a queue?

"Order confirmed" must arrive before "ready for pickup", and only one request is sent at a time. The queue keeps that order, and a failed email can be queued again without being lost.

How does the customer agree to the Privacy Notice?

At checkout, two separate boxes that start unticked: one for the Privacy Notice (under the Data Privacy Act of 2012) and one for the order terms. `validateRequest` refuses the order unless both are ticked, and the order keeps a record of the agreement — which version of the notice, and when — in `order.consent` and its history.

Checkout & Delivery

Screens	Checkout step 2 — contact details, pickup (a date) or delivery (address, courier, date and time), and the two agreements
Files	js/store.js (buildAreaIndex, lookupArea, deliveryQuote, slotLoad, pickupPlacesLeft) · js/orders.js (validateRequest, isValidEmail, isValidPhone) · js/shop-basket.js (consentHtml) · js/shop-privacy.js
Data	areaIndex — hash table, 53 buckets; DELIVERY_AREAS (35 cities with aliases); COURIERS; orders (bookings already taken)
DSA	Hash table · Validation

1 Business process

What task or problem from the client are you solving?

At checkout the customer gives their name, email and mobile number and chooses pickup (a date only, Monday to Saturday) or delivery (address, date and time).

The delivery fee depends on the city: free in Brgy. Lawa, ₱20–₱50 for nearby towns the shop delivers to itself (Friday to Sunday), and a courier rate (Flash Express or Lalamove) for Metro Manila and the rest of Bulacan. The fee must be worked out the moment the city is typed, however it is typed.

Personal data may only be used with the customer's consent (Data Privacy Act of 2012). Before the order is placed the customer agrees, in two separate boxes, to the Privacy Notice and to the shop's order terms; both texts open in a sheet from the banner, the footer and checkout.

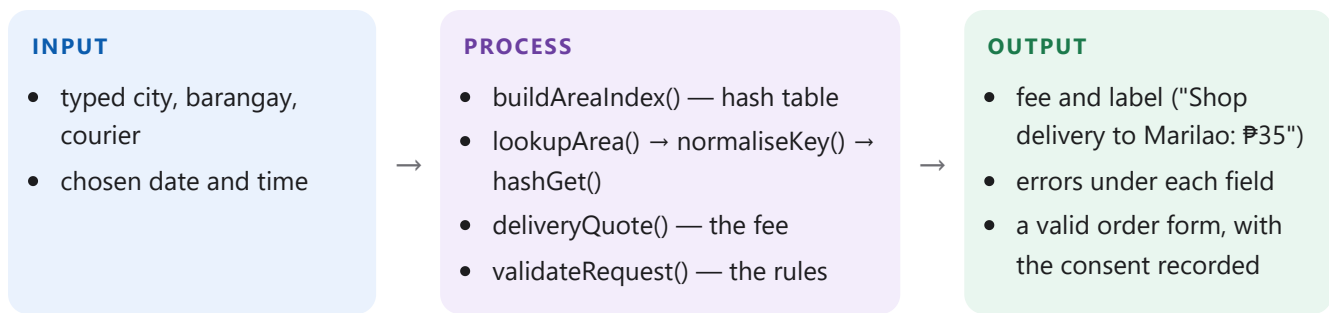
2 Input

What data is needed by the module?

Input	What it is
Customer types / picks	Name, email, mobile, pickup or delivery; street, barangay, city; courier; date; delivery time; rush; the two agreement boxes.
Stored: DELIVERY_AREAS	35 cities with aliases, their service (shop or courier) and fee or region.
Stored: COURIERS	Each courier's rate per region.
Stored: orders	Bookings already taken, for time-slot and pickup-day capacity.

3 Process

What data structure and/or algorithm did you use? How does it work step by step?



Step by step (Hash table · Validation):

- 1 At start-up, put every city name and alias, normalised, into a hash table (53 buckets).
- 2 When a city is typed: normalise it (" QUEZON city " → "quezon city"), hash it, scan that bucket — the area record in $O(1)$ on average. If "Marilao, Bulacan" is not found, try the part before the comma.
- 3 Shop area: the fee is the area's fee, or free if the barangay is Lawa. Courier area: look up the chosen courier's rate for the area's region.
- 4 Check everything by scanning characters (no regular expressions): the email format, the mobile number, the date against the lead time, Sundays (pickup) or Friday–Sunday (shop delivery), and the places left that day or time.
- 5 Both agreements must be ticked. The placed order records { privacyVersion, terms, at } in order.consent and a line in its history.

4 Output

What result does the module produce?

- The fee notice under the address, the courier choices with their rates, the places left for each time or pickup date, and field errors.
- Under every date and time — at checkout, on the confirmation, Track order, the receipt and the emails — a note that they are estimates: bad weather, traffic, road closures or courier delays can move them (scheduleNote()).

5 Complexity analysis

What is the time complexity? What is the space complexity?

Operation	Time	Space	Note
buildAreaIndex()	$O(n)$	$O(n)$	Puts every city and alias in the hash table.
lookupArea()	$O(1)$ average + $O(n)$ to tidy the text	$O(n)$	Finds the area for whatever was typed.
deliveryQuote()	$O(n)$	$O(1)$	Works out the fee.
validateRequest()	$O(n^2)$	$O(n)$	Checks the checkout form.

What n means here: For the look-up, n = the length of the typed city (to normalise it); the table holds 45 names. For the form check, n = the cart lines (each one's product is looked up — a loop inside a loop) and the orders (for capacity).

Each Time and Space is read from the comment above the function in the source, where every function in js/ states its own cost.

6 Justification

Why did you choose that data structure or algorithm? Why is it appropriate?

Customers type city names in every possible way. Normalising then hashing turns any spelling into a direct look-up — $O(1)$ on average — instead of comparing the input with all 45 names on every keystroke.

Aliases ("QC", "Sta Maria", "Paranaque") are simply extra keys pointing at the same record, which a hash table handles naturally.

Checking by scanning characters is required by the project rules (no regular expressions) and gives a precise message for each kind of mistake. Fees live in data, not code: the owner can add a city by adding one record.

7 Demonstration

Show that the module works correctly — the input, the process and the output.

- 1 Add a bouquet and check out. Choose Delivery and type " quezon CITY ": the courier choices appear with ₱120 / ₱220.
- 2 Type "Marilao": "Shop delivery to Marilao: ₱35", Friday to Sunday.
- 3 Choose Pickup: only a date — no time slots — and the places left that day.
- 4 Press Review without ticking the agreements: checkout stops, with a message under each box. "Privacy Notice" opens the notice.

The screenshot shows the 'Checkout' screen with the address '450 España Blvd.' and two courier options: 'Flash Express' for ₱120 and 'Lalamove' for ₱220. The delivery date is set to '10/09/2026'.

Delivery to Manila: courier choices with their rates

The screenshot shows the 'Checkout' screen with the pickup date set to '10/09/2026'. A message indicates that pickup dates are estimates and may be affected by weather or busy periods.

Pickup: a date only

The screenshot shows the 'Checkout' screen with two separate agreement sections: 'I agree to the Privacy Notice' and 'I agree to the order terms'. Both are marked as agreed.

The two agreements, asked separately

Trace — the delivery-area look-ups (computed by our code)

lookupArea() and deliveryQuote() for six typed addresses

Typed	normaliseKey	Bucket	Area	Label	Fee	Status
"quezon CITY "	quezon city	46	Quezon City	Lalamove to Quezon City (estimate)	₱220	estimate
"Marilao, Bulacan"	marilao bulacan	46	Marilao	Shop delivery to Marilao	₱35	final
"Meycauayan"	meycauayan	3	Meycauayan	Free delivery within Brgy. Lawa	₱0	final
"meycauayan city"	meycauayan city	40	Meycauayan	Shop delivery to Meycauayan	₱20	final
"QC"	qc	31	Quezon City	Flash Express to Quezon City (estimate)	₱120	estimate
"Davao"	davao	49	not found	Flash Express — fee set by the shop	₱0	manual

The table built by buildAreaIndex()

45 names and aliases in 53 buckets; 30 buckets in use; longest chain 3; load factor 0.85.

[46] quezon city → pasig → hagonoy
[31] qc → navotas
[48] marilao

validateRequest() — a pickup on Sunday 11 October with a bad email

Field	Message
email	Enter an email address like name@gmail.com — your confirmation goes there.
date	The shop is closed on Sundays. Choose another pickup date.

Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

js/store.js · lines 443–452 · buildAreaIndex() — Puts every city and alias in the hash table.

```
443  /* Every city and alias, normalised, to its area.  Time O(n) · Space O(n) */
444  function buildAreaIndex() {
445  ❶    var table = hashCreate(53);
446      for (var i = 0; i < DELIVERY_AREAS.length; i++) {
447          var area = DELIVERY_AREAS[i];
448  ❷    hashPut(table, normaliseKey(area.city), area);
449  ❸    for (var k = 0; k < area.aliases.length; k++) hashPut(table, normaliseKey(area.aliases[k]), area);
450      }
451      return table;
452  }
```

- ❶ line 445 53 buckets for 45 keys.
- ❷ line 448 The city's normalised name...
- ❸ line 449 ...and each alias, all pointing at the same record.

Time $O(n)$ · Space $O(n)$

js/store.js · lines 454–462 · lookupArea() — Finds the area for whatever was typed.

```
454  /* " quezon  city " finds Quezon City.  Time  $O(1)$  average +  $O(n)$  to tidy the text · Space  $O(n)$  */
455  function lookupArea(city) {
456      if (isBlank(city)) return null;
457  ❶    var found = hashGet(areaIndex, normaliseKey(city));
458      if (found) return found;
459      // "Marilao, Bulacan" - try the part before the first comma.
460  ❷    var head = cutText(city, ',')[0];
461      return hashGet(areaIndex, normaliseKey(head));
462  }
```

❶ Line 457 Normalise and look up.

❷ Line 460 Not found? Try the part before a comma ("Marilao, Bulacan").

Time $O(1)$ average + $O(n)$ to tidy the text · Space $O(n)$

js/store.js · lines 483–511 · deliveryQuote() — Works out the fee.

```
483  /* Work out the delivery fee for an address.  Time  $O(n)$  · Space  $O(1)$ 
484      status: final    the shop's own fee, nothing to confirm
485      estimate a courier planning rate, the owner confirms it
486      manual  no rate known, the owner enters it          */
487  function deliveryQuote(f) {
488      if (!f || f.mode !== 'delivery') {
489  ❶    return { service: 'pickup', fee: 0, status: 'final', label: 'Pickup at the shop', area: '',
        courier: '' };
490      }
491      var area = lookupArea(f.city);
492      if (area && area.service === 'inhouse') {
493  ❷    var free = isIn(area.freeBarangays || [], barangayKey(f.barangay));
494      return {
495  ❸    service: 'inhouse', fee: free ? 0 : area.fee, status: 'final', area: area.city, courier: '',
        label: free ? 'Free delivery within Brgy. Lawa' : 'Shop delivery to ' + area.city
496      };
497      }
498      }
499      var courier = courierById(f.courier);
500      var courierName = courier ? courier.name : 'Courier';
501      if (area && courier) {
502  ❹    var rate = courierRate(courier.id, area.region);
503      if (rate > 0) {
504      return { service: 'courier', fee: rate, status: 'estimate', area: area.city, courier:
        courier.id,
505      label: courierName + ' to ' + area.city + ' (estimate)' };
506      }
507      }
508  ❺    return { service: 'courier', fee: 0, status: 'manual', area: area ? area.city : strip(f.city),
        courier: courier ? courier.id : '',
509      label: courierName + ' - fee set by the shop' };
510      }
511  }
```

❶ Line 489 Pickup: free.

❷ Line 493 Shop area: free in Brgy. Lawa...

❸ Line 495 ...otherwise the area's fee.

❹ Line 502 Courier area: the chosen courier's rate for the region.

❺ Line 508 No rate on file: the fee must be set by the owner.

Time $O(n)$ · Space $O(1)$

```

150  /* form: { name, phone, handle, mode, date, slot, address, barangay, city,
151           courier, rush, notes, consent, terms } - consent: the customer
152           agreed to the Privacy Notice; terms: to the order terms. Both are
153           asked separately and both are required.
154           Returns a list of { field, message }; empty means valid.
155           Time O(n²) · Space O(n) */
156  function validateRequest(form, items, fromIso, ignoreId) {
157      var errors = [];
158      /* Record one problem.           Time O(1) · Space O(1) */
159      function fail(field, message) { listAdd(errors, { field: field, message: message }); }
160
161      if (items.length === 0) fail('basket', 'Add at least one item to your cart.');
```

162 for (var i = 0; i < items.length; i++) {

163 var product = productById(items[i].productId);

164 if (!product || !product.active) fail('basket', items[i].productName + ' is no longer available. Remove it to continue.');

165 }

166 var name = strip(form.name);

167 if (name.length < 2 || name.length > 60) fail('name', 'Enter your name (2 to 60 characters).');

168 if (!isValidPhone(form.phone)) fail('phone', 'Enter a mobile number like 0917 123 4567.');

169 ❶ if (!isValidEmail(form.email)) fail('email', 'Enter an email address like name@gmail.com - your confirmation goes there.');

170 if (String(form.handle || '').length > 60) fail('handle', 'Keep the Facebook or Instagram name under 60 characters.');

171 if (String(form.notes || '').length > MAX_NOTE) fail('notes', 'Notes are limited to ' + MAX_NOTE + ' characters.');

172 ❷ if (form.consent !== true) fail('consent', 'Please read the Privacy Notice and tick the box, so the shop may use your details for this order.');

173 ❸ if (form.terms !== true) fail('terms', 'Please read the order terms and tick the box to agree to them.');

174

175 if (!modeById(form.mode)) fail('mode', 'Choose pickup or delivery.');

176 var earliest = earliestDate(items, form.rush === true, fromIso);

177 if (!dateFromIso(form.date)) fail('date', 'Choose a date.');

178 ❹ else if (form.date < earliest) fail('date', 'The earliest the shop can have this ready is ' + formatDateLong(earliest) + '.');

179

180 if (form.mode === 'delivery') {

181 if (isBlank(form.address)) fail('address', 'Enter the street and house number.');

182 if (isBlank(form.city)) fail('city', 'Enter the city or town.');

183 else if (needsCourier(form.city)) {

184 if (!courierById(form.courier)) fail('courier', 'Choose a courier for this address.');

185 } else if (dateFromIso(form.date) && !isInhouseDay(form.date)) {

186 ❺ fail('date', 'The shop delivers on ' + INHOUSE_DAY_NAMES + ' only.');

187 }

188 }

189

190 if (form.mode === 'delivery') {

191 if (!isIn(TIME_SLOTS, form.slot)) fail('slot', 'Choose a delivery time.');

192 else if (dateFromIso(form.date) && slotLoad(form.date, form.slot, ignoreId) >= SLOT_CAPACITY) {

193 fail('slot', 'That time is fully booked. Choose another.');

194 }

195 } else if (dateFromIso(form.date) && weekdayOf(form.date) === 0) {

196 ❻ fail('date', 'The shop is closed on Sundays. Choose another pickup date.');

197 } else if (dateFromIso(form.date) && pickupPlacesLeft(form.date, ignoreId) === 0) {

198 // Pickups are booked by date alone.

199 fail('date', 'Pickups are fully booked on ' + formatDateLong(form.date) + '. Choose another date.');

200 }

201 // A price-list cart is never a quote request, so it needs a delivery

202 // rate on file. Only a cart with a quote product can ask for one.

203 var hasQuoteItem = countWhere(items, function (item) { return item.kind === 'quote'; }) > 0;

204 if (form.mode === 'delivery' && !isBlank(form.city) && courierById(form.courier) && !hasQuoteItem && deliveryQuote(form).status === 'manual') {

205

```
206 5      fail('city', 'We have no delivery rate for ' + strip(form.city) + ' yet. Choose pickup or one of
        the listed cities, or message us at ' +
207          SHOP_INFO.social + '.');
208      }
209      return errors;
210  }
```

- ❶ line 169 An email is required — confirmations go there.
- ❷ line 178 Not earlier than the lead time allows.
- ❸ line 186 Shop deliveries only Friday to Sunday.
- ❹ line 196 Pickups: Monday to Saturday, by date only.
- ❺ line 206 A price-list cart needs a rate on file — it is never turned into a quote.
- ❻ line 172 The Privacy Notice must be agreed to...
- ❼ line 173 ...and, separately, the order terms.

Time $O(n^2)$ · Space $O(n)$

Quotation Desk

Screens	Order desk → "Quote requests" tab, and the quote editor
Files	js/structures.js (cqEnqueue, cqDequeue, cqGrow) · js/orders.js (leaveQueue, quoteHistory, recentQuotes, pastQuoteFrom, quoteDraft, sendQuote) · js/admin-desk.js · js/admin-orders.js
Data	quoteQueue — circular queue of order ids { items, head, tail, count, capacity }; orders (past quotations)
DSA	Circular queue · Insertion sort

1 Business process

What task or problem from the client are you solving?

Quote requests for the five custom gifts arrive while the owner is busy. They must be answered fairly — first come, first served — and pricing each one by hand from scratch is slow.

So requests wait in a first-in, first-out queue, and each quotation is pre-filled from the owner's own past quotes for the same product and size; usually the owner only checks it and presses send.

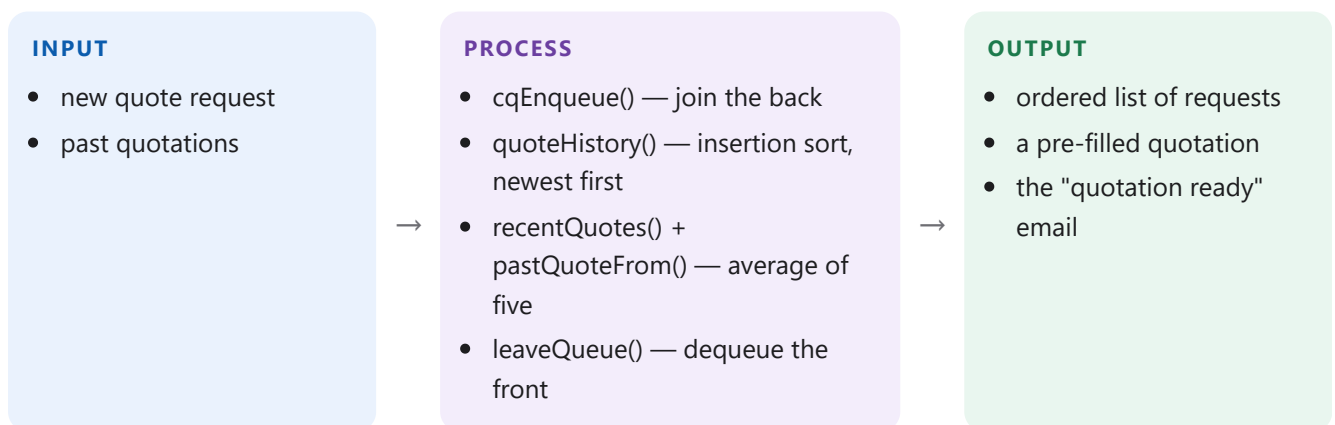
2 Input

What data is needed by the module?

Input	What it is
Stored: quoteQueue	Order ids waiting for a quotation, in arrival order.
Stored: orders	Past quotations: product, size, materials, labour, quantity, date.
Owner types	Materials, labour and item cost per line; delivery and rush fees; a note.

3 Process

What data structure and/or algorithm did you use? How does it work step by step?



Step by step (Circular queue · Insertion sort):

- 1 A request joins the back of a circular queue (enqueue). The desk lists the queue front to back; the front is next.

- 2 Pre-fill: quoteHistory() reads every quoted order once and collects its quoted lines, per piece and without their add-ons; insertion sort puts them newest first. For each line of the request, recentQuotes() walks that list and stops at the first five for the same product and size (or any size, marked); their average, times the quantity, fills the line.
- 3 When the quotation is sent, the request leaves the queue — from the front in $O(1)$ when it is the next in line (dequeue) — and the customer is emailed.
- 4 When the buffer is full it doubles in size, unrolling so the head is at slot 0.

4 Output

What result does the module produce?

- The "Quote requests" tab in order, with position numbers.
- The quote editor, pre-filled and marked "Filled in from your last N quotes".
- The quotation email to the customer.

5 Complexity analysis

What is the time complexity? What is the space complexity?

Operation	Time	Space	Note
cqEnqueue()	$O(1)$ amortised ($O(n)$ on a grow)	$O(1)$	Joins the back of the queue.
cqDequeue()	$O(1)$	$O(1)$	Leaves from the front.
cqGrow()	$O(n)$	$O(n)$	Doubles a full buffer.
leaveQueue()	$O(n)$	$O(n)$	Takes an id out when it is quoted or voided.
quoteHistory()	$O(n^2)$	$O(n)$	Every quoted line the owner has priced, newest first.
recentQuotes()	$O(n)$	$O(1)$	The five most recent quotes for one product.
pastQuoteFrom()	$O(n)$	$O(1)$	What the owner charged before.
sendQuote()	$O(n)$	$O(n)$	Sends the quotation.

What n means here: For the queue, n = requests waiting. For the pre-fill, n = past quoted lines — every line of every quoted order.

Each Time and Space is read from the comment above the function in the source, where every function in js/ states its own cost.

6 Justification

Why did you choose that data structure or algorithm? Why is it appropriate?

A queue is the fair-service structure: first in, first out. A circular buffer reuses the slots freed at the front, so enqueue and dequeue are $O(1)$ and the array never creeps forward as removing from the front of a plain array would.

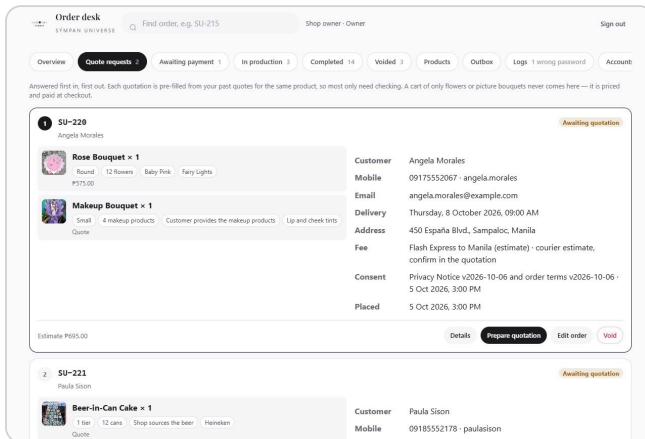
Insertion sort puts the past quotes in date order and is stable. The list is read back to front first, so it is already nearly newest first — close to the $O(n)$ best case. The history is built once per draft, so each line of the request searches one list instead of every order again.

Averaging the five most recent quotes follows the owner's current prices rather than old ones, and pre-filling from the owner's own history automates the job without inventing prices.

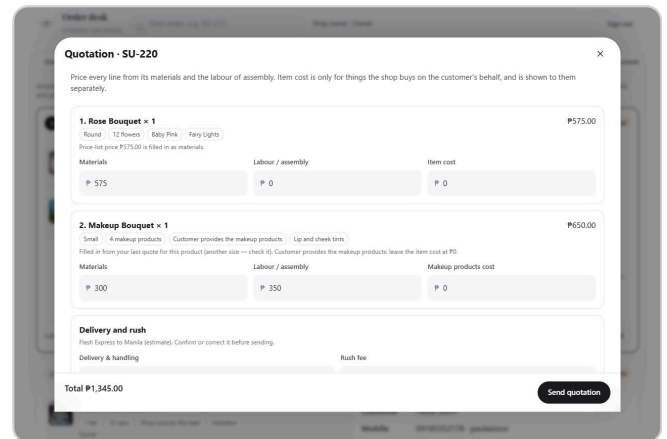
7 Demonstration

Show that the module works correctly — the input, the process and the output.

- 1 Check out a cart with a Money Bouquet: it becomes a quote request.
- 2 In the order desk, open "Quote requests": it is last in line; SU-220 is first.
- 3 Press "Prepare quotation": the figures are already filled in from past Money Bouquet quotes. Send it.



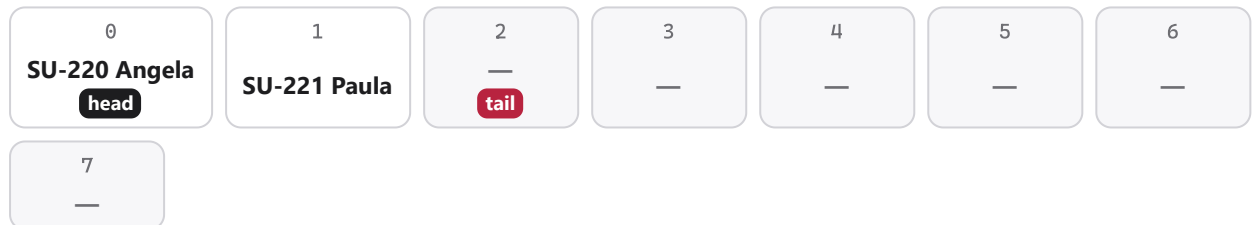
Quote requests, first in first out



The quote editor, pre-filled from past quotes

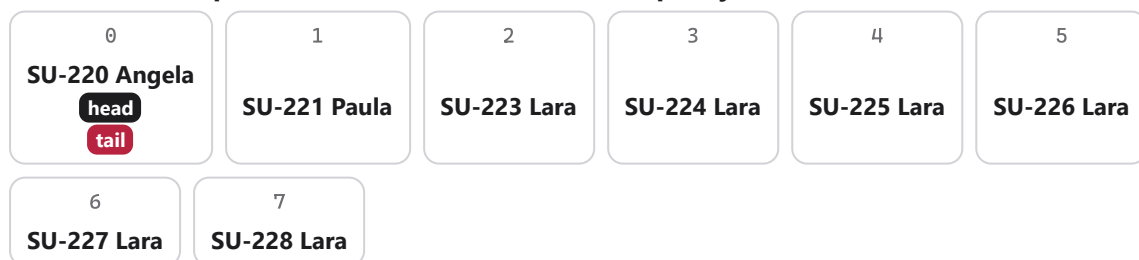
Trace — the quote queue and the pre-fill (computed by our code)

The quote queue after the demo history



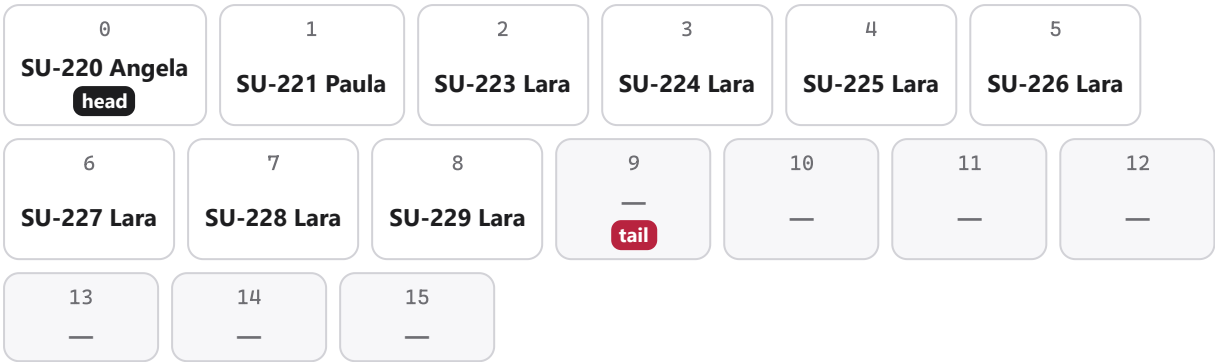
$$\text{capacity} = 8 \cdot \text{count} = 2 \cdot \text{head} = 0 \cdot \text{tail} = 2$$

After 6 more requests: the buffer is full (count = capacity)



$$\text{capacity} = 8 \cdot \text{count} = 8 \cdot \text{head} = 0 \cdot \text{tail} = 0$$

The 7th request: `cqEnqueue()` finds it full, `cqGrow()` doubles it and unrolls it so the head is at 0



capacity = 16 · count = 9 · head = 0 · tail = 9

`sendQuote()` on the front request: `leaveQueue()` dequeues it in $O(1)$



capacity = 16 · count = 8 · head = 1 · tail = 9

`quoteDraft()` for a Medium Money Bouquet — the owner's past quotes, per piece without add-ons

`quoteHistory()` read every quoted order once and insertion-sorted its 9 quoted lines newest first.

`recentQuotes()` then walked that list and stopped after the first five for this product and size:

Quoted on	Materials	Labour
28 Aug 2026, 4:00 PM	₱250	₱300

Their average: materials ₱250, labour ₱300 per piece (from 1 quote). `quoteDraft()` fills these in for the owner, scaled by the quantity.

Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

`js/structures.js` · lines 137–144 · `cqEnqueue()` — Joins the back of the queue.

```
137  /* Join at the back.           Time O(1) amortised (O(n) on a grow) · Space O(1) */
138  function cqEnqueue(queue, value) {
139  ❶    if (queue.count === queue.capacity) cqGrow(queue);
140  ❷    queue.items[queue.tail] = value;
141  ❸    queue.tail = (queue.tail + 1) % queue.capacity;
142    queue.count++;
143    return queue.count;
144  }
```

❶ line 139 Full? Double the buffer first.

❷ line 140 Write at the tail...

3 line 141 ...and move the tail on, wrapping around with modulo.

Time $O(1)$ amortised ($O(n)$ on a grow) · Space $O(1)$

js/structures.js · lines 146–154 · cqDequeue() — Leaves from the front.

```
146  /* Leave from the front, or null when empty. Time  $O(1)$  · Space  $O(1)$  */
147  function cqDequeue(queue) {
148      if (queue.count === 0) return null;
149  1  var value = queue.items[queue.head];
150      queue.items[queue.head] = null;
151  2  queue.head = (queue.head + 1) % queue.capacity;
152      queue.count--;
153      return value;
154  }
```

1 Line 149 Read the front.

2 Line 151 Move the head on, wrapping around.

Time $O(1)$ · Space $O(1)$

js/structures.js · lines 125–135 · cqGrow() — Doubles a full buffer.

```
125  /* Double the buffer when full, unrolling it so head is index 0.
126                                     Time  $O(n)$  · Space  $O(n)$  */
127  function cqGrow(queue) {
128      var bigger = [], cap = queue.capacity * 2;
129      for (var i = 0; i < cap; i++) bigger[i] = null;
130  1  for (var k = 0; k < queue.count; k++) bigger[k] = queue.items[(queue.head + k) % queue.capacity];
131      queue.items = bigger;
132      queue.capacity = cap;
133      queue.head = 0;
134      queue.tail = queue.count;
135  }
```

1 Line 130 Copy the items in queue order, so the head lands at slot 0.

Time $O(n)$ · Space $O(n)$

js/orders.js · lines 335–341 · leaveQueue() — Takes an id out when it is quoted or voided.

```
335  /* Take an id out of a queue: from the front in  $O(1)$  when it is the next
336      in line – the usual case – or from the middle in  $O(n)$  when it is not.
337                                     Time  $O(n)$  · Space  $O(n)$  */
338  function leaveQueue(queue, id) {
339  1  if (cqFront(queue) === id) return cqDequeue(queue) === id;
340  2  return cqRemove(queue, id);
341  }
```

1 Line 339 Normally it is at the front: $O(1)$ dequeue.

2 Line 340 Otherwise remove it from the middle, keeping the order.

Time $O(n)$ · Space $O(n)$

js/orders.js · lines 354–379 · quoteHistory() — Every quoted line the owner has priced, newest first.

```
354  /* Every quoted line of a quote product the shop has priced, per piece and
355      without its add-ons, newest quotation first. Learned from the order
356      records, so it improves as the shop quotes more. Built once per draft:
357      each line of the new order then searches this one list.
358                                     Time  $O(n^2)$  · Space  $O(n)$  */
359  function quoteHistory(ignoreId) {
360      var seen = [];
```

```

361     for (var o = 0; o < orders.length; o++) {
362         var order = orders[o];
363 ❶ if (order.id === ignoreId || order.quotedAt === 0 || order.status === 'requested') continue;
364         for (var i = 0; i < order.items.length; i++) {
365             var past = order.items[i];
366             if (past.kind === 'quote' && lineTotal(past) > 0) {
367                 // Take the past add-ons out, so the new line's own add-ons are
368                 // not counted twice when they are added back.
369 ❷ var base = past.materials - past.estimate;
370                 listAdd(seen, { seq: seen.length, productId: past.productId, size: past.size, stamp:
order.quotedAt,
371                             materials: (base > 0 ? base : 0) / past.quantity, labor: past.labor /
past.quantity });
372             }
373         }
374     }
375     // Newest first. Orders are kept oldest first, so reading the list back to
376     // front leaves it nearly in order – insertion sort's best case. Lines
377     // quoted at the same moment keep their order (seq), first line first.
378 ❸ return insertionSort(backwards(seen), function (a, b) { return b.stamp - a.stamp || a.seq - b.seq; });
379 }

```

❶ Line 363 Only orders the owner has already quoted.

❷ Line 369 Per piece, with the add-ons taken out.

❸ Line 378 Read back to front, then insertion sort newest first; lines quoted at the same moment keep their order.

Time $O(n^2)$ · Space $O(n)$

js/orders.js · lines 381–391 · recentQuotes() — The five most recent quotes for one product.

```

381  /* The five most recent history lines for the item's product – only at
382     the same size when sameSize is true. The history is newest first, so
383     a linear search that stops at five is enough. Time  $O(n)$  · Space  $O(1)$  */
384  function recentQuotes(history, item, sameSize) {
385      var out = [];
386 ❶ for (var h = 0; h < history.length && out.length < 5; h++) {
387          var line = history[h];
388 ❷ if (line.productId === item.productId && (!sameSize || line.size === item.size)) listAdd(out,
line);
389      }
390      return out;
391  }

```

❶ Line 386 Walk the history from the newest, stopping after five.

❷ Line 388 Keep lines of the same product (and size).

Time $O(n)$ · Space $O(1)$

js/orders.js · lines 393–406 · pastQuoteFrom() — What the owner charged before.

```

393  /* What the owner charged before for the same product and size, per piece:
394     the average of the five most recent quotations, or of any size when
395     that size was never quoted. Returns { materials, labor, count, sameSize }
396     per piece, or null with no history. Time  $O(n)$  · Space  $O(1)$  */
397  function pastQuoteFrom(history, item) {
398 ❶ var recent = recentQuotes(history, item, true), sameSize = recent.length > 0;
399 ❷ if (!sameSize) recent = recentQuotes(history, item, false);
400     if (recent.length === 0) return null;
401     return {
402 ❸ materials: roundMoney(sumRecursive(recent, function (r) { return r.materials; }) / recent.length),
403         labor: roundMoney(sumRecursive(recent, function (r) { return r.labor; }) / recent.length),
404         count: recent.length, sameSize: sameSize
405     };

```

```
406 }
```

- 1 line 398 The same size first...
- 2 line 399 ...or any size if there is none.
- 3 line 402 Average them.

Time $O(n)$ · Space $O(1)$

js/orders.js · lines 470–500 · sendQuote() — Sends the quotation.

```
470  /* Send (or revise) the quotation.          Time O(n) · Space O(n) */
471  function sendQuote(id, draft, stamp) {
472      var order = orderById(id);
473      if (!order) return { ok: false, errors: [{ field: 'order', message: 'No such order.' }] };
474      if (order.status !== 'requested' && order.status !== 'quoted') {
475          return { ok: false, errors: [{ field: 'order', message: order.ref + ' is past the quotation stage.'
476      }} ];
477      1 var errors = validateQuote(order, draft);
478      if (errors.length > 0) return { ok: false, errors: errors };
479
480      var priced = [];
481      for (var i = 0; i < order.items.length; i++) {
482          listAdd(priced, copyRecord(order.items[i], {
483              materials: roundMoney(draft.lines[i].materials),
484              labor: roundMoney(draft.lines[i].labor),
485              itemExpense: roundMoney(draft.lines[i].itemExpense)
486          }));
487      }
488      var revising = order.status === 'quoted';
489      order.items = priced;
490      order.deliveryFee = roundMoney(draft.deliveryFee);
491      order.rushFee = roundMoney(draft.rushFee);
492      order.quoteNote = strip(draft.note);
493      order.quotedAt = stamp;
494      if (order.delivery.service !== 'pickup') order.delivery = settledDelivery(order, order.delivery,
495      order.deliveryFee);
496      2 if (!revising) leaveQueue(quoteQueue, order.id);
497      order.status = 'quoted';
498      addHistory(order, stamp, (revising ? 'Quotation revised: ' : 'Quotation sent: ') +
499      peso(orderTotal(order)), deskActor());
500      3 emailQuoteReady(order, stamp);
501      return { ok: true, order: order };
502  }
```

- 1 line 477 Every line must be priced; a courier fee must be set.
- 2 line 495 Leave the queue.
- 3 line 498 Email the customer that it is approved.

Time $O(n)$ · Space $O(n)$

Payment, Receipts & Email

Screens	Checkout step 3 (payment), the receipt, Track order (paying a quotation), Order desk → Outbox
Files	js/orders.js (gcashProblem, receiptWithRef, placeOrder, acceptQuote, issueReceipt) · js/notify.js (queueEmail) · js/mail.js (drainMail) · js/email-config.js
Data	receipts — array of snapshots; outbox — array of emails; mailQueue — circular queue of outbox positions; EMAIL_CONFIG
DSA	Circular queue (outgoing mail)

1 Business process

What task or problem from the client are you solving?

The shop takes only two payments: a 50% down payment or full payment, both by GCash with the 13-digit reference number. Every payment needs a receipt, and the customer must be told by email: order confirmed (with the tracking number), quotation ready, payment received, ready for pickup, cancelled.

A GCash reference can only pay once. Emails must go out in the order they were written, and none may be lost if the connection drops.

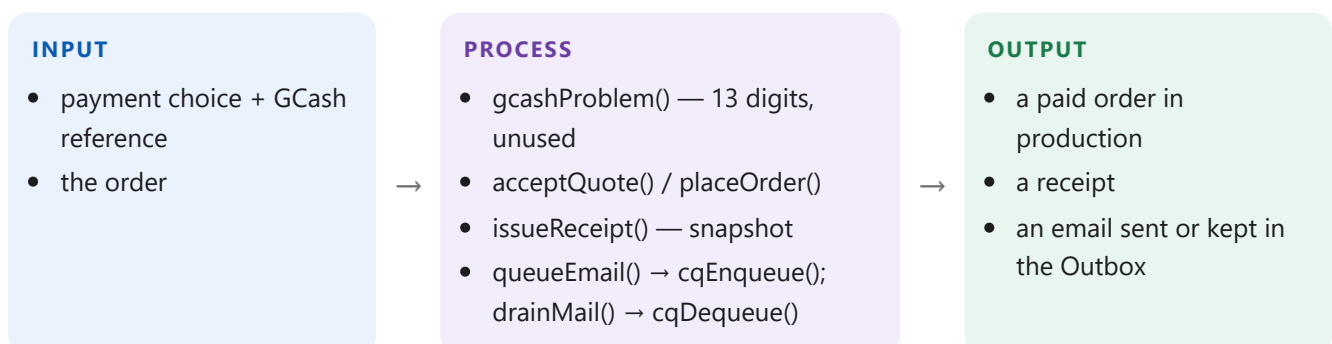
2 Input

What data is needed by the module?

Input	What it is
Customer picks / types	50% or 100%; the GCash reference number.
Stored: the order	Lines, totals, customer email.
Stored: receipts	Every reference already used.
Stored: EMAIL_CONFIG	The EmailJS IDs (blank = emails kept in the Outbox, marked "not sent").

3 Process

What data structure and/or algorithm did you use? How does it work step by step?



Step by step (Circular queue (outgoing mail)):

- 1 Check the reference: 13 digits, and not already on any receipt (a linear search).
- 2 Price-list cart: place the order and take the payment in one step; if anything fails, no order is left behind.

- 3 Record the payment; take a snapshot of the order into a numbered receipt (OR-0001...), so later edits never change it.
- 4 Write the email, store it in the outbox array and put its position in a circular queue. Every 1.5 s the sender takes the front of the queue and posts it to EmailJS, one at a time, marking it sent or failed.

4 Output

What result does the module produce?

- The receipt (printable) with paid and outstanding amounts.
- The "Order confirmed" screen with the tracking number.
- Emails to the customer, and the Outbox tab listing each one.

5 Complexity analysis

What is the time complexity? What is the space complexity?

Operation	Time	Space	Note
gcashProblem()	$O(n)$	$O(n)$	The checks every payment shares.
placeOrder()	$O(n^2)$	$O(n)$	Checkout for a price-list cart: place and pay in one step.
acceptQuote()	$O(n)$	$O(n)$	Records the payment.
issueReceipt()	$O(n)$	$O(n)$	Snapshots the order into a receipt.
queueEmail()	$O(n)$	$O(n)$	Writes an email and queues it.
drainMail()	$O(n^2)$ – each send redraws the outbox	$O(1)$	Sends the queue, one at a time.

What n means here: For the reference check, n = receipts so far. For the queue, n = emails waiting. For a receipt, n = the order's lines.

Each Time and Space is read from the comment above the function in the source, where every function in js/ states its own cost.

6 Justification

Why did you choose that data structure or algorithm? Why is it appropriate?

Emails must leave in the order they were written ("confirmed" before "ready"), and only one may be in flight at a time — a first-in, first-out queue is exactly that. The circular buffer keeps enqueue and dequeue $O(1)$.

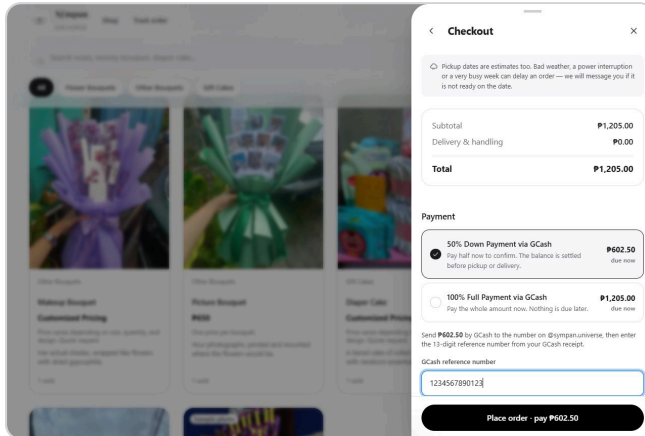
Receipts are snapshots (copies), not live views of the order, because a receipt records what was paid at that moment.

Checking the reference against every receipt is a linear search, $O(n)$; with a few hundred receipts a year that is instant and needs no extra index.

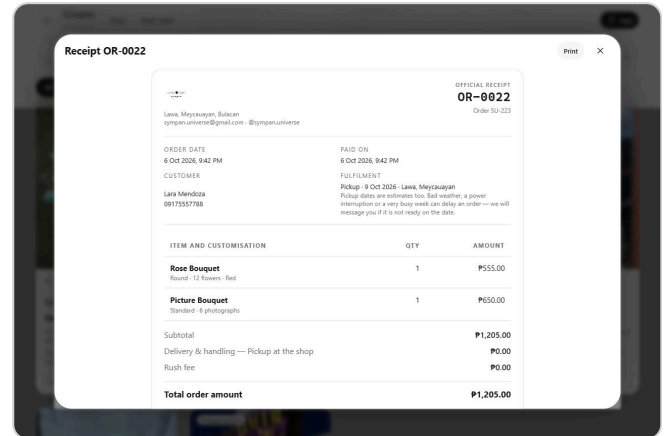
7 Demonstration

Show that the module works correctly — the input, the process and the output.

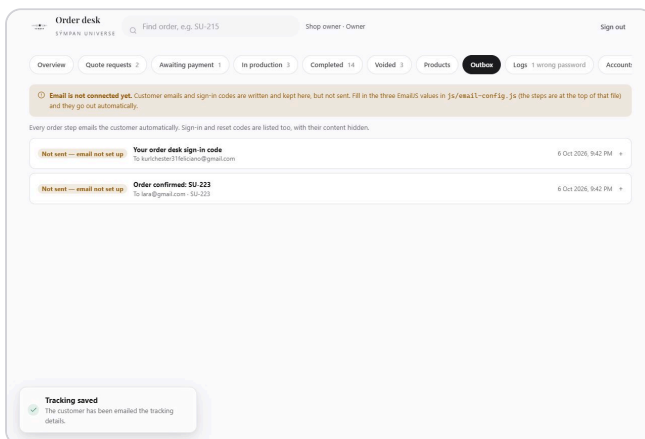
- 1 Add a Rose Bouquet, check out with pickup, choose "100% Full Payment via GCash", enter 1234567890123 and place the order.
- 2 The receipt opens; the confirmation shows the tracking number.
- 3 Try to pay another order with the same reference: refused.
- 4 In the order desk, open "Outbox": the "Order confirmed" email is there.



Checkout: the two GCash payments



The receipt



The Outbox

Trace — a payment, its receipt and its email (computed by our code)

placeOrder() with a 5-digit reference

Refused before anything is created: "Enter the 13-digit GCash reference number.". Orders still: 22.

placeOrder() with "1234 567 890123", 50% down payment

Receipt field	Value
Receipt no.	OR-0022
Order (tracking) no.	SU-223
Customer	Lara Mendoza, 09175557788, lara@gmail.com
Items	Rose Bouquet × 1 — Round, 12 flowers, Red, Card · Rose
Subtotal	₱575.00
Delivery & handling	₱0.00
Total	₱575.00
Method	50% Down Payment via GCash
Amount paid	₱287.50
Remaining balance	₱287.50
Status	Partially paid

Order SU-223 is now "In production".

payBalance() with the same reference

"That GCash reference is already on receipt OR-0022. Check the number."

queueEmail() — the confirmation joins the mail queue

outbox[0]: to lara@gmail.com, subject "Order confirmed: SU-223", status queued.



capacity = 8 · count = 1 · head = 0 · tail = 1

Hi Lara Mendoza,

Your order is confirmed and we are making it now.

• Rose Bouquet × 1 (Round, 12 flowers, Red, Card · Rose) – ₱575.00

Total: ₱575.00

Down payment: ₱287.50 by GCash (ref 1234567890123)

Remaining balance: ₱287.50 – due before pickup

Receipt: OR-0022

Pickup: Thursday, 8 October 2026

Pickup dates are estimates too. Bad weather, a power interruption or a very busy week can delay an order – we will message you if it is not ready on the date.

Tracking number: SU-223

Keep this number. Quote it when you message us, or use Track order on our site with your email or mobile number.

Thank you,

Sýmpan Universe

Lawa, Meycauayan, Bulacan · @sympan.universe

drainMail() with EmailJS not set up

Dequeued; status is now "simulated" — kept in the Outbox, not sent. Queue count: 0.

Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

js/orders.js · lines 603–609 · gcashProblem() — The checks every payment shares.

```
603  /* The checks every GCash payment shares: '' when it may go ahead.
604                                     Time O(n) · Space O(n) */
605  function gcashProblem(gcashRef) {
606  ❶    if (!isValidGcashRef(gcashRef)) return 'Enter the 13-digit GCash reference number.';
607  ❷    var used = receiptWithRef(gcashRef);
608        return used ? 'That GCash reference is already on receipt ' + used.no + '. Check the number.' : '';
609  }
```

❶ line 606 Exactly 13 digits.

❷ line 607 Not already used on a receipt.

Time $O(n)$ · Space $O(n)$

js/orders.js · lines 302–324 · placeOrder() — Checkout for a price-list cart: place and pay in one step.

```
302  /* Checkout for a priced cart: the order is placed and paid in one step, so
303     nothing waits on the owner. Nothing is created unless the payment
304     details are valid.           Time  $O(n^2)$  · Space  $O(n)$  */
305  function placeOrder(form, methodId, gcashRef, stamp) {
306      var items = repricedCart();
307      ❶ var errors = validateRequest(form, items, isoFromStamp(stamp));
308      if (errors.length > 0) return { ok: false, errors: errors };
309      var preview = deliveryQuote(form);
310      if (needsQuote(items, preview)) return { ok: false, errors: [{ field: 'basket', message: 'This cart
needs a quote from the shop first.' }] };
311      if (!paymentById(methodId)) return { ok: false, errors: [{ field: 'payment', message: 'Choose 50% down
payment or full payment.' }] };
312      ❷ var problem = gcashProblem(gcashRef);
313      if (problem) return { ok: false, errors: [{ field: 'gcash', message: problem }] };
314
315      ❸ var made = submitRequest(form, stamp);
316      if (!made.ok) return made;
317      ❹ var paid = acceptQuote(made.order.id, methodId, gcashRef, stamp);
318      if (!paid.ok) {
319          // Should never happen after the checks above; never leave a half-made order.
320      ❺ voidOrder(made.order.id, 'Payment could not be recorded', 'system', stamp);
321          return { ok: false, errors: [{ field: 'gcash', message: paid.error }] };
322      }
323      return { ok: true, order: made.order, receipt: paid.receipt };
324  }
```

- ❶ line 307 Check the form first.
- ❷ line 312 Check the payment before anything is created.
- ❸ line 315 Create the order (priced automatically).
- ❹ line 317 Take the payment.
- ❺ line 320 Never leave a half-made order.

Time $O(n^2)$ · Space $O(n)$

js/orders.js · lines 611–636 · acceptQuote() — Records the payment.

```
611  /* The quotation is accepted and paid – by the customer on Track order, or
612     recorded by the order desk (actor: the person signed in). Time  $O(n)$  · Space  $O(n)$  */
613  function acceptQuote(id, methodId, gcashRef, stamp, actor) {
614      var order = orderById(id);
615      if (!order || order.status !== 'quoted') return { ok: false, error: 'This order has no quotation
waiting for payment.' };
616      ❶ if (stamp - order.quotedAt > QUOTE_EXPIRY_DAYS * 24 * 3600000) {
617          voidOrder(order.id, 'Quotation expired – not paid within ' + QUOTE_EXPIRY_DAYS + ' days', 'system',
stamp);
618          return { ok: false, error: 'This quotation has expired. Send a new request and the shop will quote
it again.' };
619      }
620      var method = paymentById(methodId);
621      if (!method) return { ok: false, error: 'Choose 50% down payment or full payment.' };
622      var problem = gcashProblem(gcashRef);
623      if (problem) return { ok: false, error: problem };
624      if (orderTotal(order) <= 0) return { ok: false, error: 'This quotation has no amount to pay.' };
625
626      var total = orderTotal(order);
627      order.paymentMethod = method.id;
628      ❷ order.amountPaid = method.share === 1 ? total : roundMoney(total * method.share);
629      order.status = 'paid';
```

```

630 ③ enterProduction(order);
631      addHistory(order, stamp, 'Quotation accepted - ' + peso(order.amountPaid) + ' paid by GCash', actor ||
    'Customer',
632          'GCash reference ' + digitsOnly(gcashRef));
633 ④ var receipt = issueReceipt(order, method.share === 1 ? 'Full payment' : 'Down payment',
    order.amountPaid, gcashRef, stamp);
634 ⑤ emailOrderConfirmed(order, receipt, stamp);
635      return { ok: true, order: order, receipt: receipt };
636  }

```

- ① Line 616 An expired quotation cannot be paid.
- ② Line 628 Full or half of the total.
- ③ Line 630 Into the production line (module 10).
- ④ Line 633 Issue the receipt...
- ⑤ Line 634 ...and the confirmation email.

Time $O(n)$ · Space $O(n)$

js/orders.js · lines 546–579 · issueReceipt() — Snapshots the order into a receipt.

```

546  /* Snapshot the order as it stands at the moment money changes hands, so a
547      receipt never shifts when the order is edited later.
548                               Time  $O(n)$  · Space  $O(n)$  */
549  function issueReceipt(order, kind, amount, gcashRef, stamp) {
550      var lines = [];
551      for (var i = 0; i < order.items.length; i++) {
552          var item = order.items[i];
553          listAdd(lines, { name: item.productName, quantity: item.quantity, specs: itemSpecs(item),
554              notes: item.notes, amount: lineTotal(item) });
555      }
556      var method = paymentById(order.paymentMethod);
557      // A balance is paid by GCash after the original payment, so say so
558      // rather than repeating "100% Full Payment" on a top-up receipt.
559      var methodName = kind === 'Balance payment'
560          ? 'GCash - balance after ' + (method ? toLower(method.short) : 'down payment')
561          : (method ? method.name : 'GCash');
562      var receipt = {
563  ①      no: 'OR-' + leftPad(counters.receipt, 4, '0'),
564          orderId: order.id, ref: order.ref, stamp: stamp, orderDate: order.createdAt,
565          kind: kind, method: methodName, gcashRef: digitsOnly(gcashRef),
566          amount: roundMoney(amount),
567          customer: copyRecord(order.customer),
568          fulfilment: fulfilmentSummary(order), mode: order.fulfilment.mode,
569          items: lines,
570          subtotal: orderSubtotal(order), deliveryFee: order.deliveryFee,
571          deliveryLabel: order.delivery.label, rushFee: order.rushFee,
572          total: orderTotal(order), paidToDate: order.amountPaid,
573  ②      balance: orderBalance(order), status: paymentStatus(order)
574      };
575      counters.receipt++;
576      listAdd(receipts, receipt);
577      listAdd(order.receipts, receipt);
578      return receipt;
579  }

```

- ① Line 563 The next receipt number.
- ② Line 573 Paid to date, balance and status, as they are now.

Time $O(n)$ · Space $O(n)$

js/notify.js · lines 20–34 · queueEmail() — Writes an email and queues it.

```
20  /* Record an email and queue it for sending. Returns the record, or null
21     while mail is off.                               Time O(n) · Space O(n) */
22  function queueEmail(to, subject, lines, kind, orderId, stamp) {
23      // The demo customers and the demo staff account live on example.com:
24      // their messages are not even queued.
25  ❶ if (!mailEnabled || !isValidEmail(to) || isDemoAddress(to)) return null;
26      var record = {
27          id: counters.mail, to: toLower(strip(to)), subject: subject, body: glue(lines, '\n'),
28          kind: kind, orderId: orderId, stamp: stamp, status: 'queued', note: ''
29      };
30      counters.mail++;
31  ❷ listAdd(outbox, record);
32  ❸ cqEnqueue(mailQueue, outbox.length - 1);
33      return record;
34  }
```

❶ line 25 Never email the demo customers (exactly example.com).

❷ line 31 Keep it in the outbox...

❸ line 32 ...and queue its position for sending.

Time O(n) · Space O(n)

js/mail.js · lines 20–65 · drainMail() — Sends the queue, one at a time.

```
20  /* Send the next queued email, then the next, one at a time.
21                                     Time O(n²) – each send redraws the outbox · Space O(1) */
22  function drainMail() {
23      if (mailSending || cqIsEmpty(mailQueue)) return;
24  ❶ var record = outbox[cqDequeue(mailQueue)];
25      if (!mailConfigured()) {
26  ❷ record.status = 'simulated';
27          record.note = 'Email is not set up yet (js/email-config.js).';
28          drainMail();
29          return;
30      }
31      mailSending = true;
32      record.status = 'sending';
33  ❸ fetch(EMAILJS_URL, {
34          method: 'POST',
35          headers: { 'Content-Type': 'application/json' },
36          body: JSON.stringify({
37              service_id: EMAIL_CONFIG.serviceId,
38              template_id: EMAIL_CONFIG.templateId,
39              user_id: EMAIL_CONFIG.publicKey,
40              template_params: {
41                  to_email: record.to, subject: record.subject, message: record.body,
42                  from_name: EMAIL_CONFIG.fromName, reply_to: EMAIL_CONFIG.replyTo
43              }
44          })
45      }).then(function (response) {
46  ❹ record.status = response.ok ? 'sent' : 'failed';
47          record.note = response.ok ? '' : 'EmailJS answered ' + response.status;
48          return response.ok ? '' : response.text();
49      }).then(function (detail) {
50          if (detail) record.note = record.note + ': ' + detail;
51      }, function () {
52          record.status = 'failed';
53          record.note = 'No connection to EmailJS.';
54      }).then(function () {
55          if (record.status === 'failed') {
56              var order = record.orderId ? orderById(record.orderId) : null;
```

```

57         logAudit(Date.now(), 'System', 'Email could not be sent: ' + record.subject, order ? order.ref
      : '',
58         record.note + ' (to ' + record.to + ')', 'warn');
59     }
60     mailSending = false;
61     if (typeof onMailChanged === 'function') onMailChanged();
62     if (typeof onSignInMailChanged === 'function') onSignInMailChanged();
63     drainMail();
64     });
65 }

```

- ❶ line 24 Take the front of the queue.
- ❷ line 26 EmailJS not set up: keep it, marked "not sent".
- ❸ line 33 Otherwise post it to EmailJS.
- ❹ line 46 Mark the result; then send the next one.

Time $O(n^2)$ – each send redraws the outbox · Space $O(1)$

PART 4

Behind the counter

Module 10 Production & Tracking · Module 11 Order Records · Module 12 Order Desk & Accounts · Module 13 Security & Audit Logs

60-second pitch

These modules are the owner's side. Production & Tracking keeps rush orders in a min-heap by due date and regular orders in a circular queue by payment order, finds any order for a customer with a binary search on the order number, and attaches the courier's tracking number or link to a delivery. Order Records keeps every completed and voided order, sorted by date with insertion sort, and undoes "Mark ready" with a stack. The Order Desk keeps every person's credentials in an array, finds an account by email through a hash table, signs them in with a salted password hash and an emailed six-digit code, and its Overview puts every order and receipt into its day, month or year by reading the record's own date. Security & Audit Logs records every sign-in attempt and who did what to each order in one append-only array, counts wrong passwords per email in a hash table, and lets a forgotten password be reset with an emailed code.

Structures and algorithms to be able to explain

Where	Structure / algorithm	Time	Space
Rush lane	Min-heap keyed (due date, arrival)	$O(\log n)$ insert / extract	$O(n)$
Standard lane	Circular queue (FIFO)	$O(1)$	$O(n)$
Track an order	Binary search on the order number	$O(\log n)$	$O(1)$
Undo "Mark ready"	Stack (LIFO)	$O(1)$ push / pop	$O(n)$
Completed orders by date	Insertion sort + one grouping pass	$O(n)$ nearly in order, $O(n^2)$ worst	$O(n)$
Find an account at sign-in	Hash table email $\rightarrow$ id, then binary search on the credentials array	$O(1)$ average + $O(\log n)$	$O(n)$
Check a password	Salted FNV-1a hash, a fixed number of rounds	$O(n)$ in the password length	$O(n)$
Overview by period	Bucketing by the record's date	$O(n^2)$ – the lines of every order	$O(n)$
Courier tracking link	Character-by-character check (https only)	$O(n)$ in the link's length	$O(n)$
Security & audit log	Append-only array, read from the back	$O(1)$ append (in the log's length) · $O(n)$ newest first	$O(n)$
Codes per address per hour	Hash table (email $\rightarrow$ count this hour)	$O(1)$ average per code	$O(n)$
Wrong passwords per email	Hash table	$O(1)$ average per attempt	$O(n)$

Likely panel questions

Why a heap for rush orders and not a sorted array?

Inserting into a sorted array shifts items: $O(n)$. A heap inserts, and removes the earliest due date, in $O(\log n)$ by sifting along one path of the tree.

How is a tie on the due date broken?

The key is (due date, arrival number). `heapCompare` compares dates first, then arrival, so equal dates are served in the order they were paid — the heap is stable.

Why is the orders array always sorted?

Order numbers come from a counter that only goes up, and new orders are appended, so the array stays in number order without sorting — which is what makes binary search possible.

What is the worst case of your binary search?

About $\log_2 n$ comparisons: 5 for the 22 demo orders, 10 for 1,000 orders. A linear search could need all n .

How does undo put an order back exactly?

Each "Mark ready" pushes { order id, its place in line }. Undo pops the newest entry and puts the order back: into the heap with its original key, or into the queue at its old position.

Is the two-step sign-in real security?

It keeps customers off the desk: each password is kept only as a salted hash, the code only as a hash, codes expire after five minutes and allow three tries, and five wrong passwords in a row pause that email for 30 seconds. Real security needs a server — anything in a browser can be read with developer tools.

Why keep the credentials in an array and a hash table?

The array is the list of accounts in id order. Ids only grow, so a new account is appended and the array stays sorted — an account is found by id with binary search, $O(\log n)$. Signing in starts from an email, so a hash table email $\rightarrow$ id finds the account in $O(1)$ on average instead of a linear search through every account.

Why a salt, and why 200 rounds?

A salt is random text stored with each account and hashed together with its password, so two people with the same password get different hashes and a ready-made list of hashed passwords is useless. Repeating the hash 200 times makes every guess 200 times slower. A real system uses `bcrypt` or `Argon2` on a server.

Why is the log an append-only array?

Events only ever happen in time order and are never changed, so adding each one at the end keeps the array sorted by time for free: appending is $O(1)$, "newest first" is a read from the back with no sort, and the 24-hour summary can stop at the first entry older than a day.

Why does "Forgot password?" answer the same for any email?

So the page never tells a stranger which emails have desk accounts (account enumeration). A code is only made for the email of an active account; for any other the screen, the waits and the tries behave the same, but nothing is sent.

Could a tracking link run code on the customer's page?

No. `trackingLinkProblem` accepts only a plain `https://` address on a named site — no `javascript:` or other schemes, no spaces, quotes or angle brackets, and no "user@" part. For Flash Express and Lalamove the site must be the courier's own (`flashexpress.ph`, `lalamove.com`); for another courier it may not be an IP address or a punycode look-alike. The link is escaped when shown and opens in a new tab with `rel="noopener noreferrer"`. Tracking is only added once a courier delivery is marked ready.

Couldn't someone flood the owner's inbox with codes?

Each code path has its own limits (30 seconds between codes, three per sign-in or reset), and on top of that a hash table counts every code an address is sent in the hour — sign-in and reset together. Starting again does not reset it, so one address gets at most six codes an hour. A reload clears it like all data here, so the shop also sets a rate limit in EmailJS.

What happens to a desk left open?

After 15 minutes without a tap or a key it signs itself out, and the sign-in form is emptied. Five wrong current passwords on the Account tab also sign the person out and pause the email, so someone at an unattended desk cannot guess the password there.

Production & Tracking

Screens	Order desk → "In production" tab, "Mark ready" and "Add tracking"; Track order on the shop
Files	js/structures.js (heapInsert, heapSiftUp, heapExtractMin, heapCompare) · js/production.js (enterProduction, productionLine, completeNext) · js/orders.js (orderById, customerLookup) · js/algorithms.js (binarySearch) · js/tracking.js (setCourierTracking, trackingLinkProblem)
Data	rushLane — min-heap keyed (due date, arrival); standardLane — circular queue; orders — array sorted by order number; order.courierTracking — { courier, number, link }
DSA	Min-heap · Circular queue · Binary search

1 Business process

What task or problem from the client are you solving?

Paid orders must be made in the right order. Rush orders (P50 extra, ready next day) go first, earliest due date first. Regular orders are made in the order they were paid. An order still owing a balance cannot be released, but it must not hold up the paid orders behind it.

Customers check their order with the tracking number (e.g. SU-215) and their mobile number or email, so the shop needs to find an order by number quickly.

Once a courier (Flash Express, Lalamove) has a parcel, the customer should be able to follow it without messaging the shop: the courier's parcel number or tracking link is attached to the order.

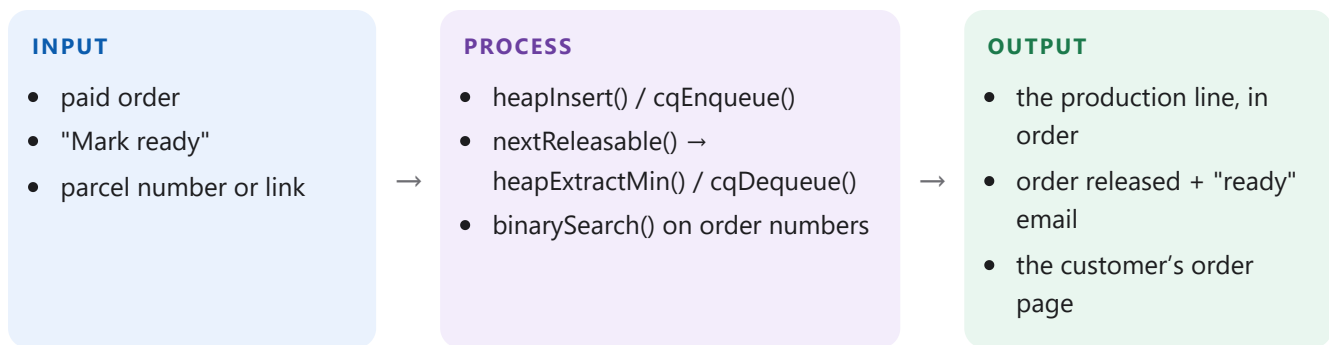
2 Input

What data is needed by the module?

Input	What it is
Stored: rushLane (heap), standardLane (queue)	Order ids in production.
Stored: orders	In ascending order-number order.
Customer types	Tracking number and mobile number or email.
Owner presses	"Mark ready".
Owner types (tracking)	The courier, its parcel number and / or tracking link, for a courier delivery marked ready.

3 Process

What data structure and/or algorithm did you use? How does it work step by step?



Step by step (Min-heap · Circular queue · Binary search):

- 1 Paid rush order: insert it into a min-heap keyed by (due date, arrival number) — it sifts up past later dates. Paid standard order: enqueue it.
- 2 Mark ready: walk the line (heap order, then queue order) to the first order that is fully paid; take it out (the heap minimum or the queue front in the usual case) and email the customer. Listing the line in heap order sorts a copy of the heap's array by the same key with insertion sort; the heap itself is untouched.
- 3 Tracking: orders are numbered in increasing order and only ever appended, so the array is already sorted — binary search halves the range at each step.
- 4 Courier tracking: once a courier delivery is marked ready (the customer was told it is on its way), the desk adds the courier's parcel number and / or link. The number may hold only letters, digits and hyphens; the link is checked character by character — `https://` only, the courier's own site (or, for another courier, no IP address or punycode), no spaces, quotes or "user@" — so it can never run code or point somewhere else. The customer sees it on Track order and gets it by email; if it is removed, they are emailed to ignore it.

4 Output

What result does the module produce?

- The "In production" tab with positions, rush badges and the "Mark ready" button.
- The customer's tracking page with the timeline.
- The "Follow your parcel" card on Track order (the courier, the number, a link to the courier), the tracking email, and the tracking line on the desk's order card.

5 Complexity analysis

What is the time complexity? What is the space complexity?

Operation	Time	Space	Note
<code>heapInsert()</code>	$O(\log n)$	$O(1)$	Adds an order to the rush lane.
<code>heapSiftUp()</code>	$O(\log n)$	$O(1)$	Restores the heap upward.
<code>heapExtractMin()</code>	$O(\log n)$	$O(1)$	Removes the earliest-due order.
<code>heapCompare()</code>	$O(1)$	$O(1)$	Which order goes first.
<code>completeNext()</code>	$O(n^2)$	$O(n)$	"Mark ready".
<code>binarySearch()</code>	$O(\log n)$	$O(1)$	Finds an order by number.
<code>customerLookup()</code>	$O(\log n) + O(n)$ for the text	$O(n)$	Shows an order only to its customer.

Operation	Time	Space	Note
<code>trackingLinkProblem()</code>	$O(n)$	$O(n)$	Is a tracking link safe to show the customer?
<code>setCourierTracking()</code>	$O(n)$	$O(n)$	Attaches the courier's tracking to an order.

What n means here: For the heap and the queue, n = orders in production. For binary search, n = all orders.

Each Time and Space is read from the comment above the function in the source, where every function in `js/` states its own cost.

6 Justification

Why did you choose that data structure or algorithm? Why is it appropriate?

A min-heap always gives the earliest due date, in $O(\log n)$ per insert or removal — keeping a sorted list instead would cost $O(n)$ per insert. Adding the arrival number to the key makes it stable: equal dates are served in the order they were paid.

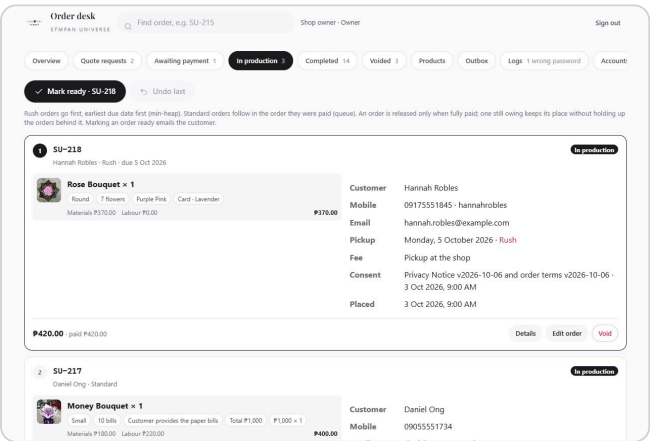
Standard orders are fair first-in, first-out: a circular queue, $O(1)$ at both ends.

Binary search finds an order among n in $O(\log n)$ — about 10 steps for 1,000 orders — and costs nothing extra because order numbers are handed out in increasing order.

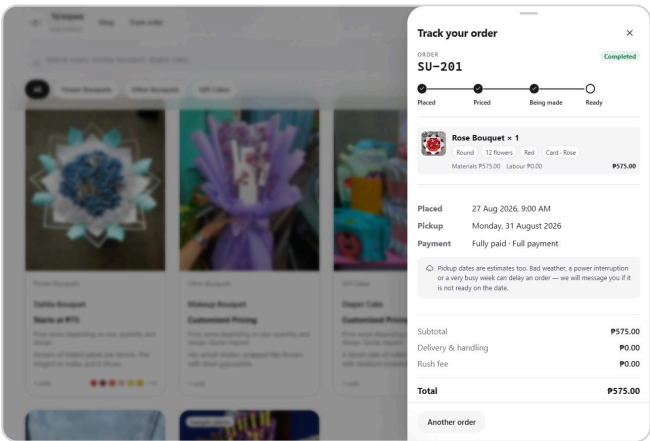
7 Demonstration

Show that the module works correctly — the input, the process and the output.

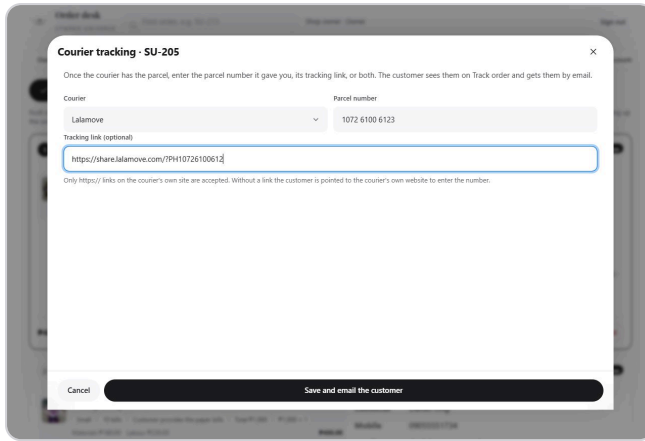
- 1 In the order desk, open "In production": rush orders first, then standard.
- 2 Press "Mark ready": the first fully paid order is released and the customer emailed; an order owing a balance keeps its place.
- 3 On the shop, press "Track order" and enter SU-201 with 0917 555 0142.
- 4 In the desk, find SU-205 (a Lalamove delivery) and press "Add tracking": type Lalamove's order number and paste its share link. Track order for SU-205 now shows "Follow your parcel".



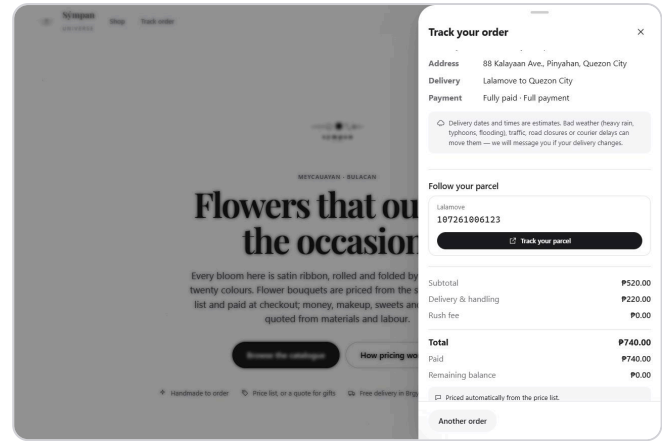
The production line



Tracking an order



The desk adds the courier's parcel number and link



The customer follows the parcel

Trace — the rush heap and the order search (computed by our code)

The rush lane after the demo history

[0]
SU-218 Hannah
key (20261003, 1)

heapInsert() for SU-223, due 2026-10-10 (key 20261010)

[0]
SU-218 Hannah
key (20261003, 1)

[1]
SU-223 Lara
key (20261010, 2)

heapInsert() for SU-224, due 2026-10-06 (key 20261006)

[0]
SU-218 Hannah
key (20261003, 1)

[1]
SU-223 Lara
key (20261010, 2)

[2]
SU-224 Lara
key (20261006, 3)

heapInsert() for SU-225, due 2026-10-08 (key 20261008)

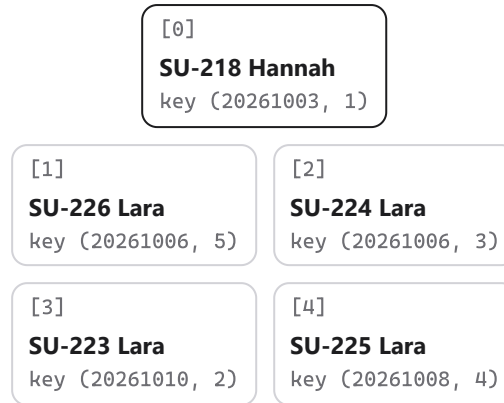
[0]
SU-218 Hannah
key (20261003, 1)

[1]
SU-225 Lara
key (20261008, 4)

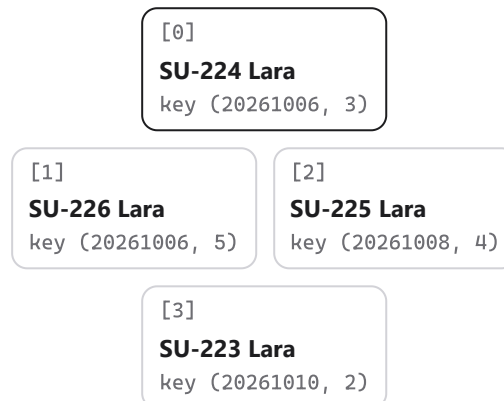
[2]
SU-224 Lara
key (20261006, 3)

[3]
SU-223 Lara
key (20261010, 2)

heapInsert() for SU-226, due 2026-10-06 (key 20261006)



completeNext() — "Mark ready" takes SU-218, the earliest due date; the last leaf sinks from the root



Equal dates are served in payment order, because the key's second part is the arrival number.

binarySearch(orders, 215) — the order numbers looked at

213 → 220 → 216 → 214 → 215 ⇒ index 14 (SU-215) in 5 steps among 26 orders; a linear scan would need 15.

Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

js/structures.js · lines 272–281 · **heapInsert()** — Adds an order to the rush lane.

```
272  /* Insert with a priority. `seq` may be given to restore an item at its
273     exact former key.                               Time O(log n) · Space O(1) */
274  function heapInsert(heap, value, priority, seq) {
275      var arrival = seq === undefined ? heap.nextSeq : seq;
276      if (arrival >= heap.nextSeq) heap.nextSeq = arrival + 1;
277  ❶  heap.items[heap.size] = { value: value, priority: priority === undefined ? 1 : priority, seq: arrival
    };
278      heap.size++;
279  ❷  heapSiftUp(heap, heap.size - 1);
280      return heap.size;
281  }
```

❶ line 277 Put it at the end of the array (the next leaf).

❷ line 279 Sift it up while it beats its parent.

Time O(log n) · Space O(1)

js/structures.js · lines 248–257 · heapSiftUp() — Restores the heap upward.

```
248  /* Bubble a node up while it beats its parent. Time O(log n) · Space O(1) */
249  function heapSiftUp(heap, index) {
250      var child = index;
251      while (child > 0) {
252  ❶      var parent = Math.floor((child - 1) / 2);
253  ❷      if (heapCompare(heap.items[child], heap.items[parent]) >= 0) break;
254  ❸      heapSwap(heap, child, parent);
255      child = parent;
256      }
257  }
```

❶ Line 252 The parent of index i is $(i - 1) \div 2$.

❷ Line 253 Stop once the parent is not later.

❸ Line 254 Otherwise swap and continue upward.

Time $O(\log n)$ · Space $O(1)$

js/structures.js · lines 283–292 · heapExtractMin() — Removes the earliest-due order.

```
283  /* Remove and return the smallest key's value. Time O(log n) · Space O(1) */
284  function heapExtractMin(heap) {
285      if (heap.size === 0) return null;
286  ❶      var top = heap.items[0];
287      heap.size--;
288  ❷      heap.items[0] = heap.items[heap.size];
289      heap.items.length = heap.size;
290  ❸      if (heap.size > 0) heapSiftDown(heap, 0);
291      return top.value;
292  }
```

❶ Line 286 The minimum is always at the root.

❷ Line 288 Move the last leaf to the root...

❸ Line 290 ...and sink it to its place.

Time $O(\log n)$ · Space $O(1)$

js/structures.js · lines 235–239 · heapCompare() — Which order goes first.

```
235  /* Negative when a must leave before b. Time O(1) · Space O(1) */
236  function heapCompare(a, b) {
237  ❶      if (a.priority !== b.priority) return a.priority - b.priority;
238  ❷      return a.seq - b.seq;
239  }
```

❶ Line 237 Earlier due date first...

❷ Line 238 ...then whoever paid first.

Time $O(1)$ · Space $O(1)$

js/production.js · lines 64–94 · completeNext() — "Mark ready".

```
64  /* Release the next fully paid order. Records where it stood, so undo can
65     put it back exactly. Finding it walks the line,  $O(n^2)$ ; taking it
66     from the front is  $O(1)$  for the queue,  $O(\log n)$  for the heap.
67                                     Time  $O(n^2)$  · Space  $O(n)$  */
68  function completeNext(stamp) {
69      var line = productionLine();
70      if (line.length === 0) return { ok: false, error: 'Nothing is in production.' };
71  ❶      var order = nextReleasable();
```

```

72     if (!order) {
73         var owing = renderEach(line, function (o) { return o.ref; }, ', ');
74         return { ok: false, error: 'Every order in production still owes a balance (' + owing + '). Record
a balance payment first.' };
75     }
76     var skipped = [];
77     for (var i = 0; i < line.length && line[i] !== order; i++) listAdd(skipped, line[i]);
78     var position = -1;
79     if (order.lane === 'rush') {
80         // Rush comes first, so a releasable rush order is the heap's minimum
81         // unless one ahead of it still owes.
82 ❷ if (heapPeek(rushLane) === order.id) heapExtractMin(rushLane);
83         else heapRemove(rushLane, order.id);
84     } else {
85         position = positionIn(cqValues(standardLane), order.id);
86 ❸ leaveQueue(standardLane, order.id);
87     }
88     order.status = 'completed';
89     order.completedAt = stamp;
90 ❹ stackPush(completedStack, { id: order.id, position: position });
91     addHistory(order, stamp, 'Ready and released', deskActor());
92 ❺ emailOrderReady(order, stamp);
93     return { ok: true, order: order, skipped: skipped };
94 }

```

- ❶ line 71 The first fully paid order in line.
- ❷ line 82 Rush: usually the heap minimum.
- ❸ line 86 Standard: usually the queue front.
- ❹ line 90 Remember it for undo (module 11).
- ❺ line 92 Email "ready for pickup / on its way".

Time $O(n^2)$ · Space $O(n)$

js/algorithms.js · lines 84–98 · binarySearch() — Finds an order by number.

```

84  /* Binary search over an array sorted ascending by keyOf. Order numbers are
85     handed out in increasing sequence and orders are only ever appended, so
86     the order list is sorted for free. Returns the index or -1.
87                                     Time  $O(\log n)$  · Space  $O(1)$  */
88  function binarySearch(sorted, target, keyOf) {
89     var low = 0, high = sorted.length - 1;
90     while (low <= high) {
91 ❶     var mid = Math.floor((low + high) / 2);
92         var key = keyOf(sorted[mid]);
93 ❷     if (key === target) return mid;
94 ❸     if (key < target) low = mid + 1;
95 ❹     else high = mid - 1;
96     }
97     return -1;
98 }

```

- ❶ line 91 Look at the middle.
- ❷ line 93 Found.
- ❸ line 94 Too small: search the right half...
- ❹ line 95 ...too big: search the left half.

Time $O(\log n)$ · Space $O(1)$

js/orders.js · lines 71–79 · customerLookup() — Shows an order only to its customer.

```

71  /* A customer sees an order only with its tracking number and the mobile

```

```

72     number or email used on it.           Time O(log n) + O(n) for the text · Space O(n) */
73     function customerLookup(ref, contact) {
74 ❶     var order = orderByRef(ref);
75         if (!order || isBlank(contact)) return null;
76         var byEmail = order.customer.email && toLower(strip(contact)) === order.customer.email;
77         var byPhone = digitsOnly(contact).length > 0 && normalisePhone(order.customer.phone) ===
            normalisePhone(contact);
78 ❷     return byEmail || byPhone ? order : null;
79     }

```

❶ Line 74 Binary search by number.

❷ Line 78 Only if the mobile number or email matches.

Time O(log n) + O(n) for the text · Space O(n)

js/tracking.js · lines 77–114 · trackingLinkProblem() — Is a tracking link safe to show the customer?

```

77     /* ' when a tracking link is safe to show the customer, or what is wrong.
78     It must be a plain https:// address on a named host: no spaces, quotes or
79     angle brackets, and no "user@" part that could disguise the real site.
80     For Flash Express and Lalamove the host must be the courier's own site;
81     for another courier it may not be an IP address or punycode.
82                                     Time O(n) · Space O(n) */
83     function trackingLinkProblem(link, courierId) {
84         var prefix = 'https://';
85         if (link.length > TRACKING_LINK_MAX) return 'Keep the tracking link under ' + TRACKING_LINK_MAX + '
            characters.';
86 ❶     if (!beginsWith(link, prefix)) return 'The tracking link must start with https://';
87         for (var i = 0; i < link.length; i++) {
88             var c = link.charAt(i);
89 ❷     if (isSpace(c) || c === '"' || c === '\'' || c === '<' || c === '>' || c === '\\' || c === '\') {
90                 return 'The tracking link cannot contain spaces, quotes or angle brackets.';
91             }
92         }
93         var host = '', dots = 0;
94         for (var k = prefix.length; k < link.length; k++) {
95             var h = link.charAt(k);
96             if (h === '/' || h === '?' || h === '#') break;
97 ❸     if (!isLetter(h) && !isDigit(h) && h !== '-' && h !== '.') return 'Use the courier\'s own tracking
            address (only its site name before the first /).';
98             if (h === '.') dots++;
99             host += h;
100        }
101        host = toLower(host);
102 ❹     if (host.length < 4 || dots === 0 || host.charAt(0) === '.' || host.charAt(host.length - 1) === '.') {
103            return 'That does not look like a web address.';
104        }
105        var courier = courierId === OTHER_COURIER.id ? null : courierById(courierId);
106        if (courier && courier.hosts) {
107            if (!firstWhere(courier.hosts, function (d) { return hostUnder(host, d); })) {
108                return 'Use a link on ' + courier.name + '\s own site (' + glue(courier.hosts, ' or ') + ').';
109            }
110        } else if (isDisguisedHost(host)) {
111            return 'Use the courier\'s own web address, not a number or a disguised name.';
112        }
113        return '';
114    }

```

❶ Line 86 Only https:// — never javascript: or another scheme.

❷ Line 89 One pass refuses spaces, quotes and angle brackets.

❸ Line 97 The site name may hold only letters, digits, hyphens and dots — so no "user@" part can disguise it.

❹ Line 102 It must look like a real site name.

Time $O(n)$ · Space $O(n)$

js/tracking.js · lines 148–167 · `setCourierTracking()` — Attaches the courier's tracking to an order.

```
148  /* Attach the courier's tracking to an order, or change it. The customer is
149     emailed.                                Time  $O(n)$  · Space  $O(n)$ 
150     Returns { ok, order } or { ok: false, errors }. */
151  function setCourierTracking(id, draft, stamp) {
152      var order = orderById(id);
153      ❶ if (!canAttachTracking(order)) {
154          return { ok: false, errors: [{ field: 'order', message: 'Courier tracking can be added once a
155             courier delivery is marked ready.' }] };
156      ❷ var checked = checkTracking(draft);
157          if (checked.errors.length > 0) return { ok: false, errors: checked.errors };
158          var t = checked.tracking, before = order.courierTracking;
159          if (before && before.courier === t.courier && before.number === t.number && before.link === t.link) {
160              return { ok: false, errors: [{ field: 'order', message: 'Nothing was changed.' }] };
161          }
162      ❸ order.courierTracking = { courier: t.courier, number: t.number, link: t.link, stamp: stamp };
163      ❹ addHistory(order, stamp, (before ? 'Courier tracking changed: ' : 'Courier tracking added: ') +
164          trackingSummary(order.courierTracking),
165          deskActor(), t.link);
165      ❺ emailTrackingAdded(order, stamp);
166          return { ok: true, order: order };
167      }
```

- ❶ line 153 Only courier deliveries marked ready.
- ❷ line 156 Check the number and the link.
- ❸ line 162 Attach it to the order...
- ❹ line 163 ...write it to the history and the audit log...
- ❺ line 165 ...and email the customer.

Time $O(n)$ · Space $O(n)$

Order Records

Screens	Order desk → "Completed" and "Voided" tabs; "Undo last" in production
Files	js/structures.js (stackPush, stackPop) · js/production.js (completeNext, undoCompletion, completedByDate, voidOrder, expireQuotes)
Data	completedStack — stack of { id, position }; orders (completion and void times, reasons)
DSA	Stack · Insertion sort

1 Business process

What task or problem from the client are you solving?

Every finished order must be kept with all its details, organised by date. Cancelled orders are kept too, marked "VOIDED – NON-REFUNDABLE", never deleted. Quotations the customer never pays are cancelled automatically after three days.

People make mistakes: if the owner marks the wrong order ready, they need to undo it — and the order must go back exactly where it was.

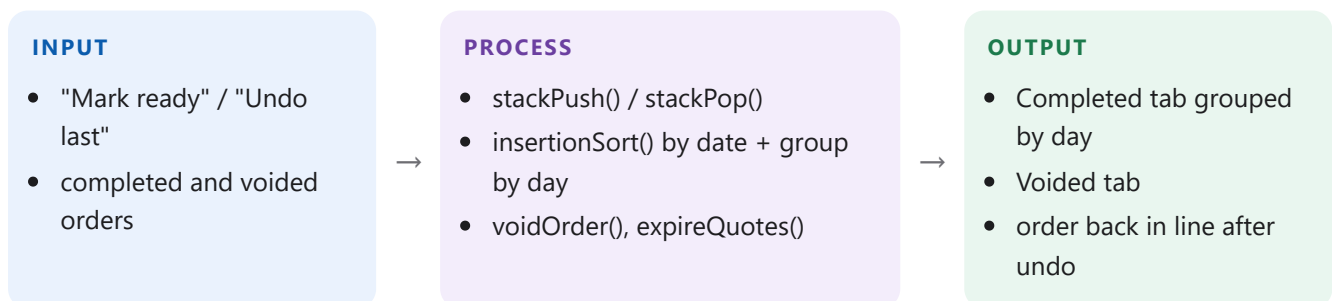
2 Input

What data is needed by the module?

Input	What it is
Stored: orders	Completion and void times, reasons, payments.
Stored: completedStack	The completions made this session, newest on top.
Owner presses	"Undo last"; "Void" with a reason.
The clock	Every minute, for expiring quotations.

3 Process

What data structure and/or algorithm did you use? How does it work step by step?



Step by step (Stack · Insertion sort):

- 1 Each "Mark ready" pushes { order, its place in line } onto a stack.
- 2 Undo pops the top — the most recent completion — and puts the order back: into the heap with its original key, or into the queue at its old position.

- 3 Completed orders: read the list back to front (it is kept oldest first), insertion sort it by completion time, newest first, then group consecutive orders of the same day with a day total.
- 4 Voiding takes the order out of its queue or lane, keeps the record and any payment, and emails the customer. Every minute, quotations older than three days are voided by "System".

4 Output

What result does the module produce?

- The "Completed" tab (days with totals; each order opens in full).
- The "Voided" tab with reason, who and when.
- The cancellation email.

5 Complexity analysis

What is the time complexity? What is the space complexity?

Operation	Time	Space	Note
<code>stackPush()</code>	$O(1)$	$O(1)$	Pushes onto the stack.
<code>stackPop()</code>	$O(1)$	$O(1)$	Pops the top.
<code>undoCompletion()</code>	$O(n)$ worst	$O(n)$	Undo the last "Mark ready".
<code>completedByDate()</code>	$O(n)$ here (nearly in order), $O(n^2)$ worst	$O(n)$	Completed orders by day.
<code>voidOrder()</code>	$O(n)$	$O(n)$	Cancels, but keeps the record.
<code>expireQuotes()</code>	$O(n^2)$	$O(n)$	The automatic clean-up.

What n means here: n = number of orders (completed or voided); for the stack, n = completions this session.

Each Time and Space is read from the comment above the function in the source, where every function in js/ states its own cost.

6 Justification

Why did you choose that data structure or algorithm? Why is it appropriate?

Undo is last in, first out by nature — you undo the most recent action first — which is exactly a stack, $O(1)$ push and pop.

Storing the position with each completion means undo restores the order exactly, not just "somewhere in production".

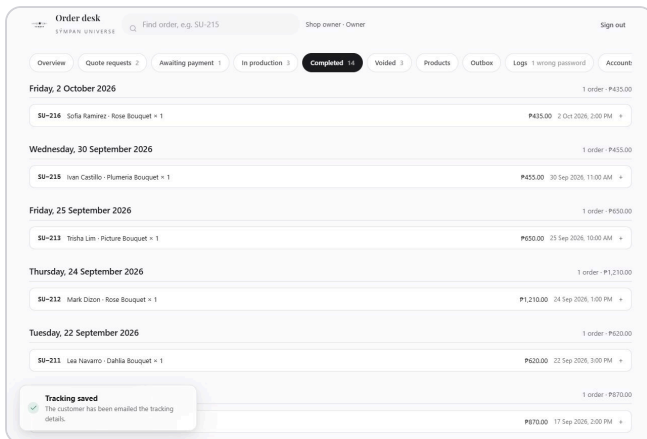
Insertion sort gives a stable date order. Orders are completed mostly in the order they were placed, so reading the oldest-first array back to front leaves it nearly newest first already: the sort runs close to its $O(n)$ best case (15 comparisons for 15 orders, instead of the worst case of 105). Grouping consecutive same-day orders is then a single $O(n)$ pass.

7 Demonstration

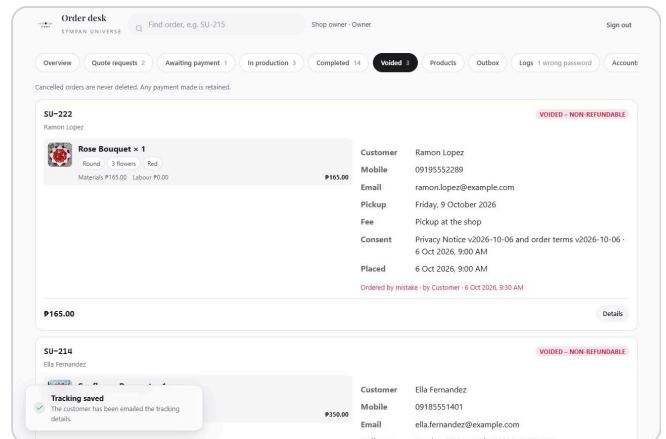
Show that the module works correctly — the input, the process and the output.

- 1 In "In production" press "Mark ready", then "Undo last": the order is back at its place.
- 2 Open "Completed": orders grouped by day, newest first, each with a day total.

3 Void an order awaiting payment: it moves to "Voided" as VOIDED – NON-REFUNDABLE.



Completed orders grouped by day



Voided orders kept on record

Trace — undo and the records by date (computed by our code)

Two "Mark ready": SU-218 then SU-217 are pushed

1

SU-217 (place 0)

top

0

SU-218 (place -1)

undoCompletion() pops SU-217 (last in, first out) and puts it back in its lane

0

SU-218 (place -1)

top

SU-217 is "In production" again.

completedByDate() — insertion sort newest first, then grouped by day (first six days)

Day	Orders	Day total
Monday, 5 October 2026	SU-218	₱420.00
Thursday, 1 October 2026	SU-216	₱435.00
Tuesday, 29 September 2026	SU-215	₱455.00
Thursday, 24 September 2026	SU-213	₱650.00
Wednesday, 23 September 2026	SU-212	₱1,210.00
Monday, 21 September 2026	SU-211	₱620.00

insertionSort made 15 comparisons for the 15 completed orders. They are read back to front first, so the list is already nearly newest first — close to the best case, $n - 1 = 14$. Sorting the oldest-first array directly would be the worst case, $n(n - 1)/2 = 105$.

voidedOrders() — kept on record as VOIDED – NON-REFUNDABLE

Order	Reason	By	Retained
SU-222	Ordered by mistake	Customer	₱0.00
SU-214	Customer cancelled after production started	Order desk	₱235.00
SU-210	Found another gift	Customer	₱0.00

expireQuotes(four days later)

Voided automatically: SU-219 — "Quotation expired — not paid within 3 days" by System.

Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

js/structures.js · lines 341–346 · stackPush() — Pushes onto the stack.

```
341  /*                               Time O(1) · Space O(1) */
342  function stackPush(stack, value) {
343  ❶    stack.top++;
344  ❷    stack.items[stack.top] = value;
345      return stack.top + 1;
346  }
```

❶ line 343 Move the top up one...

❷ line 344 ...and store the value there.

Time O(1) · Space O(1)

js/structures.js · lines 348–355 · stackPop() — Pops the top.

```
348  /* Remove and return the top, or null.   Time O(1) · Space O(1) */
349  function stackPop(stack) {
350  ❶    if (stack.top === NIL) return null;
351  ❷    var value = stack.items[stack.top];
352      stack.items.length = stack.top;
353  ❸    stack.top--;
354      return value;
355  }
```

❶ line 350 Empty: nothing to undo.

❷ line 351 Take the top value...

❸ line 353 ...and move the top down.

Time O(1) · Space O(1)

js/production.js · lines 96–116 · undoCompletion() — Undo the last "Mark ready".

```
96  /* Put the last completion back exactly where it came from: the front of
97     its lane in O(1) / O(log n), or its old place in O(n) if it had been
98     released from behind an order still owing.   Time O(n) worst · Space O(n) */
99  function undoCompletion(stamp) {
100  ❶    var entry = stackPop(completedStack);
101      if (entry === null) return { ok: false, error: 'There is nothing to undo.' };
102      var order = orderById(entry.id);
103      order.status = 'paid';
104      order.completedAt = 0;
```

```

105     if (order.lane === 'rush') {
106         // An order moved to the rush lane after completing has no arrival
107         // number yet; it takes the next one.
108         if (order.laneSeq < 0) order.laneSeq = rushLane.nextSeq;
109 ❷     heapInsert(rushLane, order.id, dueKey(order), order.laneSeq);
110     } else {
111 ❸     if (entry.position <= 0) cqRequeueFront(standardLane, order.id);
112 ❹     else cqInsertAt(standardLane, entry.position, order.id);
113     }
114     addHistory(order, stamp, 'Completion undone – back in production', deskActor());
115     return { ok: true, order: order };
116 }

```

- ❶ Line 100 The most recent completion.
- ❷ Line 109 Rush: back in the heap with its original key.
- ❸ Line 111 Standard: back at the front...
- ❹ Line 112 ...or at its old place if it was released from behind an order still owing.

Time $O(n)$ worst · Space $O(n)$

js/production.js · lines 118–137 · completedByDate() — Completed orders by day.

```

118  /* Completed orders grouped by the day they were completed, newest first.
119     Orders are kept oldest first and are mostly completed in that order, so
120     the list is read back to front before the insertion sort: it is then
121     nearly in order already, the sort's best case.
122                                     Time  $O(n)$  here (nearly in order),  $O(n^2)$  worst · Space  $O(n)$  */
123  function completedByDate() {
124  ❶     var sorted = insertionSort(backwards(ordersWithStatus('completed')), function (a, b) { return
        b.completedAt - a.completedAt || a.id - b.id; });
125     var groups = [];
126     for (var i = 0; i < sorted.length; i++) {
127         var day = isoFromStamp(sorted[i].completedAt);
128         var last = groups.length > 0 ? groups[groups.length - 1] : null;
129  ❷     if (!last || last.date !== day) {
130         last = { date: day, orders: [], total: 0 };
131         listAdd(groups, last);
132     }
133     listAdd(last.orders, sorted[i]);
134  ❸     last.total = roundMoney(last.total + orderTotal(sorted[i]));
135     }
136     return groups;
137 }

```

- ❶ Line 124 Read back to front, then insertion sort newest first — nearly in order already.
- ❷ Line 129 A new day starts a new group...
- ❸ Line 134 ...and the day total grows.

Time $O(n)$ here (nearly in order), $O(n^2)$ worst · Space $O(n)$

js/production.js · lines 143–161 · voidOrder() — Cancels, but keeps the record.

```

143  /*                                     Time  $O(n)$  · Space  $O(n)$  */
144  function voidOrder(id, reason, by, stamp) {
145     var order = orderById(id);
146     if (!order) return { ok: false, error: 'No such order.' };
147     if (order.status === 'completed') return { ok: false, error: order.ref + ' is completed and cannot be
        voided.' };
148     if (order.status === 'voided') return { ok: false, error: order.ref + ' is already voided.' };
149
150  ❶     if (order.status === 'requested') leaveQueue(queueQueue, order.id);
151  ❷     if (order.status === 'paid') leaveProduction(order);

```

```

152 ❸ order.status = 'voided';
153    order.voidedAt = stamp;
154    order.voidedBy = by === 'admin' ? deskActor() : (by === 'system' ? 'System' : 'Customer');
155    order.voidReason = strip(reason) || (by === 'admin' ? 'Cancelled by the order desk' : 'Cancelled by the
customer');
156    addHistory(order, stamp, 'VOIDED – NON-REFUNDABLE' +
157      (order.amountPaid > 0 ? ' (' + peso(order.amountPaid) + ' retained)' : ''),
158      by === 'admin' ? deskActor() : (by === 'system' ? 'System' : 'Customer'), order.voidReason,
      'warn');
159 ❹ emailOrderVoided(order, stamp);
160    return { ok: true, order: order };
161  }

```

❶ line 150 Leave the quote queue...

❷ line 151 ...or the production line.

❸ line 152 Marked VOIDED – NON-REFUNDABLE; payments are retained.

❹ line 159 Tell the customer.

Time $O(n)$ · Space $O(n)$

js/production.js · lines 163–174 · `expireQuotes()` — The automatic clean-up.

```

163  /* Void every quotation left unpaid for QUOTE_EXPIRY_DAYS, so the owner
164     never has to chase one. Run on start-up and every minute after.
165                                     Time  $O(n^2)$  · Space  $O(n)$  */
166  function expireQuotes(now) {
167    var limit = QUOTE_EXPIRY_DAYS * 24 * 3600000, expired = [];
168  ❶ var stale = keepWhere(orders, function (o) { return o.status === 'quoted' && now - o.quotedAt > limit;
    });
169    for (var i = 0; i < stale.length; i++) {
170  ❷ voidOrder(stale[i].id, 'Quotation expired – not paid within ' + QUOTE_EXPIRY_DAYS + ' days',
      'system', now);
171      listAdd(expired, stale[i]);
172    }
173    return expired;
174  }

```

❶ line 168 Quotations unpaid for more than three days...

❷ line 170 ...are voided by "System".

Time $O(n^2)$ · Space $O(n)$

Order Desk & Accounts

Screens	index.html#admin — sign-in, the six-digit code, the Overview (daily, monthly, yearly), Accounts
Files	js/accounts.js (staffAccounts, accountByEmail, passwordHash, checkPassword, addStaffAccount) · js/admin-signin.js (deskSignIn, issueOtp, verifyOtp) · js/reports.js (bucketOf, tallyPeriods, periodReport) · js/admin-overview.js · js/admin-account.js
Data	staffAccounts — array of { id, name, email, role, salt, passHash, active } in id order; staffEmailIndex — hash table email → id; failedSignIns (pauses); deskState (the code's hash); orders and receipts
DSA	Credentials array · Hash table · Bucketing by date

1 Business process

What task or problem from the client are you solving?

The shop is run from the order desk, at index.html#admin — linked from nowhere on the shop. Each person has their own account: the owner, and staff the owner adds. Signing in takes an email and password, then a six-digit code sent to that account's own email (two-step verification). Everything they do is logged under their name.

An owner sees everything, adds and disables accounts, and reads the logs; staff handle orders, production and products. The Overview answers "how is the shop doing?" for any day, month or year: money collected, sales, orders placed, pieces sold, orders made ready, voids, and that period's best sellers.

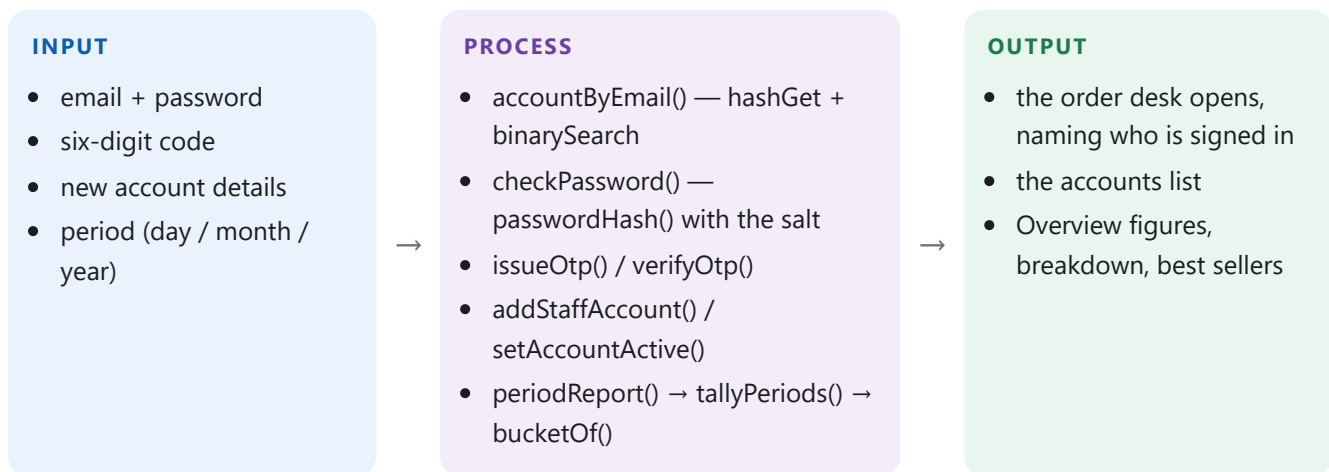
2 Input

What data is needed by the module?

Input	What it is
Person types	Email, password, the emailed code; a day, month or year.
Owner types (Accounts)	A new person's name, email, role and temporary password; disable / enable.
Stored: staffAccounts, staffEmailIndex	Each account's name, email, role, salt and salted password hash; the email → id index.
Stored: orders, receipts	Everything the Overview counts.

3 Process

What data structure and/or algorithm did you use? How does it work step by step?



Step by step (Credentials array · Hash table · Bucketing by date):

- 1 Find the account: the typed email is trimmed and lower-cased, hashed to its bucket in staffEmailIndex ($O(1)$ on average) to get the account id, and binary search finds that id in staffAccounts, which is always in id order ($O(\log n)$).
- 2 Check the password: hash the typed password with that account's own salt — FNV-1a, repeated 200 rounds — and compare with the stored hash. A wrong password, an unknown email and a disabled account all get the same message; five in a row for one email pause it for 30 seconds.
- 3 Two-step: make a random six-digit code, keep only its hash, email it to the account's address; it lasts five minutes, allows three tries, and a new one can be sent after 30 seconds (three per sign-in, and at most six codes an hour to one address however often the sign-in is started again). The right code makes that account the person signed in; 15 minutes without a tap or a key signs them out.
- 4 Accounts: an owner adds a person — the next id, so appending keeps the array sorted — and indexes their email; or disables one (never deletes, so the history stays). Nobody can disable themselves, and the last active owner stays.
- 5 Overview: a period is a key "2026-10-05", "2026-10" or "2026", and a record falls in it when its date begins with the key. For the breakdown, bucketOf() reads the day (month view) or the month (year view) straight off the record's date and uses it as the row number, so one pass over the orders and receipts fills every row.

4 Output

What result does the module produce?

- The sign-in and code screens; the header names who is signed in.
- The Overview: figure cards, day-by-day or month-by-month table, best sellers.
- Accounts (owners) or My account (staff): change your own password; owners also see, add and disable accounts.
- The tabs: quote requests, payments, production, completed, voided, products, outbox — and Logs for owners.

5 Complexity analysis

What is the time complexity? What is the space complexity?

Operation	Time	Space	Note
passwordHash()	$O(n)$ — PASSWORD_ROUNDS is fixed	$O(n)$	The stored form of a password.

Operation	Time	Space	Note
accountByEmail()	$O(1)$ average + $O(\log n)$	$O(n)$	Finds an account from the email typed.
accountById()	$O(\log n)$	$O(1)$	Binary search on the credentials array.
deskSignIn()	$O(n)$	$O(n)$	Step 1: email and password.
issueOtp()	$O(n)$ for the email	$O(1)$	Makes a code.
verifyOtp()	$O(n)$	$O(1)$	Step 2: the code.
addStaffAccount()	$O(n)$	$O(n)$	An owner adds a person.
bucketOf()	$O(1)$	$O(1)$	Which row a moment belongs to.
tallyPeriods()	$O(n^2)$ – the lines of every order	$O(1)$	One pass fills every row.
periodReport()	$O(n^2)$	$O(n)$	The whole Overview for a period.

What n means here: For hashing, n = characters of the password or code — 200 rounds is a fixed number, so still $O(n)$. Finding an account: $O(1)$ on average in the hash table, then $O(\log n)$ in the number of accounts. For the Overview, n = orders and receipts — and each order's lines are added up (a loop inside a loop).

Each Time and Space is read from the comment above the function in the source, where every function in `js/` states its own cost.

6 Justification

Why did you choose that data structure or algorithm? Why is it appropriate?

Credentials in an array in id order: a new account takes the next id, so appending keeps the array sorted and binary search finds any account by id. Sign-in starts from an email, so a hash table email → id answers that in $O(1)$ on average instead of a linear search through every account.

Only a salted hash of each password is stored — never the password. Each account has its own salt, so equal passwords do not share a hash, and 200 rounds make guessing slower. Codes expire and are limited in tries. This keeps customers off the desk; real security needs a server (with `bcrypt` or `Argon2`), because anyone can read a browser-only site's code.

Every date in the system is "YYYY-MM-DD", so "is this in October 2026?" is "does it begin with 2026-10?" — one rule handles days, months and years, and a year is the sum of its months.

Reading each record's own day or month as its row number fills all 31 rows of a month in one pass, instead of scanning every order again for each row.

7 Demonstration

Show that the module works correctly — the input, the process and the output.

- 1 Open `index.html#admin`. Sign in with `kurlchester31feliciano@gmail.com` and `admin123`.
- 2 Enter the six-digit code (emailed; shown on screen while EmailJS is not set up).
- 3 The Overview opens on today. Switch to Monthly, then Yearly; use `<` `>` to move.
- 4 Open Accounts: the owner and Bea Cruz (staff). Sign out and sign in as `bea.cruz@example.com` with `Staff2026` — a demo address, so her code is shown on screen. She sees "My account" but no Logs tab.

Order desk

Staff sign-in.

Email

kurlchester31feliciano@gmail.com

Password

Continue

Forgot password?

Back to the shop

Step 1: email and password

Check your email

Enter the six-digit code to finish signing in.

Sign-in code

Email isn't connected yet (j@email-config.io), so the code is shown here instead.

6 6 9 9 2 9

Verify and sign in

Send a new code

Use a different account

Step 2: the six-digit code

Order desk

Shop owner - Owner

Sign out

Overview

Quote requests 2

Awaiting payment 1

In production 3

Completed 14

Voided 3

Products

Outbox

Logs 1 wrong password

Account

RIGHT NOW 2 to quote 1 awaiting payment 3 in production P003 balance owed

Daily

Monthly

Yearly

October 2026

Collected

P1,223

3 Cash payments

Sales

P1,625

Paid orders placed in this period

Orders placed

6

3 quote requests

Pieces sold

3

from paid orders

Made ready

1

Completed and released

Voided

1

PO returned

Day by day

DAY

ORDERS

PIECES SOLD

READY

COLLECTED

Thu 1

0

0

0

P200

Fri 2

0

0

1

0

Best sellers

Rose Bouquet

2 sold 2 orders

Picture Bouquet

1 sold 1 order

The Overview, monthly

Order desk

Shop owner - Owner

Sign out

Current password

New password

New password, again

8 to 64 characters, with at least one letter and one number, and no spaces.

Change password

Desk accounts

The credentials are kept in an array; a hash table finds an account by its email when someone signs in. Passwords are stored only as salted hashes. Accounts are disabled, never deleted, so their history stays. Use all data on this site, accounts and passwords changed here last until the page is released.

PERSON

ROLE

STATUS

LAST SIGN-IN

Shop owner

kurlchester31feliciano@gmail.com

Owner

Active

4 Oct 2026, 9:42 PM

You

Bea Cruz

bea.cruz@example.com

Staff

Active

Never

Disable

Add an account

Name

Email (the sign-in code goes here)

Role

Temporary password

Temporary password, again

Staff — Orders, products

Give the person the temporary password yourself; they sign in with it and the emailed code, then change it here.

Add account

Accounts: the credentials array

Trace — the accounts, the sign-in and the Overview (computed by our code)

staffAccounts — the credentials array, in id order

id	Name	Email	Role	Status	Salt	passHash
1	Shop owner	kurlchester31feliciano@gmail.com	owner	active	su-owner-2026	217525834
2	Bea Cruz	bea.cruz@example.com	staff	active	su-staff-2026	2345561029

staffEmailIndex (hash table email → id): "kurlchester31feliciano@gmail.com" → 1, "bea.cruz@example.com" → 2. No password is stored: passwordHash(su-owner-2026, "admin123") = 217525834 matches the owner's passHash; the same password with another salt gives 210248380.

deskSignIn() — step 1: accountByEmail(), then checkPassword()

Email / password	Account found	Result
"kurlchester31feliciano@gmail.com" / "wrong"	Shop owner (id 1)	That email or password is not right. 4 tries left.
"someone@gmail.com" / "guess1"	none	That email or password is not right. 4 tries left.
"someone@gmail.com" / "guess2"	none	That email or password is not right. 3 tries left.
"someone@gmail.com" / "guess3"	none	That email or password is not right. 2 tries left.
"someone@gmail.com" / "guess4"	none	That email or password is not right. 1 try left.
"someone@gmail.com" / "guess5"	none	Too many attempts. Sign-in for this email is paused for 30 seconds.
"bea.cruz@example.com" / "Staff2026"	Bea Cruz (id 2)	accepted → code step
" kurlchester31feliciano@gmail.com " / "admin123"	Shop owner (id 1)	accepted → code step

Five wrong passwords in a row paused only someone@gmail.com; the owner could still sign in. Spaces around an email are ignored.

issueOtp() and verifyOtp() — step 2

Action	Result
issueOtp()	code 053670 — only its hash 2677215748 is kept; expires in 5 minutes
verifyOtp("000000")	That code is not right. 2 tries left.
verifyOtp("053670")	signed in as Shop owner (deskUserId = 1)

addStaffAccount() — appended with the next id, indexed by email

Who adds	Result
Bea Cruz (staff)	Only an owner can add accounts.
Shop owner (owner): Lia Reyes	added with id 3
Shop owner (owner): LIA.reyes@gmail.com again	That email already has an account.

id	Name	Email	Role	Status	Salt	passHash
1	Shop owner	kurlchester31feliciano@gmail.com	owner	active	su-owner-2026	217525834
2	Bea Cruz	bea.cruz@example.com	staff	active	su-staff-2026	2345561029
3	Lia Reyes	lia.reyes@gmail.com	staff	active	su-566169235552	3709132585

accountByld(3): binarySearch looked at ids 2 → 3 ⇒ index 2.

setAccountActive() — disable, never delete

Action	Result
Disable Lia Reyes	disabled
Lia signs in with the right password	That email or password is not right. 4 tries left.
The owner disables their own account	You cannot disable your own account.

periodReport("month", "2026-09")

Figure	Value
Collected	₱10,522.50 (18 payments)
Sales	₱9,895.00
Orders placed	14
Pieces sold	13
Made ready	12
Voided	2 (₱235.00 retained)

Day by day (days with activity). All 30 day rows were filled by one tallyPeriods() pass: bucketOf() read each order's and receipt's date and returned its day number as the row index — 120 bucketOf() calls in all for the month view, instead of one scan of every order per day.

Day	Orders	Pieces	Ready	Collected
2026-09-01	0	0	1	₱592.50
2026-09-02	1	1	0	₱0.00
2026-09-03	0	0	1	₱375.00
2026-09-05	1	2	0	₱0.00
2026-09-06	0	0	1	₱1,115.00
2026-09-07	1	1	0	₱0.00
2026-09-08	0	0	1	₱750.00
2026-09-09	1	1	0	₱915.00
2026-09-10	0	0	2	₱0.00
2026-09-11	1	1	0	₱0.00
2026-09-12	0	0	0	₱1,050.00
2026-09-13	1	1	0	₱0.00
2026-09-14	0	0	0	₱870.00
2026-09-15	1	0	1	₱1,050.00
2026-09-16	0	0	1	₱0.00
2026-09-17	1	1	0	₱0.00
2026-09-18	0	0	0	₱310.00
2026-09-19	1	1	0	₱0.00
2026-09-20	0	0	0	₱1,210.00
2026-09-21	1	1	1	₱310.00
2026-09-22	0	0	0	₱650.00
2026-09-23	1	0	1	₱0.00
2026-09-24	0	0	1	₱235.00
2026-09-25	1	1	0	₱0.00
2026-09-26	0	0	0	₱455.00

Day	Orders	Pieces	Ready	Collected
2026-09-27	1	1	0	₱0.00
2026-09-28	0	0	0	₱435.00
2026-09-29	1	1	1	₱0.00
2026-09-30	0	0	0	₱200.00

periodReport("year", "2026") — a year is the sum of its months

Year collected: ₱11,810.00; sum of the 12 month rows: ₱11,810.00. Best sellers of the year: Rose Bouquet (7), Money Bouquet (3), Plumeria Bouquet (2), Dahlia Bouquet (1), Makeup Bouquet (1).

Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

js/accounts.js · lines 42–50 · passwordHash() — The stored form of a password.

```

42  /* The stored form of a password: FNV-1a over salt and password, repeated
43     a fixed number of rounds so guessing is slower.
44                                     Time O(n) - PASSWORD_ROUNDS is fixed · Space O(n) */
45  function passwordHash(salt, password) {
46      var p = String(password === undefined || password === null ? '' : password);
47  ❶  var h = fnv1a(salt + '|' + p);
48  ❷  for (var r = 1; r < PASSWORD_ROUNDS; r++) h = fnv1a(salt + '|' + h + '|' + p);
49      return h;
50  }
```

❶ line 47 Hash the account's salt with the password...

❷ line 48 ...and again, 200 rounds in all.

Time O(n) - PASSWORD_ROUNDS is fixed · Space O(n)

js/accounts.js · lines 79–84 · accountByEmail() — Finds an account from the email typed.

```

79  /* An account by email: one hash-table look-up, then the binary search.
80                                     Time O(1) average + O(log n) · Space O(n) */
81  function accountByEmail(email) {
82  ❶  var id = hashGet(staffEmailIndex, accountEmailKey(email));
83  ❷  return id === null ? null : accountById(id);
84  }
```

❶ line 82 The email's bucket gives the account id: O(1) on average...

❷ line 83 ...then binary search on the id-ordered array.

Time O(1) average + O(log n) · Space O(n)

js/accounts.js · lines 72–77 · accountById() — Binary search on the credentials array.

```

72  /* An account by id: binary search, the array is in id order.
73                                     Time O(log n) · Space O(1) */
74  function accountById(id) {
75  ❶  var at = binarySearch(staffAccounts, Number(id), function (a) { return a.id; });
76      return at === -1 ? null : staffAccounts[at];
77  }
```

❶ line 75 The array is in id order, so O(log n).

Time $O(\log n)$ · Space $O(1)$

js/admin-signin.js · lines 39–74 · deskSignIn() — Step 1: email and password.

```
39  /* Step 1. Returns '' when the code can be sent, or the message to show.
40     Never says which of the two was wrong, nor whether the account exists
41     or is disabled.                               Time  $O(n)$  · Space  $O(n)$  */
42  function deskSignIn(email, password, now) {
43      ❶ var guard = signInGuard(email), who = typedEmail(email);
44      ❷ if (now < guard.lockedUntil) {
45          logSecurity(now, 'locked-try', 'warn', who, 'Sign-in tried while this email is paused', '');
46          return 'Too many attempts. Try again in ' + Math.ceil((guard.lockedUntil - now) / 1000) + '
seconds.';
47      }
48      ❸ var account = accountByEmail(email);
49      ❹ if (account && account.active && checkPassword(account, password)) {
50          guard.streak = 0;
51          if (!canSendCode(account.email, now)) {
52              logSecurity(now, 'codes-max', 'warn', who, 'Password accepted for ' + account.name + ', but its
sign-in codes for this hour are used up', '');
53              return tooManyCodesMessage();
54          }
55          deskState.pendingId = account.id;
56          deskState.stage = 'otp';
57          deskState.codesSent = 0;
58          logSecurity(now, 'password-ok', 'info', who, 'Password accepted for ' + account.name + ' - sign-in
code step', '');
59          return '';
60      }
61      noteFailedSignIn(email, now);
62      if (guard.streak >= ADMIN_MAX_ATTEMPTS) {
63          guard.streak = 0;
64      ❺ guard.lockedUntil = now + ADMIN_LOCK_MS;
65          logSecurity(now, 'locked', 'alert', who, 'Sign-in paused for ' + (ADMIN_LOCK_MS / 1000) + ' seconds
after ' + ADMIN_MAX_ATTEMPTS +
66              ' wrong passwords in a row', '');
67          return 'Too many attempts. Sign-in for this email is paused for ' + (ADMIN_LOCK_MS / 1000) + '
seconds.';
68      }
69      var left = ADMIN_MAX_ATTEMPTS - guard.streak;
70      logSecurity(now, 'password-wrong', 'warn', who, 'Wrong email or password',
71          (account && !account.active ? 'The account is disabled. ' : (account ? '' : 'No account has
this email. ')) +
72          left + ' ' + plural(left, 'try', 'tries') + ' left');
73      return 'That email or password is not right. ' + left + ' ' + plural(left, 'try', 'tries') + ' left.';
74  }
```

- ❶ line 43 This email's sign-in record (module 13); the log keeps the email typed — never other text.
- ❷ line 44 ...paused? Say how long.
- ❸ line 48 Find the account.
- ❹ line 49 Active, and the salted hashes match: on to the code.
- ❺ line 64 Five misses in a row pause this email for 30 seconds.

Time $O(n)$ · Space $O(n)$

js/admin-signin.js · lines 76–85 · issueOtp() — Makes a code.

```
76  /* Make a fresh code and keep only its hash, and count it against the
77     account's hourly allowance. Returns the code so it can be emailed.
78                                     Time  $O(n)$  for the email · Space  $O(1)$  */
79  function issueOtp(now) {
80      ❶ var code = otpCode(), account = accountById(deskState.pendingId);
```

```

81 ② if (account) noteCodeSent(account.email, now);
82    deskState.codesSent = (deskState.codesSent || 0) + 1;
83 ③ deskState.otp = { hash: fnv1a(code), expires: now + OTP_LIFETIME_MS, tries: 0, sentAt: now };
84    return code;
85  }

```

① line 80 Six random digits from the browser's secure random source...

② line 81 ...counted against the address's codes for this hour.

③ line 83 Keep only its hash, its expiry and a try counter.

Time O(n) for the email · Space O(1)

js/admin-signin.js · lines 98–124 · verifyOtp() — Step 2: the code.

```

98  /* Step 2. Returns '' when signed in, or the message to show.
99                                Time O(n) · Space O(1) */
100 function verifyOtp(code, now) {
101     var otp = deskState.otp, account = accountById(deskState.pendingId), owner = account ? account.email :
102     '';
103     if (deskState.stage !== 'otp' || !otp) return 'Start again with your email and password.';
104 ① if (now > otp.expires) {
105     deskState.otp = null;
106     logSecurity(now, 'code-expired', 'warn', owner, 'Expired sign-in code used', '');
107     return 'That code has expired. Send a new one.';
108  }
109  if (otp.tries >= OTP_MAX_TRIES) return 'Too many wrong codes. Send a new one.';
110 ② if (fnv1a(digitsOnly(code)) === otp.hash && digitsOnly(code).length === OTP_LENGTH && account &&
    account.active) {
111     deskState.authed = true;
112     deskState.stage = 'password';
113     deskState.otp = null;
114 ③ deskUserId = account.id;
115     account.lastSignIn = now;
116     deskState.lastActive = now;
117     logSecurity(now, 'code-ok', 'info', owner, 'Signed in to the order desk as ' + account.name + ' ('
    + account.role + ')', '');
118     return '';
119  }
120 ④ otp.tries++;
121     var left = OTP_MAX_TRIES - otp.tries;
122     logSecurity(now, left > 0 ? 'code-wrong' : 'code-max', left > 0 ? 'warn' : 'alert', owner,
    left > 0 ? 'Wrong sign-in code' : 'Too many wrong sign-in codes', '');
123     return left > 0 ? 'That code is not right. ' + left + ' ' + plural(left, 'try', 'tries') + ' left.' :
    'Too many wrong codes. Send a new one.';
124 }

```

① line 103 Expired after five minutes.

② line 109 Right code...

③ line 113 ...and this account is the person signed in.

④ line 119 Wrong code: one try fewer (three in all).

Time O(n) · Space O(1)

js/accounts.js · lines 139–163 · addStaffAccount() — An owner adds a person.

```

139  /* An owner adds an account. The new person signs in with the temporary
140     password and the emailed code, then changes it.
141     draft: { name, email, role, password, confirm } Time O(n) · Space O(n)
142     Returns { ok, account } or { ok: false, error }. */
143  function addStaffAccount(draft, by, now) {
144 ① if (!isOwner(by)) return { ok: false, error: 'Only an owner can add accounts.' };
145     var name = strip(String(draft.name || '')), email = accountEmailKey(draft.email);

```

```

146     if (name.length < 2 || name.length > 60) return { ok: false, error: 'Enter the person\'s name (2 to 60
characters).' };
147     if (!isValidEmail(email) || isDemoAddress(email)) return { ok: false, error: 'Enter an email address
the person can read – the sign-in code goes there.' };
148 ❷    if (hashHas(staffEmailIndex, email)) return { ok: false, error: 'That email already has an account.' };
149     // The logs name people, so two accounts may not share a name.
150     if (firstWhere(staffAccounts, function (a) { return toLower(a.name) === toLower(name); })) {
151         return { ok: false, error: 'An account already has that name. Add a surname or an initial.' };
152     }
153     if (!firstWhere(ACCOUNT_ROLES, function (r) { return r.id === draft.role; })) return { ok: false,
error: 'Choose a role.' };
154     var problem = passwordProblem(draft.password, draft.confirm, null);
155     if (problem) return { ok: false, error: 'Temporary password: ' + problem };
156 ❸    var last = staffAccounts.length > 0 ? staffAccounts[staffAccounts.length - 1].id : 0;
157     var account = { id: last + 1, name: name, email: email, role: draft.role, salt: '', passHash: 0,
active: true, createdAt: now, lastSignIn: 0 };
158     setAccountPassword(account, draft.password);
159 ❹    listAdd(staffAccounts, account);
160 ❺    hashPut(staffEmailIndex, email, account.id);
161     logSecurity(now, 'account-added', 'warn', by.name, 'Account added: ' + name + ' (' + draft.role + ')',
email);
162     return { ok: true, account: account };
163 }

```

- ❶ line 144 Owners only.
- ❷ line 148 One account per email.
- ❸ line 156 The next id...
- ❹ line 159 ...appended: the array stays in id order.
- ❺ line 160 Index the email for sign-in.

Time $O(n)$ · Space $O(n)$

js/reports.js · lines 49–60 · `bucketOf()` — Which row a moment belongs to.

```

49  /* Which bucket a moment falls in: -1 outside the period; otherwise 0, or
50     with a breakdown the day of the month ('day') or the month of the year
51     ('month') minus one – the bucket's index, read straight off the date.
52                                     Time  $O(1)$  · Space  $O(1)$  */
53  function bucketOf(stamp, key, part) {
54      if (!(stamp > 0)) return -1;
55      var iso = isoFromStamp(stamp);
56 ❶    if (!beginsWith(iso, key)) return -1;
57 ❷    if (part === 'day') return readNumber(iso, 8, 10) - 1;
58 ❸    if (part === 'month') return readNumber(iso, 5, 7) - 1;
59      return 0;
60  }

```

- ❶ line 56 Outside the period: no row.
- ❷ line 57 Month view: the day of the month is the row.
- ❸ line 58 Year view: the month is the row.

Time $O(1)$ · Space $O(1)$

js/reports.js · lines 62–96 · `tallyPeriods()` — One pass fills every row.

```

62  /* Add every order and receipt into the bucket its date picks. One pass
63     fills every bar of the chart at once; nothing is read twice.
64                                     Time  $O(n^2)$  – the lines of every order · Space  $O(1)$  */
65  function tallyPeriods(buckets, key, part) {
66      for (var o = 0; o < orders.length; o++) {
67 ❶    var order = orders[o], at = bucketOf(order.createdAt, key, part);

```

```

68     if (at >= 0) {
69         var t = buckets[at];
70         t.placed++;
71         if (countWhere(order.items, function (item) { return item.kind === 'quote'; }) > 0)
t.quoteRequests++;
72         if (countsAsSale(order)) {
73             t.sales = roundMoney(t.sales + orderTotal(order));
74 ❷      t.units += sumRecursive(order.items, function (item) { return item.quantity; });
75         }
76     }
77     if (order.status === 'completed') {
78         at = bucketOf(order.completedAt, key, part);
79         if (at >= 0) buckets[at].completed++;
80     }
81     if (order.status === 'voided') {
82         at = bucketOf(order.voidedAt, key, part);
83         if (at >= 0) {
84             buckets[at].voided++;
85             buckets[at].retained = roundMoney(buckets[at].retained + order.amountPaid);
86         }
87     }
88 }
89 for (var r = 0; r < receipts.length; r++) {
90 ❸      at = bucketOf(receipts[r].stamp, key, part);
91     if (at >= 0) {
92         buckets[at].collected = roundMoney(buckets[at].collected + receipts[r].amount);
93         buckets[at].payments++;
94     }
95 }
96 }

```

❶ line 67 Each order goes straight to its row...

❷ line 74 ...adding its pieces if it was paid.

❸ line 90 Money collected: each receipt to its row.

Time $O(n^2)$ – the lines of every order · Space $O(1)$

js/reports.js · lines 106–133 · periodReport() — The whole Overview for a period.

```

106  /* Everything the Overview shows for one period, with a breakdown one
107     level down: the day's orders, the month's days, the year's months.
108     The breakdown's bars are filled by one tallyPeriods pass.
109                                     Time  $O(n^2)$  · Space  $O(n)$  */
110  function periodReport(kind, key) {
111      var rows = [], part = '';
112      if (kind === 'year') {
113 ❶      part = 'month';
114          for (var m = 1; m <= 12; m++) listAdd(rows, { key: key + '-' + twoDigits(m), label: MONTH_SHORT[m -
1], totals: emptyTotals() });
115      } else if (kind === 'month') {
116 ❷      part = 'day';
117          var days = daysInMonth(key);
118          for (var d = 1; d <= days; d++) listAdd(rows, { key: key + '-' + twoDigits(d), label: String(d),
totals: emptyTotals() });
119      }
120      if (rows.length > 0) {
121          var buckets = [];
122          for (var b = 0; b < rows.length; b++) listAdd(buckets, rows[b].totals);
123 ❸      tallyPeriods(buckets, key, part);
124      }
125      var placed = kind === 'day'
126      ? insertionSort(keepWhere(orders, function (o) { return inPeriod(o.createdAt, key); })), function
(a, b) { return a.createdAt - b.createdAt; })
127      : [];

```

```

128     return {
129         kind: kind, key: key, label: periodLabel(kind, key),
130         totals: periodTotals(key), rows: rows, orders: placed,
131         ④ best: topSellers(salesTally(function (o) { return inPeriod(o.createdAt, key); })), 5)
132     };
133 }

```

- ① line 113 A year: twelve month rows...
- ② line 116 ...a month: one row per day.
- ③ line 123 One pass over the orders and receipts fills them all.
- ④ line 131 The period's best sellers (module 1).

Time $O(n^2)$ · Space $O(n)$

Security & Audit Logs

Screens	Order desk → Logs tab; "Forgot password?" on the sign-in card
Files	js/audit.js (logEvent, logEntries, recentSecurityCounts, signInGuard, noteFailedSignIn) · js/admin-reset.js (startPasswordReset, completePasswordReset) · js/accounts.js (passwordProblem, setAccountPassword) · js/admin-signin.js · js/admin-logs.js
Data	auditLog — append-only array of { id, stamp, kind, code, level, actor, action, detail, ref }; failedSignIns — hash table email → { count, streak, lockedUntil }; each account's salted password hash and the reset code's hash
DSA	Append-only log · Hash table

1 Business process

What task or problem from the client are you solving?

The owner needs to know who did what to an order and when — a customer's checkout, a quotation, a payment recorded at the desk, a void — and to see attempts to get into the order desk: wrong passwords, wrong codes, lock-outs. Before, the only record was each order's own history, with no actor and nothing about signing in.

A forgotten password must not lock anyone out. "Forgot password?" emails a reset code to the account's own address and lets the person choose a new password — without telling a stranger which emails have desk accounts. The Logs tab is for owners only.

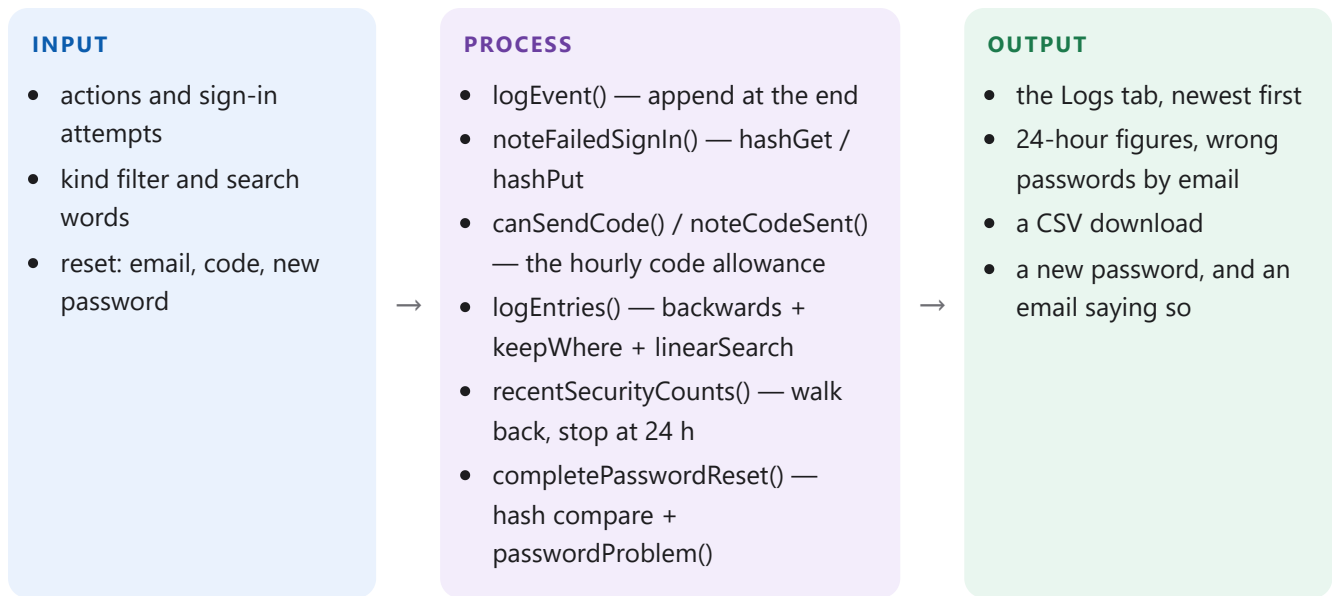
2 Input

What data is needed by the module?

Input	What it is
Every action	Order steps with their actor (Customer, the signed-in person's name, System), product changes, accounts added or disabled, emails that could not be sent.
Every sign-in step	The email typed; right or wrong passwords, codes, locks, resets, sign-outs; failed customer look-ups (the tracking number only).
Owner picks / types	Everything / Security / Orders & products; search words; Download CSV.
Person types (reset)	Their account's email, the emailed code, the new password twice.

3 Process

What data structure and/or algorithm did you use? How does it work step by step?



Step by step (Append-only log · Hash table):

- 1 Every event calls logEvent(), which appends one record at the end of the auditLog array — $O(1)$ however long the log is. Order steps go through addHistory(), so an order's own history and the audit log always agree, and each entry says who acted.
- 2 A wrong password also adds one to that email's row in the failedSignIns hash table — its total and its streak in a row (the email is hashed to its bucket: $O(1)$ on average). The fifth in a row pauses that email for 30 seconds; other people can still sign in. Text that is not an email address (a password typed in the wrong box) is kept only as "(not an email address)".
- 3 Code emails: a second hash table, codeSends, counts the codes each address was sent this hour. Sign-in and reset codes share it, and starting a sign-in or a reset again does not clear it, so nobody can flood an inbox — or use up the shop's monthly email allowance — by pressing Back and starting over.
- 4 Logs tab: the array is already in time order, so "newest first" is a read from the back — backwards(), $O(n)$, no sort. A kind filter keeps the matching entries; the search is a linear search with naive string matching over each entry's text.
- 5 24-hour summary: walk back from the newest entry and stop at the first one older than a day — only the recent end of the log is read.
- 6 Password reset: six random digits; only their FNV-1a hash is kept, with an expiry (10 minutes), a try counter (3) and a send counter (3). The new password is checked character by character — 8 to 64 characters, a letter and a digit, no spaces, not the current one — then that account gets a new salt and hash, any pause on its email is lifted, and the change is logged as an alert.

4 Output

What result does the module produce?

- The Logs tab: four 24-hour figures, the list (when, level, which log, who, what, the order), and "Wrong passwords by email".
- A CSV file of what is shown, safe to open in a spreadsheet (no typed text can run as a formula).
- The reset screens, the reset-code email (the code is never in the subject) and the "password changed" email to the owner.

5 Complexity analysis

What is the time complexity? What is the space complexity?

Operation	Time	Space	Note
<code>logEvent()</code>	$O(n)$ for the actor's text	$O(1)$	Appends one entry to the log.
<code>addHistory()</code>	$O(n)$ for the text	$O(1)$	Every order step lands in two places.
<code>codeBudget()</code>	$O(1)$ average + $O(n)$ for the text	$O(1)$	How many codes an address was sent this hour.
<code>signInGuard()</code>	$O(1)$ average + $O(n)$ for the text	$O(1)$	The sign-in record for an email.
<code>noteFailedSignIn()</code>	$O(1)$ average + $O(n)$ for the text	$O(1)$	Counts a wrong password against the email typed.
<code>logEntries()</code>	$O(n^2)$ – naive string matching on every entry	$O(n)$	The list on the Logs tab.
<code>recentSecurityCounts()</code>	$O(n)$	$O(1)$	The 24-hour figures.
<code>deskSignIn()</code>	$O(n)$	$O(n)$	Every attempt is logged.
<code>startPasswordReset()</code>	$O(n)$	$O(n)$	Step 1 of a reset.
<code>completePasswordReset()</code>	$O(n)$	$O(n)$	The code, then the new password.
<code>setAccountPassword()</code>	$O(n)$	$O(n)$	Stores a new password — as a hash only.
<code>passwordProblem()</code>	$O(n)$	$O(n)$	The password rules, by hand.

What n means here: For the list, the summary and the CSV, n = entries in the log; for the hash table, n = emails stored. Appending is $O(1)$ in the log's length (only the actor's text, at most 80 characters, is copied). For hashing and the password check, n = the characters typed.

Each Time and Space is read from the comment above the function in the source, where every function in `js/` states its own cost.

6 Justification

Why did you choose that data structure or algorithm? Why is it appropriate?

An append-only array is the natural shape of a log: events are only ever added at the end and never changed, so the array is in time order without sorting, appending is $O(1)$, and "newest first" is just reading it backwards — no $O(n^2)$ sort.

Because the log is in time order, the 24-hour summary stops at the first entry older than a day instead of reading the whole log.

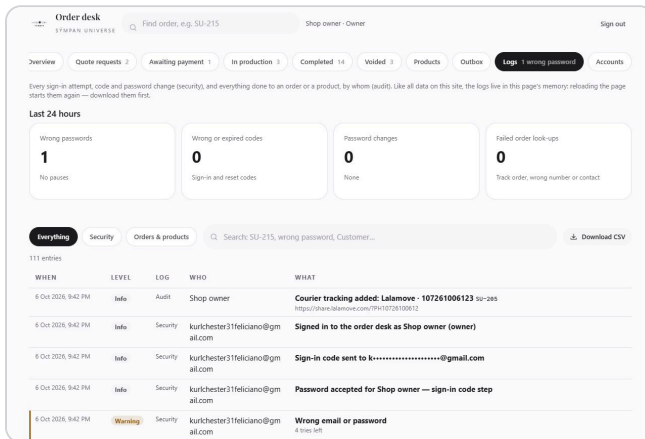
Counting wrong passwords (and codes sent) per email means "find this email's counter" on every attempt: a hash table answers that in $O(1)$ on average, where a list would need a linear search every time. Its keys are whatever people type, so the table doubles its buckets when it fills, keeping the chains short.

The reset keeps only a hash of the code, limits its time, tries and sends, and answers the same for any email, so a stranger can neither guess a code easily nor learn which emails have accounts. Like the rest of the site it runs in the browser: it keeps customers out, but real security needs a server.

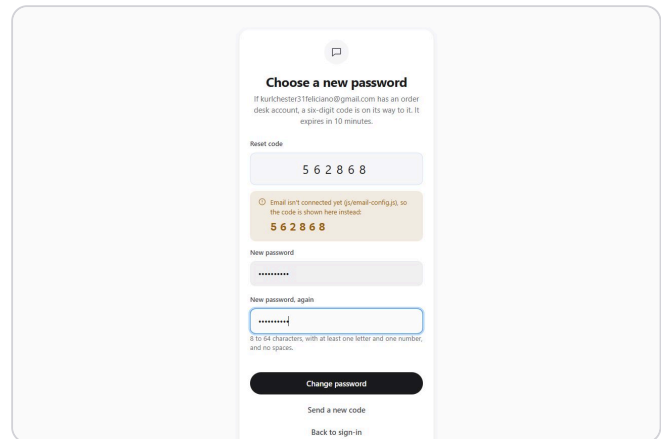
7 Demonstration

Show that the module works correctly — the input, the process and the output.

- 1 Open index.html#admin. Type one wrong password, then sign in properly with kurlchester31feliciano@gmail.com, admin123 and the code.
- 2 Open Logs: the wrong password, the code and the sign-in are at the top, and the 24-hour figures count them.
- 3 Type SU-217 in the search box: only that order's steps, each with who did it. Press Security: only security entries.
- 4 Sign out, press "Forgot password?", enter kurlchester31feliciano@gmail.com, then the code and a new password — and sign in with it. The reset is logged as an alert.



The Logs tab: the last 24 hours and the newest entries



Password reset: the code and a new password

Trace — the log, the summary and a password reset (computed by our code)

logEvent() — every event appended at the end

The demo history wrote 102 entries; three sign-in attempts, the code and one payment recorded at the desk added 6 more, numbered 103 to 108. The newest, read from the back:

#	Kind	Level	Who	What
108	audit	info	Shop owner	Balance of ₱200.00 paid by GCash · SU-217
107	security	info	kurlchester31feliciano@gmail.com	Signed in to the order desk as Shop owner (owner)
106	security	info	kurlchester31feliciano@gmail.com	Password accepted for Shop owner — sign-in code step
105	security	warn	stranger@gmail.com	Wrong email or password
104	security	warn	stranger@gmail.com	Wrong email or password
103	security	warn	kurlchester31feliciano@gmail.com	Wrong email or password
102	audit	warn	Customer	VOIDED – NON-REFUNDABLE · SU-222

noteFailedSignIn() — the failedSignIns hash table

Email typed	Wrong in all	In a row now	Has an account
stranger@gmail.com	2	2	no
kurlchester31feliciano@gmail.com	1	0	yes

The owner's right password set its streak back to 0; the total stays for the Logs tab.

logEntries() — backwards, keep one kind, then search

logEntries("security", "") keeps 5 of 108 entries. logEntries("all", "SU-217") finds 5, each with who acted:

#	Kind	Level	Who	What
108	audit	info	Shop owner	Balance of ₱200.00 paid by GCash · SU-217
86	audit	info	Customer	Quotation accepted — ₱200.00 paid by GCash · SU-217
85	audit	info	Order desk	Quotation sent: ₱400.00 · SU-217
84	audit	info	Customer	Quote request submitted · SU-217
83	audit	info	Customer	Agreed to the Privacy Notice (version 2026-10-06) and the order terms (version 2026-10-06) · SU-217

recentSecurityCounts() — the last 24 hours

Figure	Value
Security events	5
Wrong passwords	3
Pauses	0
Wrong or expired codes	0
Password changes	0
Failed order look-ups	0

The walk started at the newest entry and stopped after 15 of the 108 entries — the first one older than a day ends it.

A password reset

Call	Result
startPasswordReset("stranger@gmail.com")	no code made (known = false); the screen says the same as for a real account
startPasswordReset(owner)	code 352481 — only its hash 1697575728 is kept; expires in 10 minutes
completePasswordReset("000000", ...)	That code is not right. 2 tries left.
completePasswordReset(code, "short")	Use 8 to 64 characters.
completePasswordReset(code, "Ribbon2026")	changed

Afterwards checkPassword(owner, "Ribbon2026") = true and checkPassword(owner, "admin123") = false; the owner's salt is now su-047794175110. The last entry: "Password of Shop owner changed with a reset code" (alert).

csvField() — a typed text never runs as a spreadsheet formula

Text	In the CSV
=HYPERLINK("http://x")	"=HYPERLINK("http://x")"
Lara "LM" Mendoza	"Lara ""LM"" Mendoza"

Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

js/audit.js · lines 34–45 · `logEvent()` — Appends one entry to the log.

```
34  /* Append one entry. The log only ever grows at its end.
35                                     Time O(n) for the actor's text · Space O(1) */
36  function logEvent(stamp, e) {
37      var entry = {
38  ❶      id: counters.log, stamp: stamp, kind: e.kind, code: e.code || '', level: e.level || 'info',
39          actor: textCut(strip(String(e.actor || 'System')), LOG_ACTOR_MAX),
40          action: e.action, detail: e.detail || '', ref: e.ref || ''
41      };
42      counters.log++;
43  ❷      listAdd(auditLog, entry);
44      return entry;
45  }
```

❶ line 38 Build the entry with the next number...

❷ line 43 ...and add it at the end: $O(1)$, and the log stays in time order.

Time $O(n)$ for the actor's text · Space $O(1)$

js/orders.js · lines 32–39 · `addHistory()` — Every order step lands in two places.

```
32  /* A line in the order's own history – and the same event in the audit
33      log, with who did it: 'Customer', the person signed in to the order desk
34      (deskActor(), accounts.js) or 'System'.
35                                     Time O(n) for the text · Space O(1) */
36  function addHistory(order, stamp, text, actor, detail, level) {
37  ❶      listAdd(order.history, { stamp: stamp, text: text });
38  ❷      logAudit(stamp, actor || 'System', text, order.ref, detail, level);
39  }
```

❶ line 37 The order's own history...

❷ line 38 ...and the audit log, with who did it.

Time $O(n)$ for the text · Space $O(1)$

js/audit.js · lines 92–106 · `codeBudget()` — How many codes an address was sent this hour.

```
92  /* How many codes an address has been sent this hour, from the hash table.
93      Sign-in and reset codes share it, and starting either again does not
94      clear it, so nobody can flood an inbox (or the shop's monthly email
95      allowance) by pressing Back and starting over.
96                                     Time O(1) average + O(n) for the text · Space O(1) */
97  function codeBudget(email, now) {
98      var key = typedEmail(email);
99  ❶      var row = hashGet(codeSends, key);
100      if (row === null) {
101          row = { email: key, since: now, count: 0 };
102          hashPut(codeSends, key, row);
103      }
104  ❷      if (now - row.since >= CODE_SEND_WINDOW_MS) { row.since = now; row.count = 0; }
105      return row;
106  }
```

❶ line 99 The address's row, by its hash...

❷ line 104 ...starting a fresh count once the hour is over.

Time $O(1)$ average + $O(n)$ for the text · Space $O(1)$

js/audit.js · lines 68–80 · `signInGuard()` — The sign-in record for an email.

```
68  /* The sign-in record for an email that was typed – wrong passwords in
69     all (count), wrong ones in a row (streak) and when its pause ends – found
70     in the hash table in  $O(1)$  on average, or made.
71                                     Time  $O(1)$  average +  $O(n)$  for the text · Space  $O(1)$  */
72  function signInGuard(email) {
73      var key = typedEmail(email);
74      ❶ var row = hashGet(failedSignIns, key);
75      if (row === null) {
76      ❷   row = { email: key, count: 0, streak: 0, lockedUntil: 0, last: 0 };
77          hashPut(failedSignIns, key, row);
78      }
79      return row;
80  }
```

❶ Line 74 Find the email's row by its hash...

❷ Line 76 ...or make it the first time the email is typed.

Time $O(1)$ average + $O(n)$ for the text · Space $O(1)$

js/audit.js · lines 82–90 · `noteFailedSignIn()` — Counts a wrong password against the email typed.

```
82  /* One more wrong password for the email that was typed.
83                                     Time  $O(1)$  average +  $O(n)$  for the text · Space  $O(1)$  */
84  function noteFailedSignIn(email, stamp) {
85      var row = signInGuard(email);
86      ❶ row.count++;
87      ❷ row.streak++;
88      row.last = stamp;
89      return row;
90  }
```

❶ Line 86 One more in all...

❷ Line 87 ...and one more in a row (a right password resets it).

Time $O(1)$ average + $O(n)$ for the text · Space $O(1)$

js/audit.js · lines 136–146 · `logEntries()` — The list on the Logs tab.

```
136  /* The entries of one kind ('all', 'security' or 'audit') whose text has
137     every search word, newest first. The log is already in time order, so
138     reading it back to front is enough – no sort.
139                                     Time  $O(n^2)$  – naive string matching on every entry · Space  $O(n)$  */
140  function logEntries(kind, query) {
141      ❶ var newest = backwards(auditLog);
142      var ofKind = kind === 'security' || kind === 'audit'
143      ❷   ? keepWhere(newest, function (e) { return e.kind === kind; })
144         : newest;
145      ❸ return linearSearch(ofKind, query, logText);
146  }
```

❶ Line 141 Newest first: read the time-ordered log from the back.

❷ Line 143 Keep one kind, if one was picked.

❸ Line 145 Then the search words: linear search with naive string matching.

Time $O(n^2)$ – naive string matching on every entry · Space $O(n)$

```

148  /* Security events in the last 24 hours. The log is in time order, so the
149     walk starts at the newest entry and stops at the first one older than a
150     day instead of reading the whole log.  Time O(n) · Space O(1) */
151  function recentSecurityCounts(now) {
152     var c = { total: 0, failedPasswords: 0, locks: 0, failedCodes: 0, passwordChanges: 0, failedLookups: 0
153     };
154     for (var i = auditLog.length - 1; i >= 0 && now - auditLog[i].stamp <= DAY_MS; i--) {
155         var e = auditLog[i];
156         if (e.kind !== 'security') continue;
157         c.total++;
158         if (e.code === 'password-wrong' || e.code === 'locked-try' || e.code === 'current-wrong')
159             c.failedPasswords++;
160         else if (e.code === 'locked') c.locks++;
161         else if (e.code === 'code-wrong' || e.code === 'code-expired' || e.code === 'code-max' ||
162             e.code === 'reset-code-wrong' || e.code === 'reset-expired' || e.code === 'reset-max')
163             c.failedCodes++;
164         else if (e.code === 'password-changed') c.passwordChanges++;
165         else if (e.code === 'lookup-failed') c.failedLookups++;
166     }
167     return c;
168 }

```

❶ Line 153 Walk back from the newest entry; stop at the first one older than a day.

❷ Line 157 Count each event by its fixed code, not its wording.

Time O(n) · Space O(1)

```

39  /* Step 1. Returns '' when the code can be sent, or the message to show.
40     Never says which of the two was wrong, nor whether the account exists
41     or is disabled.  Time O(n) · Space O(n) */
42  function deskSignIn(email, password, now) {
43     var guard = signInGuard(email), who = typedEmail(email);
44     if (now < guard.lockedUntil) {
45         logSecurity(now, 'locked-try', 'warn', who, 'Sign-in tried while this email is paused', '');
46         return 'Too many attempts. Try again in ' + Math.ceil((guard.lockedUntil - now) / 1000) + '
seconds.';
47     }
48     var account = accountByEmail(email);
49     if (account && account.active && checkPassword(account, password)) {
50         guard.streak = 0;
51         if (!canSendCode(account.email, now)) {
52             logSecurity(now, 'codes-max', 'warn', who, 'Password accepted for ' + account.name + ', but its
sign-in codes for this hour are used up', '');
53             return tooManyCodesMessage();
54         }
55         deskState.pendingId = account.id;
56         deskState.stage = 'otp';
57         deskState.codesSent = 0;
58         logSecurity(now, 'password-ok', 'info', who, 'Password accepted for ' + account.name + ' - sign-in
code step', '');
59         return '';
60     }
61     noteFailedSignIn(email, now);
62     if (guard.streak >= ADMIN_MAX_ATTEMPTS) {
63         guard.streak = 0;
64         guard.lockedUntil = now + ADMIN_LOCK_MS;
65         logSecurity(now, 'locked', 'alert', who, 'Sign-in paused for ' + (ADMIN_LOCK_MS / 1000) + ' seconds
after ' + ADMIN_MAX_ATTEMPTS +
66             ' wrong passwords in a row', '');
67         return 'Too many attempts. Sign-in for this email is paused for ' + (ADMIN_LOCK_MS / 1000) + '
seconds.';

```

```

68     }
69     var left = ADMIN_MAX_ATTEMPTS - guard.streak;
70     logSecurity(now, 'password-wrong', 'warn', who, 'Wrong email or password',
71       (account && !account.active ? 'The account is disabled. ' : (account ? '' : 'No account has
this email. ')) +
72       left + ' ' + plural(left, 'try', 'tries') + ' left');
73     return 'That email or password is not right. ' + left + ' ' + plural(left, 'try', 'tries') + ' left.';
74   }

```

- ❶ Line 58 A right password: on to the code step.
- ❷ Line 61 A wrong one counts against the email typed...
- ❸ Line 65 ...and the fifth in a row pauses that email, logged as an alert.

Time $O(n)$ · Space $O(n)$

js/admin-reset.js · lines 47–63 · startPasswordReset() — Step 1 of a reset.

```

47  /* Step 1, once resetRequestProblem() allows it. Returns the code to email,
48     or '' when the email has no active account – the screen, the waits and
49     the hourly allowance behave the same either way. Time  $O(n)$  · Space  $O(n)$  */
50  function startPasswordReset(email, now) {
51    ❶ var account = accountByEmail(email), known = !!account && account.active;
52     deskState.reset = { email: typedEmail(email), known: known,
53       accountId: known ? account.id : 0, accountEmail: known ? account.email : '',
54       hash: 0, expires: now + RESET_LIFETIME_MS, tries: 0, sentAt: now, sends: 0 };
55     deskState.stage = 'reset-code';
56     if (!known) {
57       ❷ logSecurity(now, 'reset-unknown', 'warn', typedEmail(email), 'Password reset asked for an email
with no active account', '');
58       deskState.reset.sends = 1;
59       ❸ noteCodeSent(email, now);
60       return '';
61     }
62     ❹ return issueResetCode(now);
63   }

```

- ❶ Line 51 Is it the email of an active account?
- ❷ Line 57 If not: no code is made, but it is logged — and the screen says the same...
- ❸ Line 59 ...and it uses the hourly allowance just like a real code, so the limits tell nothing either.
- ❹ Line 62 If so: a six-digit code, kept only as a hash.

Time $O(n)$ · Space $O(n)$

js/admin-reset.js · lines 93–126 · completePasswordReset() — The code, then the new password.

```

93  /* Steps 2 and 3. The code is checked first and each wrong one uses a try;
94     a password that breaks the rules does not, so it can just be fixed.
95     Returns '' when the password changed. Time  $O(n)$  · Space  $O(n)$  */
96  function completePasswordReset(code, password, confirm, now) {
97     var r = deskState.reset;
98     if (deskState.stage !== 'reset-code' || !r) return 'Start again with your email.';
99     if (r.tries >= RESET_MAX_TRIES) return 'Too many wrong codes. Send a new one.';
100    ❶ if (now > r.expires) {
101       logSecurity(now, 'reset-expired', 'warn', r.email, 'Expired password reset code used', '');
102       return 'That code has expired. Send a new one.';
103     }
104     var digits = digitsOnly(code);
105    ❷ if (!r.known || digits.length !== OTP_LENGTH || fnv1a(digits) !== r.hash) {
106       r.tries++;
107       var left = RESET_MAX_TRIES - r.tries;
108       logSecurity(now, left > 0 ? 'reset-code-wrong' : 'reset-max', left > 0 ? 'warn' : 'alert', r.email,
109         left > 0 ? 'Wrong password reset code' : 'Too many wrong password reset codes', '');

```

```

110         return left > 0 ? 'That code is not right. ' + left + ' ' + plural(left, 'try', 'tries') + ' left.'
111             : 'Too many wrong codes. Send a new one.';
112     }
113     var account = accountById(r.accountId);
114     ❸ var problem = passwordProblem(password, confirm, account);
115     if (problem) return problem;
116     ❹ setAccountPassword(account, password);
117     var guard = signInGuard(account.email);
118     guard.streak = 0;
119     ❺ guard.lockedUntil = 0;
120     deskState.reset = null;
121     deskState.stage = 'password';
122     deskState.otp = null;
123     ❻ logSecurity(now, 'password-changed', 'alert', r.email, 'Password of ' + account.name + ' changed with a
    reset code',
124         'The new password lasts until the page is reloaded');
125     return '';
126 }

```

- ❶ Line 100 An expired code is refused — the same for every email.
- ❷ Line 105 Compare hashes; a wrong code uses one of three tries.
- ❸ Line 114 Then the password rules — a weak password uses no try.
- ❹ Line 116 A new salt and hash for that account...
- ❺ Line 119 ...any pause on its email is lifted...
- ❻ Line 123 ...and log it as an alert.

Time $O(n)$ · Space $O(n)$

js/accounts.js · lines 133–137 · `setAccountPassword()` — Stores a new password — as a hash only.

```

133  /* Give an account a new password with a fresh salt.  Time  $O(n)$  · Space  $O(n)$  */
134  function setAccountPassword(account, password) {
135     ❶ account.salt = makeSalt();
136     ❷ account.passHash = passwordHash(account.salt, password);
137 }

```

- ❶ Line 135 A fresh random salt...
- ❷ Line 136 ...and the salted hash; the password itself is never kept.

Time $O(n)$ · Space $O(n)$

js/accounts.js · lines 114–131 · `passwordProblem()` — The password rules, by hand.

```

114  /* A password the desk will accept: 8 to 64 characters, at least one letter
115     and one number, no spaces, typed the same twice, and – for an account
116     that has one – not the password it already uses. ' ' or what is wrong.
117                                     Time  $O(n)$  · Space  $O(n)$  */
118  function passwordProblem(password, confirm, account) {
119     var p = String(password === undefined || password === null ? '' : password), letters = 0, digits = 0;
120     ❶ if (p.length < PASSWORD_MIN || p.length > PASSWORD_MAX) return 'Use ' + PASSWORD_MIN + ' to ' +
    PASSWORD_MAX + ' characters.';
121     for (var i = 0; i < p.length; i++) {
122         var c = p.charAt(i);
123         if (isSpace(c)) return 'Leave out spaces.';
124         ❷ if (isDigit(c)) digits++;
125         else if (isLetter(c)) letters++;
126     }
127     ❸ if (letters === 0 || digits === 0) return 'Use at least one letter and one number.';
128     if (p !== String(confirm === undefined || confirm === null ? '' : confirm)) return 'The two passwords
    do not match.';
129     ❹ if (account && checkPassword(account, p)) return 'Choose a password different from the current one.';

```

```
130     return '';  
131 }
```

- ❶ line 120 8 to 64 characters.
- ❷ line 124 One pass counts the digits and the letters...
- ❸ line 127 ...and needs at least one of each.
- ❹ line 129 Not the password already in use.

Time $O(n)$ · Space $O(n)$

Complexity summary — all 13 modules

The main operation of each module, written with only the four notations from class: $O(1)$, $O(\log n)$, $O(n)$ and $O(n^2)$. Details are on each module's pages.

#	Module	Part	Main structure / algorithm	Time	Space
1	Catalogue & Best Sellers	Part 1	Hash-table tally · selection sort (stable)	$O(n^2)$	$O(n)$
2	Category Navigation	Part 1	N-ary tree · depth-first search with a stack	$O(n)$	$O(n)$
3	Search & Sort	Part 1	Linear search + naive string matching · insertion sort	$O(n^2)$	$O(n)$
4	Flower Customiser	Part 2	Hash table (FNV-1a, chaining) · recursive sum	$O(1)$ average	$O(n)$
5	Gift Customiser	Part 2	Greedy change-making	$O(n)$	$O(n)$
6	Cart	Part 2	Singly linked list	$O(1)$ append · $O(n)$ remove	$O(n)$
7	Checkout & Delivery	Part 3	Hash table (city → area) · character-scan validation	$O(1)$ average look-up · $O(n^2)$ form check	$O(n)$
8	Quotation Desk	Part 3	Circular queue · insertion sort of past quotes	$O(1)$ enqueue / dequeue · $O(n^2)$ draft	$O(n)$
9	Payment, Receipts & Email	Part 3	Circular queue (mail) · linear search for used references	$O(1)$ per email · $O(n)$ reference check	$O(n)$
10	Production & Tracking	Part 4	Min-heap · circular queue · binary search	$O(\log n)$ heap · $O(1)$ queue · $O(\log n)$ find	$O(n)$
11	Order Records	Part 4	Stack (undo) · insertion sort by date	$O(1)$ push / pop · $O(n)$ sort when nearly in order	$O(n)$
12	Order Desk & Accounts	Part 4	Credentials array + hash index · salted hash compare · one-pass bucketing by date	$O(n)$ sign-in · $O(n^2)$ report	$O(n)$
13	Security & Audit Logs	Part 4	Append-only log · hash table · linear search	$O(1)$ append · $O(n)$ summary · $O(n^2)$ search	$O(n)$

How we count

- Only the four notations from class: $O(1)$, $O(\log n)$, $O(n)$ and $O(n^2)$ — with "average", "amortised", "best" or "worst" where it matters, and sums such as $O(\log n) + O(n)$.
- n is the size of whatever the function goes through: records in a table (orders, lines, products, emails, receipts), items in a queue, or the characters of a text when the text itself is the job (validators, hashing, string search).
- A loop inside a loop is $O(n^2)$. Naive string matching over a table is $O(n^2)$.

- Work on one record's own fields — escaping or formatting its text, copying it — and look-ups in the shop's short fixed lists (20 colours, 2 arrangements, 4 add-ons, 4 time slots, 2 payment methods) count as constant work inside a loop over records: those lists are part of the program, not data that grows.
- An $O(\log n)$ step done once per item (a binary search per queued order) is written " $O(n^2)$ at most".

Reading Big-O in one minute

What it means

Big-O describes how the work grows when the data grows, ignoring fixed costs. $O(\log n)$ grows very slowly (double the data → one more step); $O(n)$ grows in step with the data; $O(n^2)$ grows with the square (double the data → four times the work).

Time vs. space

Time counts the steps (comparisons, moves). Space counts the extra memory a function needs besides its input — a sort that works on a copy is $O(n)$ space; a binary search is $O(1)$.

General questions — any member may be asked

Why is the whole website one HTML page?

Our data lives in JavaScript arrays, and arrays exist only while the page is open — they cannot be carried to another HTML file. So the shop and the order desk are sections of one `index.html`, shown and hidden; the part after `#` in the address picks the view. The arrays stay alive while you move between screens.

Where is the data saved? What happens when you refresh?

Only in memory (the arrays in `js/store.js`). There is no `localStorage`, cookie or server. Refreshing the page replays the demo history again — expected under the project rule, and it means every demonstration starts from the same state.

Why didn't you use push, sort, indexOf, filter and the other built-ins?

It was a project rule, and it shows we understand what they do. Each has our own version: `listAdd` (push), `copyRange` (slice), `positionIn` (indexOf), `keepWhere` (filter), `firstWhere` (find), `renderEach` (map + join), `insertionSort` (sort), `cutText` (split), `strip` (trim), `toLower` (toLowerCase) — and no regular expressions.

What does "procedural, not OOP" mean in your code?

There are no classes, no `this` and no methods. Data is plain records (a queue is `{ items, head, tail, count, capacity }`) and every operation is a separate function that receives the record: `cqEnqueue(queue, value)`, `stackPush(stack, value)`, `hashPut(table, key, value)`.

Why insertion and selection sort, and not merge sort or quick sort?

The course allows only $O(1)$, $O(\log n)$, $O(n)$ and $O(n^2)$; merge sort is $O(n \log n)$. Quick sort is not stable and its worst case is $O(n^2)$ anyway. Our lists are small and usually almost in order — orders, emails and receipts are appended as they happen — which is insertion sort's best case, $O(n)$. Selection sort ranks the ten best sellers with a fixed, easy-to-show cost.

Aren't $O(n^2)$ sorts too slow?

Not for our data: the lists hold tens to hundreds of records, and we read them in the order that leaves them nearly sorted. The Order Records trace shows it: 15 completed orders sorted newest first take 15 comparisons, not the worst case of 105.

Why hash tables, when you already have binary search?

Binary search needs an array sorted by the key you search for. The photographs are looked up by "flower|arrangement|colour" and the delivery areas by any spelling of a city name — keys that are not stored in order. A hash table answers those in $O(1)$ on average.

How does undo work?

It is a stack. Each "Mark ready" pushes the order id and its place in the production line. Undo pops the newest entry — last in, first out — and puts the order back exactly there.

How are emails sent without a server?

Every email is written into the outbox array and its position queued. Every 1.5 seconds the sender takes the front of the queue and posts it to EmailJS (a web service that sends email for a browser page). Demo customers (example.com) are never emailed. If EmailJS is not set up, the email is kept in the Outbox marked "not sent", and the sign-in code is shown on screen instead.

How secure is it?

It is a class project that runs entirely in the browser, so we are honest about the limits. Each account's password is kept only as a salted hash and the sign-in and reset codes only as FNV-1a hashes; codes expire and allow three tries; five wrong passwords in a row pause that email for 30 seconds; one address gets at most six codes an hour; an idle desk signs itself out after 15 minutes; staff cannot see the logs, other accounts or the accounts' emails; a reset answers the same for any email; every attempt is in the security log; the order desk is linked from nowhere on the shop; and everything a customer types is escaped before it is shown. A real deployment needs a server, a proper password hash (bcrypt or Argon2) and HTTPS, because anything in a browser can be read with developer tools — and a reset password lasts only until the page is reloaded, like all data here.

What about the customers' personal data?

The shop asks only for what an order needs: name, email, mobile, and an address for a delivery. A Data Privacy Notice under the Data Privacy Act of 2012 (RA 10173) says who collects it, why, who else receives it (couriers, the email service), how long it is kept and the customer's rights, including complaining to the National Privacy Commission. A banner points to it, checkout cannot finish until the customer ticks two separate boxes — the notice and the order terms — and the order records which versions were agreed to and when. The customer may withdraw consent by messaging the shop. The site sets no cookies and saves nothing on the device.

Where do the photos of the sweets bouquet and the beer-in-can cake come from?

The shop had not photographed them yet, so they show openly licensed photographs of similar gifts found online (Wikimedia Commons and Flickr, CC BY-SA). Each is labelled "Sample photo" and credited with its author, licence and source, with a note that it is not the shop's own work; the credits are also in `assets/products/CREDITS.md`.

How do you prove the complexities?

Each function states its cost above its code, and `tests/rubric.js` fails if one is missing or uses another notation. The unit tests count insertion sort's comparisons on sorted and reversed input ($n - 1$ and $n(n - 1)/2$). The traces in this reviewer were produced by running our actual functions on the demo data.

Did you use the Date object?

Only to read the clock: `readClock()` reads the time and the time zone, and `Date.now()` gives timestamps. All date maths — adding days, the weekday, month lengths, leap years, the day a timestamp falls on — is our own day-number arithmetic in `js/core.js`.

What would you improve with more time?

A real server and database so data survives a refresh; online payment through the payment provider instead of typed references; SMS updates; and a balanced tree or database index once the order list grows to many thousands.

Glossary

Term	Meaning
Array	Items in numbered slots (0, 1, 2 ...); reading slot i is $O(1)$. Our "tables".
Key	The field a structure is ordered or looked up by (order number, product id, city name ...).
Linear search	Check items one by one until a match. $O(n)$; works on unsorted data.
Binary search	On a sorted array, compare with the middle and keep one half. $O(\log n)$.
Naïve string matching	Try the search word at every position of a text, letter by letter. $O(n^2)$ over a table.
Insertion sort	Take each item and slide the larger ones before it one place right until its spot is found. $O(n)$ when already in order, $O(n^2)$ worst. Stable.
Selection sort	For each position, find the item that belongs there among the rest. Always $n(n-1)/2$ comparisons, $O(n^2)$.
Stable sort	Equal items keep their original order.
Linked list	Nodes that each point to the next. Adding at a known end is $O(1)$; removing re-points one link.
Stack (LIFO)	Last in, first out: push and pop at the top. Used for undo and the depth-first tree walks.
Queue (FIFO)	First in, first out: enqueue at the rear, dequeue at the front. Used for queue requests, standard production and email.
Circular buffer	A queue in a fixed array whose rear wraps round to the start, so nothing is shifted: $O(1)$ enqueue and dequeue.
Min-heap	A tree kept in an array where every parent comes before its children; the smallest key is always at index 0. Insert and extract are $O(\log n)$.
N-ary tree	Each node can have any number of children. Our category tree: a root, 3 branches, 10 lines.
Depth-first search	Visit a node, then go as deep as possible before backtracking — here with an explicit stack.
Hash table	A hash function turns a key into a bucket number, so a look-up checks one short list. $O(1)$ on average.
FNV-1a	Our string hash: start at 2166136261; for each character XOR it in, then multiply by 16777619 (kept to 32 bits).
Collision / chaining	Two keys in the same bucket; they are kept in a chain (a short list) inside that bucket.
Load factor	Keys $\div$ buckets ($160 \div 199 \approx 0.8$ for the photographs).
Recursion	A function that calls itself on a smaller problem, ending at a base case.
Greedy algorithm	Take the best choice at each step (the largest note that fits) without looking back.
Amortised	The average cost per operation over many operations — enqueue is $O(1)$ amortised although a rare grow is $O(n)$.
Append-only log	A list that only ever grows at its end, in the order things happen — so it is in time order without sorting. Our security and audit log.

Term	Meaning
Salt	Random text stored with an account and hashed together with its password, so equal passwords get different hashes.
Role	What an account may do: an owner sees everything, including the logs and the accounts; staff handle orders, production and products.
Consent	The customer's recorded agreement to the Privacy Notice — which version and when — asked separately from the order terms.
Account enumeration	Learning which accounts exist from how a page answers. The password reset answers the same for any email to prevent it.
Big-O	How the work grows with the size of the data (worst case unless stated).

File map — who explains what

Part	Rules and structures (js/)	Screens (js/)
1 · The shop window	store.js (salesTally, topSellers) · algorithms.js (selectionSortDesc, linearSearch, insertionSort) · structures.js (tree, stack) · core.js (textHas)	shop-catalog.js
2 · Choosing a gift	structures.js (hash table, linked list) · store.js (buildReferenceIndex, makeFlowerItem, makeQuoteItem, basket) · algorithms.js (sumRecursive, breakIntoBills)	shop-product.js · shop-basket.js
3 · Checkout and payment	store.js (lookupArea, deliveryQuote) · orders.js (validateRequest, quoteHistory, sendQuote, placeOrder, acceptQuote, issueReceipt) · notify.js · mail.js · data.js (PRIVACY_NOTICE, ORDER_TERMS)	shop-basket.js · shop-privacy.js · shop-track.js · admin-orders.js · receipt.js
4 · Behind the counter	structures.js (heap, circular queue, stack, hash table) · production.js (productionLine, completeNext, undoCompletion, completedByDate, voidOrder) · tracking.js · algorithms.js (binarySearch) · reports.js · accounts.js · audit.js · admin-signin.js · admin-reset.js	admin-desk.js · admin-overview.js · admin-account.js · admin-logs.js · admin-orders.js · admin-products.js · shop-track.js